

FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Traffic Forecasting in Madrid: An Explainable Deep Learning Approach

by

Shani Trikambhai Baladhiya

Under the supervision of

Prof. Dr. rer. nat. Matthias Wagner

Anahita Farhang Ghahfarokhi

A thesis submitted in partial fulfillment for the
degree of Masters in High Integrity Systems

in the

Faculty 2: Computer Science and Engineering

September 2026

Declaration of Authorship

I, Shani Trikambhai Baladhiya, declare that this thesis titled, 'Traffic Forecasting in Madrid: An Explainable Deep Learning Approach', and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date: September 27th, 2026

"All models are wrong, but some are useful."

George E. P. Box

FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Abstract

by Shani Trikambhai Baladhiya

M.Sc. High Integrity Systems

Prof. Dr. rer. nat. Matthias Wagner
Anahita Farhang Ghahfarokhi
Faculty 2: Computer Science and Engineering

The city of Madrid produces very large number of observations per hour using traffic sensors, but even with this amount of data, it is not easy to accurately forecast traffic at short time horizons, especially for streets with a diverse range of road infrastructure, such as residential roads, motorway ramps, ring motorway and underground tunnels. This thesis focuses on a stratified set of 200 sensors from January 2023 to May 2026, using a prediction problem of 6 hours ahead (t+1h - t+6h) with the same evaluation methodology and the same three deep learning architectures (CNN+LSTM, N-BEATS and the Temporal Fusion Transformer (TFT)). The overall performance of the three models is compared with the seasonal naive baseline, and TFT shows the best performance (MASE=0.5377) among them, closely followed by CNN+LSTM (MASE=0.5767) and N-BEATS (MASE=0.6133). The key takeaway is that none of the three models are able to meaningfully outperform naive performance on the M30 motorway sensors, highlighting the need for spatial modelling as a next step. Explainability analysis with the intrinsic variable selection proposed by TFT and GradientSHAP for CNN+LSTM shows that the distance to railway infrastructure is the most important static features for TFT, whereas CNN+LSTM depends mostly on the autoregressive traffic signal, which is a structural difference that accounts for a part of the performance gap between the two architectures.

Acknowledgements

First and foremost, I would like to thank my primary thesis adviser, Prof. Matthias Wagner, for his guidance, important feedback and constant support during the formulation of this thesis. His wisdom and guidance lay at the core of this work.

I would also like to thank Anahita Farhang Ghahfarokhi and Dr. Fatima Butt for their guidance, constructive feedback and support during my academic journey.

I would like to thank the Frankfurt University of Applied Sciences for its fostering of my personal and professional growth, the knowledge, experiences and academic surroundings that I have gained there.

I also would like to thank my dear friend Dipali Bhuva for her continuous support, encouragement and great help throughout my journey. I felt very lucky to have her with me.

Lastly, I would like to express my gratitude to my parents for their love, patience and encouragement during my time of master's studies. They are a steady inspiration for us and stay with us.

Contents

Declaration of Authorship i

Abstract iii

Acknowledgements iv

List of Figures x

List of Tables xi

List of Codes xiii

Abbreviations xiv

Symbols xvi

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 The Forecasting Problem . . . 2
- 1.3 Deep Learning for Traffic Forecasting . . . 2
- 1.4 The Explainability Gap . . . 3
- 1.5 Research Objectives . . . 4
  - 1.5.1 Objective 1 — Data preparation . . . 4
  - 1.5.2 Objective 2 — Multi-model forecasting . . . 4
  - 1.5.3 Objective 3 — Systematic comparison . . . 4
  - 1.5.4 Objective 4 — Explainability analysis . . . 4
- 1.6 Research Questions . . . 5
  - 1.6.1 Sub-question 1 . . . 5
  - 1.6.2 Sub-question 2 . . . 5
  - 1.6.3 Sub-question 3 . . . 5
- 1.7 Thesis Structure . . . 5

2 Literature Review 7
- 2.1 The Traffic Forecasting Problem . . . 7
- 2.2 Classical and Statistical Approaches . . . 7

2.3 Deep Learning Approaches . . . 8
2.3.1 Recurrent Architectures . . . 8
2.3.2 Convolutional and Hybrid Architectures . . . 9
2.3.3 Attention-Based and Transformer Architectures . . . 10
2.3.4 Graph Neural Networks for Spatial Traffic Forecasting . . . 11
2.4 Traffic Forecasting in Madrid . . . 12
2.5 Explainability in Deep Learning for Traffic Forecasting . . . 12
2.6 Research Gap and Positioning . . . 13
2.6.1 Gap 1 - Comparison of multi-architecture on Madrid data, controlled 14
2.6.2 Gap 2 - Six Horizons simultaneous forecasting with sub-class analysis 14
2.6.3 Gap 3 - Intrinsic and Post-hoc explainability . . . 14

3 Data 15
3.1 Data Sources . . . 15
3.1.1 Traffic Sensor Data . . . 15
3.1.2 Weather Data . . . 16
3.1.2.1 OpenStreetMap Road Attribute Data . . . 16
3.2 Road Classification . . . 18
3.2.1 Urban Sub-Classification . . . 19
3.2.2 Main Artery Identification . . . 20
3.3 Geographic Distribution of Sensors . . . 20
3.4 Data Splits . . . 21
3.5 Sensor Sample for Deep Learning Training . . . 22

4 Data Preparation and Preprocessing 23
4.1 Data Quality Assessment . . . 23
4.1.1 Error Flag Analysis . . . 23
4.1.2 Physically Impossible Readings Detection . . . 24
4.1.3 Detection of Stuck-Sensor Episodes . . . 25
4.2 Missing Value Imputation . . . 26
4.2.1 Overview of Missingness . . . 26
4.2.2 Imputation Approaches in the Literature . . . 26
4.2.3 Tiered Imputation Strategy . . . 27
4.2.3.1 Tier 1 — linear interpolation for gaps 24 hours or less . . 28
4.2.3.2 Tier 2 — Seasonal-naive fill for holes > 24 hours . . . 30
4.2.4 Fabrication cap . . . 32
4.2.5 Imputation Provenance . . . 32
4.3 Feature Engineering . . . 33
4.3.1 Overview . . . 33
4.3.2 Temporal Features . . . 34
4.3.3 Lag and Rolling Window Features . . . 35
4.3.4 Road Static Features . . . 36
4.3.5 Point of Interest Proximity Features . . . 36
4.3.6 Weather Features . . . 37
4.4 Prevention of Data Leakage . . . 38
4.4.1 Normalisation statistics . . . 38
4.4.2 Lag features near split boundaries . . . 38

4.4.3 Rolling means . . . 38
4.4.4 Seasonal fill reference profiles . . . 38
4.4.5 Dataset boundary assertions . . . 39
4.5 Final Dataset Characteristics . . . 39
4.6 Feature Set by Model . . . 39

5 Methodology **41**
5.1 Normalisation . . . 41
5.1.1 The Problem with Global and Per-Class Normalisation . . . 41
5.1.2 Per-Sensor Z-Score Normalisation . . . 42
5.1.3 Loss Function Robustness and Inverse Transformation . . . 45
5.1.4 Comparable Training and Validation Loss . . . 45
5.1.5 The Generalisation Ratio and Data Difficulty . . . 46
5.2 Model Architectures . . . 47
5.2.1 CNN+LSTM . . . 47
5.2.2 N-Beats . . . 51
Doubly Residual Topology. . . . 51
Block internal structure. . . . 52
Trend stack, from polynomial basis. . . . 53
A seasonal stack using the Fourier basis. . . . 53
Weight sharing. . . . 53
Univariate input. . . . 53
5.2.3 TFT . . . 54
Variable Selection Networks. . . . 54
Gated Residual Networks. . . . 55
Sequence-to-sequence LSTM encoder-decoder. . . . 56
Interpretable Multi-Head Attention. . . . 56
Input categories. . . . 56
5.3 Evaluation Metrics and Seasonal Naive Baseline . . . 57
5.3.1 Evaluation Metrics . . . 57
5.3.1.1 Mean Absolute Error (MAE) . . . 57
5.3.1.2 Mean Absolute Scaled Error (MASE) . . . 57
5.3.1.3 Root Mean Squared Error (RMSE) . . . 57
5.3.1.4 Coefficient of Determination (R^2) . . . 58
5.3.1.5 Symmetric Mean Absolute Percentage Error (SMAPE) . . . 58
5.3.2 The Seasonal Naive Baseline . . . 59
5.3.3 Per-Horizon Reporting . . . 60

6 Exploratory Data Analysis **61**
6.1 Purpose and Approach . . . 61
6.2 Traffic Intensity Distributions by Road Sub-Class . . . 61
6.3 Temporal Patterns . . . 62
6.3.1 The Diurnal Cycle . . . 62
6.4 Weekly Patterns . . . 64
6.5 Public Holiday Effects . . . 65

7 Experimental Results **66**

7.1 CNN+LSTM . . . 66
7.1.1 Context and Motivation . . . 66
7.1.2 Training Setup and Loss Function . . . 67
7.1.3 Training Behaviour . . . 69
7.1.4 Test Results . . . 69
7.2 N-Beats . . . 72
7.2.1 Context and Motivation . . . 72
7.2.2 Experimental Progression and Final Configuration . . . 73
7.2.3 Training Behaviour . . . 76
7.2.4 Test Results . . . 76
7.3 TFT . . . 79
7.3.1 Context and Motivation . . . 79
7.3.2 Experimental Progression and Final Configuration . . . 80
7.3.3 Training Behaviour . . . 82
7.3.4 Test Results . . . 83
7.4 Summary . . . 85

8 Model Comparison **87**
8.1 Framing the Comparison . . . 87
8.2 Overall Performance . . . 87
8.3 Per-Horizon Analysis . . . 88
8.4 Sub-Class Analysis . . . 89
8.4.1 URB_normal . . . 90
8.4.2 URB_ramp . . . 90
8.4.3 M30 . . . 90
8.4.4 C30 . . . 91
8.5 Peak and Off-Peak Analysis . . . 92
8.6 Training Efficiency vs Performance . . . 93
8.7 Limitations of This Comparison . . . 94
8.7.1 Encoder length asymmetry . . . 94
8.7.2 Training loss comparability . . . 94
8.7.3 Sample size . . . 95

9 Explainability Analysis **96**
9.1 TFT v3 — Intrinsic Explainability . . . 97
9.1.1 Internal Interpretability Mechanisms of TFT . . . 97
9.1.2 Static Covariate Importance . . . 97
9.1.3 Encoder Variable Importance . . . 98
9.1.4 Decoder Variable Importance . . . 99
9.1.5 Temporal Attention Weights . . . 100
9.1.6 Per Sub-Class Attention Analysis . . . 101
9.2 CNN+LSTM v2 — GradientSHAP Attribution . . . 103
9.2.1 Motivation for Post-Hoc Explainability . . . 103
9.2.2 Overall Feature Importance . . . 104
9.2.3 Per Sub-Class Analysis . . . 106
9.3 Cross-Model Feature Importance Comparison . . . 108
9.4 Summary . . . 111

10 Conclusion **113**
10.1 Overview of the Study 113
10.2 Principal Findings 114
10.3 Limitations of the Study 116
10.4 Directions for Future Research 117

Bibliography **120**

List of Figures

5.1 CNN + LSTM Architecture . . . 48
5.2 N-BEATS Architecture . . . 51
5.3 Temporal Fusion Transformer (TFT) Architecture . . . 54

6.1 Average hourly traffic intensity profiles by road sub-class and day type . . . 63
6.2 Weekly heatmap of average traffic intensity by road sub-class . . . 64
6.3 Average traffic intensity comparison between public holidays and regular days by road sub-class . . . 65

7.1 CNN+LSTM learning curves . . . 69
7.2 CNN+LSTM v2 per-horizon test performance . . . 70
7.3 CNN+LSTM v2 prediction snapshot across four forecasting horizons . . . 73
7.4 N-BEATS v7 learning curves . . . 76
7.5 N-BEATS v7 prediction snapshot across four forecasting horizons . . . 79
7.6 TFT v3 learning curves . . . 83
7.7 TFT v3 performance across forecasting horizons . . . 85
7.8 TFT v3 representative forecast across four horizons . . . 85

8.1 Per-horizon MASE comparison of the three models . . . 89
8.2 Training time versus overall MASE for the three models . . . 93

9.1 TFT v3 temporal attention weights across the 48-hour encoder window . . . 101
9.2 TFT v3 temporal attention profiles across the four road sub-classes . . . 102
9.3 TFT v3 per sub-class encoder variable importance . . . 103
9.4 CNN+LSTM v2 overall GradientSHAP feature importance . . . 107
9.5 CNN+LSTM v2 GradientSHAP feature importance by road sub-class . . . 108
9.6 TFT v3 and CNN+LSTM v2 feature importance comparison for the 18 shared encoder features . . . 112

List of Tables

3.1 Description of traffic dataset variables . . . 16
3.2 OpenStreetMap tags used to identify relevant points of interest . . . 18
3.3 Sensor distribution by road class . . . 18
3.4 Final road sub-classification . . . 20
3.5 OSM highway types among main artery sensors . . . 20

4.1 Stuck-sensor statistics by road class . . . 25
4.2 Missingness by road class after stuck-sensor reclassification . . . 26
4.3 Imputation approaches across studies using Madrid traffic data . . . 27
4.4 Data provenance after imputation by road class . . . 33
4.5 Final feature set (28 features) . . . 34
4.6 Final dataset summary (all sensors) . . . 39

6.1 Traffic intensity descriptive statistics by road sub-class (Veh/h) . . . 62

7.1 CNN+LSTM final configuration . . . 68
7.2 CNN+LSTM v2 per horizon test performance . . . 70
7.3 CNN+LSTM v2 per road sub-class test results . . . 71
7.4 CNN+LSTM v2 peak vs off-peak test results . . . 72
7.5 N-BEATS experimental progression (validation results) . . . 74
7.6 N-BEATS v7 final configuration . . . 75
7.7 N-BEATS v7 per-horizon test results . . . 77
7.8 N-BEATS v7 per road sub-class test results . . . 77
7.9 N-BEATS v7 peak vs off-peak test results . . . 78
7.10 TFT experimental versions . . . 80
7.11 TFT v3 final configuration . . . 81
7.12 TFT v3 per-horizon test results . . . 83
7.13 TFT v3 per road sub-class test results . . . 84
7.14 TFT v3 peak vs off-peak test results . . . 84
7.15 Three-model comparison on the overall test set . . . 85

8.1 Three-model comparison, overall test set results . . . 88
8.2 Per sub-class MASE comparison (test set) . . . 90
8.3 Peak vs off-peak MASE comparison (test set) . . . 92

9.1 TFT v3 static variable selection weights (top 7 of 14) . . . 97
9.2 TFT v3 encoder variable selection weights (top 10 of 19) . . . 98
9.3 TFT v3 decoder variable selection weights (all 11 features) . . . 100
9.4 Top 5 most attended encoder positions . . . 100

9.5 Attention received by each road sub-class 102
9.6 CNN+LSTM v2 GradientSHAP importance (all 29 features) 105
9.7 CNN+LSTM v2 top 5 GradientSHAP features by road sub-class 106
9.8 TFT v3 vs CNN+LSTM v2 importance comparison (18 shared encoder features) 109

List of Codes

3.1 Mapping traffic sensors to the nearest OpenStreetMap road edge 17
4.1 Reindexing sensor observations to a continuous hourly timeline 27
4.2 Computing the length of contiguous missing-data runs 29
4.3 Time-aware interpolation for short missing-data gaps 29
4.4 Seasonal median filling of remaining missing traffic observations 31
4.5 Adding the binary POI proximity flag 36
5.1 Per-sensor z-score normalisation and application to target and lag features 42
5.2 CNN-LSTM forecasting model architecture 50
7.1 Loss function, optimizer, learning-rate scheduler, and gradient clipping . . 67
7.2 N-BEATS optimizer and learning-rate scheduler configuration 74
7.3 TFT training dataset configuration 81
7.4 Temporal Fusion Transformer model configuration 82

Abbreviations

API	Application Programming Interface
ARIMA	Autoregressive Integrated Moving Average
BN	Batch Normalisation
C30	Tunnel road sub-class (Madrid classification)
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CTNS	Convolutional Temporal Network System
DCRNN	Diffusion Convolutional Recurrent Neural Network
DuckDB	In-process analytical database (Raasveldt and Mühleisen, 2019)
EDA	Exploratory Data Analysis
ELU	Exponential Linear Unit
FC	Fully Connected (layer)
GLU	Gated Linear Unit
GNN	Graph Neural Network
GPU	Graphics Processing Unit
GRN	Gated Residual Network
GRU	Gated Recurrent Unit
ICLR	International Conference on Learning Representations
LSTM	Long Short-Term Memory
M30	Madrid ring motorway road sub-class
MAE	Mean Absolute Error
MASE	Mean Absolute Scaled Error
MIMO	Multiple Input Multiple Output
N-BEATS	Neural Basis Expansion Analysis for Interpretable Time Series Forecasting
OSM	OpenStreetMap

OSMnx	OpenStreetMap for NetworkX (Boeing, 2017)
POI	Point of Interest
ReLU	Rectified Linear Unit
RMSE	Root Mean Squared Error
R^2	Coefficient of Determination
SARIMA	Seasonal Autoregressive Integrated Moving Average
SHAP	SHapley Additive exPlanations
SMAPE	Symmetric Mean Absolute Percentage Error
T-GCN	Temporal Graph Convolutional Network
TFT	Temporal Fusion Transformer
URB	Urban road class (Madrid classification)
URB_normal	Urban normal road sub-class
URB_ramp	Urban ramp (motorway connector) road sub-class
VRAM	Video Random Access Memory
VSN	Variable Selection Network
XAI	Explainable Artificial Intelligence

Symbols

α	Variable selection weight (softmax output)	[0, 1]
$\alpha^{(j)}$	Selection weight for the j-th input feature	[0, 1]
β	Batch normalisation learned shift parameter	$\mathbb{R}$
γ	Batch normalisation learned scale parameter	$\mathbb{R}$
ϵ	Small constant for numerical stability	$\mathbb{R}^+$ (e.g. 1e$-$8)
η	Learning rate	$\mathbb{R}^+$
λ	L2 weight decay regularisation coefficient	$\mathbb{R}^+$ (1e$-$4)
μ_i	Per-sensor training-period mean (normalisation)	veh/h
σ_i	Per-sensor training-period std. dev.	veh/h
$\sigma(\cdot)$	Sigmoid activation function	[0, 1]
$\odot$	Element-wise (Hadamard) multiplication	—
C	CNN filter output (convolutional activation)	$\mathbb{R}^{B\times 64\times 48}$
c_s	LSTM cell state at timestep s	$\mathbb{R}^{128}$
d_k	Attention key dimension	$\mathbb{R}$ (16)
F	Number of forecast horizons	$\mathbb{Z}^+$ (6)
f_s	LSTM forget gate at timestep s	$\mathbb{R}^{128} \in [0,1]$
g_s	LSTM candidate cell content at timestep s	$\mathbb{R}^{128}$
h_s	LSTM hidden state at timestep s	$\mathbb{R}^{128}$
h_{48}	Final LSTM hidden state (last encoder timestep)	$\mathbb{R}^{128}$
i_s	LSTM input gate at timestep s	$\mathbb{R}^{128} \in [0,1]$
K	Attention key matrix	$\mathbb{R}^{48\times d_k}$
k	CNN kernel (filter) size	$\mathbb{Z}^+$ (3)
N	Number of prediction windows in evaluation	$\mathbb{Z}^+$
o_s	LSTM output gate at timestep s	$\mathbb{R}^{128} \in [0,1]$

$P_{99.9}(i)$	99.9th percentile of sensor i training observations	veh/h
Q	Attention query matrix	$\mathbb{R}^{6\times d_k}$
R	Generalisation ratio = val_loss / train_loss_frozen	$\mathbb{R}^+$
R^2	Coefficient of Determination	$[-\infty, 1]$
t	Forecast origin timestep	hours
V	Attention value matrix	$\mathbb{R}^{48\times d_k}$
W	Learned weight matrix (generic)	$\mathbb{R}^{m\times n}$
$x_{i,t}$	Observed traffic intensity at sensor i, timestep t	veh/h
$\tilde{x}_{i,t}$	Normalised (z-score) traffic intensity	$\mathbb{R}$
X	Encoder input matrix	$\mathbb{R}^{48\times 29}$
$\hat{y}_{i,t}$	Predicted traffic intensity (physical units)	veh/h
$\tilde{y}_{i,t}$	Predicted traffic intensity (z-score space)	$\mathbb{R}$
$y_{i,t}$	Observed traffic intensity (physical units)	veh/h
$\bar{y}$	Mean of all observed values in evaluation set	veh/h
C_i	Clean training set for sensor i (below $P_{99.9}$)	—
T_{train}	Training period timestep set	—

Dedicated to my family, my teachers, and Dipali Bhuva, in gratitude for their unwavering support and inspiration...

Chapter 1

Introduction

1.1 Motivation

Road systems in cities are designed for much less vehicle traffic than is now occurring. Traffic congestion in Madrid is not a problem that happens in an occasional inconvenient way, rather it is a daily operation problem, as it is a metropolitan area with a population of 6.8 million, and about three million registered motor cars (Moral-Carcedo, 2024) [1]. The continuous monitoring of the road surface throughout the city is based on sensors that collect data every 15 minutes on the number of vehicles that pass them by, generating hundreds of millions of observations per year, but that's not enough to eliminate congestion. What traffic management needs is an accurate forecast of what will happen next, not an account of what has happened in the past, which is how many cars that will go into a particular intersection during the next hour, if the morning rush is expected to start earlier or later than usual, and if a tunnel will cause an increase in the number of cars on already congested surface streets.

The need is addressed in short-term traffic forecasting. Because the temporal patterns in historical sensor data are present in the data (such as the diurnal pattern of rush hour traffic, the weekly variation between Monday and Saturday and the slower seasonal variation in travel demand), a forecasting model can accurately predict future traffic intensities sufficiently to inform real operational decisions: adaptive signal timing, activation of variable speed limits, dynamic route guidance, and planning for incident response. The usefulness of such predictions critically depends upon two properties: The first is accuracy – the forecast must be close enough to the truth to be useful, i.e. it must be actionable. The second is reliability – in addition to knowing what the model predicted they must also understand why, so that means of failure are identified and predictions can be trusted.

Both properties are considered in this thesis. It trains and evaluates three deep learning architectures using three and a half years of real traffic sensor data from the city of Madrid, systematically compares their performance by road type, forecast horizon and time, and analyzes what each model learned on, i.e., what features it weighted, which historical patterns it oriented to, and whether the internal logic was physically consistent with the traffic flow in Madrid.

1.2 The Forecasting Problem

The traffic intensity (number of vehicles per hour at a fixed sensor site) is a time series characterized by a rich structure. It shows high diurnal variability, with peaks in the morning and evening, a midday plateau and a minimum during the night. It has weekly periodicity and weekday patterns are systematically different from weekend patterns. It has lower seasonality, which is related to the holidays, school terms and the weather. It is also interrupted from time to time by unpredictable events (incidents, road closures, special events), which are not known from the recent sensor history but are known from the context.

It is important for a forecasting model to reflect the regular structure as well as the irregular one. This type of structured time series was the basis for the development of classical statistical techniques like ARIMA and its seasonal extensions, which led to the first generation of operational traffic forecasting tools. They, however, model each sensor independently and make the assumption of linear relationships between past and future sensor data – two assumptions that restrict their accuracy in a sensor network where sensors can affect one another, as well as in a network where traffic dynamics are nonlinear at high densities. In the last decade, these have been solved by the transition to deep learning approaches, but they brought in the models whose inner workings are much more difficult to explain.

1.3 Deep Learning for Traffic Forecasting

Among the first deep learning architectures used for traffic forecasting were Recurrent Neural Networks (RNNs) and in particular Long Short-Term Memory networks (Hochreiter and Schmidhuber, 1997) [2] which superseded the statistical assumption of linearity with a learned representation of the temporal sequence that could be used to model nonlinear temporal relationships. Later research adopted both convolutional and recurrent layers to take advantage of both local temporal shifts and long-range sequential relationships (Asadi and Regan, 2020) [3]. More recently, attention-based architectures

that were originally proposed in Vaswani et al. (2017) [4] for natural language processing have been adapted to time series forecasting, and several models proved the advantages of learned, context-aware feature selection and temporal attention over fixed convolutional-recurrent processing in terms of measurable accuracy gains (Lim et al., 2021) [5].

In addition to these architectural changes, spatial models, which integrate graph-based representations of road networks for enabling sensors to share information throughout the network, have received a significant amount of research interest (Li et al., 2018 [6]; Zhao et al., 2019 [7]; Chauhan et al., 2025 [8]). The traffic intensity prediction task for the Madrid network was set by Vélez-Serrano et al. (2021) [9] who used residual convolutional neural networks to forecast the traffic intensity in 3400 sensors. Moral-Carcedo (2024) [1] used a neural network model that included route network topology to simulate temporal and spatial traffic patterns. These studies confirm the Madrid traffic network as both a well documented and scientific challenging environment for traffic forecasting studies.

1.4 The Explainability Gap

High predictive accuracy, but no explanations is challenging to apply. Some traffic decisions (e.g. the activation of the variable speed limit or closing of the tunnel) must be justified. A black-box model that is correct 80% of the time, but fails to predict why it was wrong when it was wrong, would be of little value to an operational network manager faced with a need to take action based on the model's predictions.

There are two general lines of research for tackling this gap: those in the field of explainable artificial intelligence (XAI). The interpretability is directly embedded in the model's architecture in intrinsic methods – one such example is the variable selection networks and attention mechanism with TFT, which have been specifically designed so that the model's feature weighting and temporal focus can be read out directly from the forward pass (Lim et al., 2021) [5]. Post-hoc methods involve applying external attribution techniques to trained models, namely the most popular and widely used SHAP (SHapley Additive exPlanations, Lundberg and Lee, 2017 [10]), which estimate the most important inputs that drive each prediction, without altering the model. While these methods have been used in traffic forecasting domains (Arrieta et al., 2020) [11], comparative studies comparing the contribution of different architectures to the same set of feature importances are not so abundant.

This thesis adds to this gap by applying both intrinsic (via selecting TFTs and paying attention to them) and post-hoc (via GradientSHAP for CNN+LSTM) explainability to models trained on the same data, and comparing the importance of different features ranked by each approach to determine whether they agree on which features are important and on what they disagree.

1.5 Research Objectives

The thesis has four goals, all of which are related:

1.5.1 Objective 1 — Data preparation

Create a consolidated analytical dataset from three public data sources for Madrid (traffic sensors, meteorological stations and OpenStreetMap road infrastructure), ensuring that it is systematically cleaned, completed and enhanced for the use of deep learning time series forecasting.

1.5.2 Objective 2 — Multi-model forecasting

With the same sensor sample and the same prediction task (simultaneous forecasting for 6 hours, with 48–72 hours of historical context), train three different architecturally distinct deep learning models (CNN+LSTM, N-BEATS and TFT) using the same normalisation pipeline and evaluation protocol.

1.5.3 Objective 3 — Systematic comparison

Compare the 3 models in terms of overall performance, degradation per horizon, road sub-class behaviour and temporal patterns to isolate the impact of each architectural choice on prediction accuracy: multivariate features, temporal attention, context-aware variable selection.

1.5.4 Objective 4 — Explainability analysis

Explain and interpret the two multivariate models using intrinsic and post-hoc explainability methods, determine which features each model did use, how the two methods agreed or disagreed with respect to these features and the physical structure of Madrid traffic, and relate the explainability results to the structure of the traffic network.

1.6 Research Questions

One major research question and three sub-questions are used for structuring this thesis.

The first research question is how well can the CNN+LSTM, N-BEATS and TFT models predict traffic intensity at multiple time horizons $t + 1h$ to $t + 6h$ concurrently, for the four sub-classes in the Madrid sensor network, compared to a seasonal naive baseline, using MAE, RMSE and MASE measures.

Three sub-questions further limit the investigation:

1.6.1 Sub-question 1

Does the accuracy of the forecast and patterns of errors vary significantly between road sub-classes and are models trained on the stratified global sample valid for a more heterogeneous mix of roads, such as residential urban streets, motorway ramp, M-30 ring motorway and underground tunnel sections?

1.6.2 Sub-question 2

The accuracy of the models over different traffic cycles, daily and weekly, and if these patterns are consistent across all three architectures (as the morning peak, evening peak, weekend and off-peak weekday periods).

1.6.3 Sub-question 3

What are the features that are most important for traffic intensity predictions, according to TFT's intrinsic Variable Selection Networks and multi-head attention mechanism, and also according to the CNN+LSTM-based GradientSHAP post-hoc analysis, and whether there are differences in the feature importance ranking based on road sub-class or between the two explainability methods.

1.7 Thesis Structure

The literature is reviewed in Chapter 2, with a focus on classical and deep learning methods for traffic forecasting, methods to provide explainability for neural network solutions and previous work on traffic in the Madrid network. The three sources of data and the process of creating the analytical data set are described in Chapter 3.

The preprocessing and feature engineering is described in Chapter 4 and normalisation, different architectures of deep learning models and evaluation methodology is presented in Chapter 5. The different properties of the data set are explored in Chapter 6. The experimental result of three models are reported separately in chapter 7. The models are discussed systematically in Chapter 8. The explainability analysis is provided in Chapter 9. Chapter 10 ends with a summary of findings, limitations and directions for future work.

Chapter 2

Literature Review

2.1 The Traffic Forecasting Problem

Traffic forecasting is the prediction of future traffic conditions (usually flow, speed or density) at one or more locations based on past observations and context. This is a time series forecasting problem and has three important features. Simple models can capture simple patterns of regular travel (commuting, school schedules, weekend), which are periodic. But events, weather, and incidents can change these patterns in ways that history of just the sensors can't predict. In addition, there are spatial interdependencies between the sensors that can affect congestion propagation between locations and make single-sensor models difficult if spatial dependence is a factor.

Important also is the forecasting horizon. There is strong traffic auto-correlation, meaning that the current intensity can be used very effectively to forecast the near future, for short term forecasting (up to one hour ahead). As the length of the time series decreases, medium term forecasting (one to six hours) becomes more reliant on broader temporal patterns. The present thesis is based on simultaneous six-horizon ($t + 1h$ to $t + 6h$) forecasting that lies between short-range and medium-range forecasting. This greatly increases the sensitivity of the task to the capacity of a model to utilize future-known contextual information, such as day of week and hour of day, on each forecast horizon.

2.2 Classical and Statistical Approaches

The earliest formalised traffic forecasting methods were based on classical time series analysis. The ARIMA family used the linear lagged model with a linear combination of past values and errors, and the seasonal extensions (SARIMA) explicitly included the

daily and weekly periodicities by using seasonal differencing. For the above reasons, these models continue to be relevant as benchmarks, and some studies have found competitive results for a single sensor, short horizon forecasts under stationary traffic conditions (Vélez-Serrano et al., 2021) [9].

Their drawbacks are apparent in three respects, however. First, traffic is nonlinear at high-density flows and linear models do not capture the relationship between flow out and traffic at a traffic bottleneck (Daganzo, 1994) [12]. Second, ARIMA models each sensor individually, and thus fail to incorporate spatial correlations that flow traffic patterns across the network. Thirdly, they have a seasonal structure that is more rigid than can be adapted to changing travel patterns that may be related to urbanisation, remote working or modal shifts. In the Madrid network, Moral-Carcedo (2024) [1] demonstrated that the use of route network topology significantly improves the traffic intensity prediction beyond temporal-only models, indicating the need for complementary spatial information.

In the 2000s (Wu et al., 2004 [13]), Support Vector Machines and Support Vector Regression were developed as nonlinear models of the ARIMA-type that could be used instead of these models to capture more complex input-output relationships. While they were accurate with individual sensors, they were not well scaled for large sensor networks and needed meticulous feature engineering and kernel selection. Since then, they have been phased out as deep learning techniques have proved to perform at much higher levels of performance at similar scale (Deng et al., 2019) [14].

2.3 Deep Learning Approaches

2.3.1 Recurrent Architectures

The first application of deep learning to traffic forecasting was by recurrent neural networks, such as Long Short-Term Memory (LSTM) networks (Hochreiter and Schmidhuber, 1997) [2]. The LSTM model was one of the first models to gain prominence in the field of early deep traffic forecasting by capturing long-term historical information and eliminating the vanishing gradient problem of older recurrent architectures. LSTM was superior at all benchmarks of traffic data on the standard test set (Gu et al., 2026 [15]; Wang et al., 2021 [16]). They were also able to process multivariate inputs, which allowed them to integrate weather, calendar and lagged observations into the traffic sequences.

One weakness of a single layer LSTM is that it sits on a single size hidden state, which may result in the gradual loss of information from earlier timesteps. Hence, observations taken several days prior can also be poorly remembered over long periods of history. Hierarchical temporal representations are given by stacked LSTM and encoder-decoder architectures (Wang et al., 2021 [16]) but they are still not a complete solution to this information-compression problem. This limitation is overcome by attention mechanisms, as explained in Section 2.3.3, where the decoder can learn weights for accessing any historical position.

Bidirectional LSTMs are a variant of LSTMs which process both forwards and backwards in the sequence, allowing for the past and future context in the encoder window to be captured by each hidden state. Chauhan et al. (2025) [8] introduced TimeAttentionBiLSTM to address long-term traffic forecasting on the Madrid dataset, which involved bidirectional LSTM encoding and attention and GRU decoding. CNN-BiLSTM baselines recorded MAEs of 14–18%, exceeding the performance of the model for 12-hour forecasts. This work is pertinent for this thesis due to the use of the same sensor network in Madrid and the use of attention mechanisms for longer horizon forecasting.

2.3.2 Convolutional and Hybrid Architectures

One dimensional convolutional neural networks offer an alternative way of temporal modelling. While LSTMs work in a step-by-step manner, 1D CNNs use learned filters across local temporal windows at the same time, thus recognizing sequences without sequential processing. This makes them efficient and effective for spotting localized patterns, like a sudden start to a morning rush or an eventual ramp-up of a rush hour in the evening.

Simple convolutional models, however, have a good representation of local structure and are not efficient at capturing long-range dependencies without deep stacking. One natural approach is to fuse CNN layers with sequential modelling layers such as LSTM or GRU. This method was validated by Asadi and Regan (2020) [3] for a spatiotemporal traffic forecasting system, where CNN+LSTM improved the accuracy of the forecasting, as compared to CNN or LSTM alone. This thesis builds on their architecture to create the CNN+LSTM model specifically for six-horizon MIMO forecasting and per-sensor z-score normalisation.

For traffic flow prediction on 3,400 sensors in Madrid, Vélez-Serrano et al. (2021) [9] were able to utilize deeper architectures without losing information by implementing residual convolutional neural networks, with skip connections to pass information between layers. They reported test MAE of 37.92 veh/h for one-step-ahead prediction for their ResNet,

which was significantly higher than the performance of ARIMA. Although not a direct comparison, since different sensors, period, horizon and prediction steps were used, the result shows a performance of the convolutional deep learning on the Madrid network.

Deng et al. (2019) [14] proposed a random subspace convolutional neural network which jointly modelled the spatial and temporal relations by two dimensional convolution on the sensor network. Their approach was to model sensors as a spatial structure that is similar to an image, which resulted in 15–25% accuracy gains over methods that considered spatial and temporal dimensions independently. This is done without using a graph representation, but is an approach that foreshadows the graph representation methods to come in the next section.

Priya et al., (2025) [17] introduced the Convolutional Temporal Network System (CTNS), using a combination of CNN and LSTM layers for urban spatio-temporal traffic prediction. They were evaluated on urban datasets and showed that the hybrid architecture had higher accuracy than CNN and LSTM models, further confirming the empirical result that the hybrid architecture of local pattern extraction with sequential memory consistently outperforms the single CNN or LSTM model.

2.3.3 Attention-Based and Transformer Architectures

Transformer architecture (Vaswani et al., 2017) [4] was developed to address the recurrent sequence processing in natural language processing that was previously the standard method. In contrast to processing tokens serially and passing information around through a hidden state, attention enables each position to focus directly on all other positions via learned similarity based weights. This alleviates the problem of information compression in LSTMs and allows for parallel sequence processing while training.

Handling temporal properties like positional information, multi-step prediction and multivariate inputs of different types and scales are the challenges when using Transformers for time series forecasting. A complete adaptation of the Transformer architecture for multi-horizon time series forecasting is the Temporal Fusion Transformer (TFT) (Lim et al., 2021) [5], which is the main reference in this thesis. It distinguishes between static covariates, time varying known inputs and time varying unknown inputs. It has a novel architecture that integrates Variable Selection Networks for dynamically selecting which features to use, Gated Residual Networks to eliminate unnecessary computations, and interpretable multi-head attention to compute temporal importance weights. TFT successfully outperformed LSTM-based baselines on a variety of real world datasets, such as traffic, and offered interpretable representations. The accuracy and explainability of this combination were the reason for its selection for this thesis.

The most similar study is performed by Chauhan et al. (2025) [8] which uses the same sensor network, but predicts sensor readings at longer time horizons with the help of TimeAttentionBiLSTM. Their study, however, did not consider the use of TFT and/or SHAP-based explainability that are considered unique contributions of the present study.

2.3.4 Graph Neural Networks for Spatial Traffic Forecasting

One of the critical innovations in traffic forecasting is the incorporation of spatial dependencies between sensor networks into the model, enabled by the use of graph neural networks (GNNs). A road network can be naturally modeled as a graph, whose vertices are the sensors, where an edge represents a road link between two sensors, and where a sensor can exchange messages with its neighboring sensors. Li et al. (2018) [6] introduced Diffusion Convolutional Recurrent Neural Network (DCRNN), which integrates graph convolution based on the traffic diffusion with LSTM based temporal modelling and achieved significant gains over the non-spatial approaches on the California highway traffic data. Zhao et al. (2019) proposed T-GCN to use temporal graph convolution in combination with GRU to predict traffic speeds.

For non-graph-structured traffic data, where the topology of the road network is not available or not suitable, Rahmati Nia et al. (2025) introduced a context-aware CNN-GRU-Attention model. They do not rely on explicit spatial graphs, and their model shows that attention-based architectures can hold their own without such graphs. It was able to improve over the next-best baseline by 18% on the Madrid dataset with 12-hour granularity.

When spatial propagation is large, such as on motorways and arterial roads, congestion propagates upstream and downstream and graph-based approaches tend to work well. They do need road topology information, though, and add implementation complexity and quadratic growth with sensor count if no suitable approximations are used. This thesis is concerned with comparing the temporal forecasting architectures and their interpretability, so the implementation of graph-based models was not attempted. All three temporal models fail to significantly outperform the seasonal naive baseline significantly in the M30 case, as concluded in Chapter 7, indicating spatial modelling is a natural extension for this sub-class.

2.4 Traffic Forecasting in Madrid

Madrid Open Data Portal has been releasing open data about traffic since 2013. In order to compare different architectures for one hour ahead prediction, Vélez-Serrano

et al. (2021) [9] used 2016–2019 data from 3,400 sensors and compared three different architectures: ARIMA, LSTM and residual CNN. Their ResNet model outperformed the ARIMA model (MAE=40.11) and was competitive with the LSTM model on the test set of 2019, with MAE=37.92 veh/h. In the current study, the feature importance analysis revealed that the weekly autocorrelation is the leading seasonal pattern, which is also observed from the Kruskal-Wallis test.

In a previous study, Moral-Carcedo (2024) [1] studied the same network spatially by adding road network topology to a neural network that inferred the intensity of route use from the observed data from the sensors and the origin-destination data. The study demonstrated the capacity of modelling the structure of the road network to replicate temporal and spatial traffic-intensity patterns and to evaluate the impact of the Madrid Central Low Emission Zone. This illustrates the fact that spatial information is not available to the temporal-only models used throughout this thesis.

Gómez and Ilarri (2025) [18] released their large-scale, high-resolution dataset of traffic in the city of Madrid, accompanied by detailed documentation of the preprocessing of the data, serving as a methodological reference for dealing with data quality. They have a completeness-based sensor selection that retained sensors with at least 90% valid observations, which is similar to the filtering done in this thesis, but they used an inclusion threshold instead of explicit imputation and then excluding sensors.

Chauhan et al. (2025) [8] tested their TimeAttentionBiLSTM architecture on the Madrid dataset, with 12h, 24h, 48h and 72h prediction horizons, and achieved state of the art results across various metrics when compared with nine baseline methods. Their application with the same data set offers the most direct recent comparison for the present study, but there are differences in prediction horizon, sensors used, and evaluation procedures that prevent direct numerical comparison.

2.5 Explainability in Deep Learning for Traffic Forecasting

Explainability methods for deep learning traffic models are a fairly new research field. Arrieta et al. (2020) [11] have proposed a detailed taxonomy of the XAI approaches that categorizes them into two groups: intrinsic and post hoc. Arrieta et al. (2020) [11] have outlined an extensive taxonomy of XAI approaches, which can be split into two categories, namely intrinsic and post hoc approaches. They suggest using intrinsic methods when there are such, and using post hoc approximations when they are not.

SHAP (Lundberg and Lee, 2017) [10] aggregated a number of existing feature attribution techniques in a single theoretical framework based on Shapley values from the game

theory cooperative framework. An important characteristic of SHAP values is that they are consistent: a feature that is most important for the output in any subset of features will be assigned the highest SHAP value in that subset. This allows for the ability to compare two sets of feature importance values across different models and features, which is done in Chapter 8. If a model is differentiable, GradientSHAP offers an efficient gradient-based approximation that makes use of the fact that analytical gradients are available.

Joseph (2022) [26] tackles the practical problem for sequence models: the raw SHAP output comes in a shape of thousands of attribution values per prediction (windows $\times$ timesteps $\times$ features). The preferred way is to combine (sum) attributions over time first and then analyze the resulting set of attributions; only one importance score per feature is returned (as is done in this thesis for CNN+LSTM). Otherwise, if this aggregation is not performed, the explanation of a sequence model SHAP values needs to be done at every time step which is informative but tedious, or alternatively, to calculate attention-like "averages" where the temporal and feature-level importance are mixed together.

Unlike attribution methods, TFT's built-in Variable Selection Networks and attention mechanism give a direct insight into the model's decision process. A key advantage of these variable selection weights and attention weights is that they are actual quantities computed as explicit intermediate values during the forward step of each training iteration, not constructed via some form of post hoc approximation. In their original work, Lim et al. (2021) [5] have shown the interpretable nature of TFT's attention weights, which are found to assign attention to the same positions in the lag space as domain knowledge would suggest (one day and one week back). The present thesis validates this result for the Madrid network: for TFT, the dominant attention position is $t-23h$ (yesterday's same hour), and for CNN+LSTM, the dominant position is the GradientSHAP value of the shared features with TFT.

2.6 Research Gap and Positioning

The above-mentioned literature indicated three gaps which will be addressed in this thesis.

2.6.1 Gap 1 - Comparison of multi-architecture on Madrid data, controlled

There are difficulties in making cross-study comparison because previous studies of the Madrid network have employed various sensors, covered different time periods, and different prediction horizons and used different evaluation protocols. This thesis is the first systematic comparison of the Madrid network, which isolates architectural contribution from data confounds, by training three architecturally different models on the same data, with the same normalisation, the same 200-sensor sample, and the same 6-horizon evaluation protocol.

2.6.2 Gap 2 - Six Horizons simultaneous forecasting with sub-class analysis

The majority of studies that address prediction specific to Madrid are either one-step-ahead or very long horizon (12–72h) prediction. The range of six horizons simultaneous forecasting ($t + 1h$ to $t + 6h$) is practically relevant but not widely studied, and the detailed analysis of the sub classes (individually in URB_normal, URB_ramp, M30 and C30) gives insights beyond the sum of the individual accuracy figures.

2.6.3 Gap 3 - Intrinsic and Post-hoc explainability

We are not aware of any previous work that uses both the intrinsic explainability mechanisms of TFT and SHAP-based attribution to models trained on the same traffic network data, and then compares the resulting rankings of feature importance and relates this to the physical structure of the traffic network. The paired analysis is the main methodological innovation of this thesis that goes beyond the forecasting experiments.

Chapter 3

Data

3.1 Data Sources

This study is based on three data sets that are publicly available from the Madrid Open Data Portal (datos.madrid.es), hosted by the Madrid City Council. The portal uses a software standard called CKAN (Comprehensive Knowledge Archive Network) — an open-source data management system widely adopted by government agencies to publish structured datasets. All relevant data resources are accessible from the portal in a structured and machine-readable form via a CKAN-based API, which was used programmatically to discover and retrieve all resources. The three datasets are the Traffic History dataset, the Control Stations dataset, and the Hourly Weather dataset. These were unified by time and place, to create the single analytical base for this thesis.

3.1.1 Traffic Sensor Data

To obtain traffic intensity data, the network of electromagnetic inductive loop detectors that were installed in Madrid was used to measure the number of vehicles that passed a fixed measurement point over a fixed period of time. Data were aggregated at 15-minute intervals by sensors, and in the same manner as Vélez-Serrano et al (2021) [9] and Gómez and Ilarri (2025) [18], data were aggregated at hourly resolution for this study. The source data is for January 2023 to May 2026, covering approximately 3.4 years of observation period. The total number of rows after the importing of the data and verification of the structure was 135,056,283 rows, where there were 4,971 sensor IDs.

The following attributes are included for each observation in the traffic dataset:

Table 3.1: Description of traffic dataset variables

Column	Description	Unit
SENSOR_ID	Unique sensor identifier	–
DATETIME	Timestamp of the observation	–
TRAFFIC_INTENSITY	Vehicle flow rate	veh/h
OCCUPANCY	Fraction of time the detector is occupied by a vehicle	%
LOAD	Weighted combination of intensity and occupancy	%
AVERAGE_SPEED	Mean speed of passing vehicles	km/h
ERROR_FLAG	Validity indicator (valid / not validated / provisional)	–
INTEGRATION_PERIOD	Duration over which the count was aggregated	minutes

Geographic coordinates (LATITUDE, LONGITUDE) were also kept from the Control Stations dataset, which assigns a geographic coordinate to each sensor and where each sensor is physically installed on the road network.

3.1.2 Weather Data

The data used for the weather observations were from the network of municipal meteorological stations in Madrid. Every traffic sensor observation was matched with the closest weather station using Euclidean distance — the straight-line distance between two points computed from their latitude and longitude coordinates, based on both the sensor and weather stations' geographic coordinates. The resulting combined dataset contains information on barometric pressure (hPa), precipitation (mm), relative humidity (%), solar radiation (W/m^2), temperature (°C), wind direction (degrees) and wind speed (m/s). Distance of each sensor to its corresponding weather station (`DISTANCE_TO_WEATHER_KM`) was maintained as a data quality indicator.

3.1.2.1 OpenStreetMap Road Attribute Data

Road infrastructure attributes were retrieved from OpenStreetMap (OSM) through the OSMnx library [19], which is a programmatic interface to the OSM Overpass API. The complete network of roads in Madrid that can be driven was downloaded from the municipality as a GeoDataFrame consisting of directed edges. A spatial nearest-neighbor search over the projected road network was then performed for each traffic sensor, using the `ox.distance.nearest_edges` function to assign a road edge from the OSM road network to each traffic sensor.

Code 3.1: Mapping traffic sensors to the nearest OpenStreetMap road edge

```
# Download Madrid's drivable road network
G = ox.graph_from_place("Madrid, Spain", network_type="drive")
edges = ox.graph_to_gdfs(G, nodes=False)  # GeoDataFrame of all
    road segments with attributes

# Build sensor GeoDataFrame from your existing sensor_map_table
from shapely.geometry import Point

sensors_gdf = gpd.GeoDataFrame(
    sensor_map_table,
    geometry=gpd.points_from_xy(
        sensor_map_table["LONGITUDE"],
        sensor_map_table["LATITUDE"]
    ),
    crs="EPSG:4326"
)

nearest_edges = ox.distance.nearest_edges(
    G,
    X=sensors_gdf.geometry.x,
    Y=sensors_gdf.geometry.y,
    return_dist=False
)
```

Centroids were determined for polygon geometries after projecting to the coordinate reference system EPSG: 25830 so that they would be in metric units [20]. The following attributes were extracted for each of the sensors: highway: highway classification, lanes: number of lanes, oneway: oneway attribute, maxspeed: posted speed limit, name: road name. When multiple values were inherited from adjacent road segments, as OSM attributes do, the first value was kept in line with common practice in the traffic forecasting literature [21].

Moreover, OpenStreetMap was used to extract Points of Interest (POI) data for 6 categories that were classified as traffic generating locations in the transport literature [22]: railway stations (`railway=station`), sports stadiums (`leisure=stadium`), hospitals (`amenity=hospital`), universities (`amenity=university`), shopping centers (`shop=mall`), and bus terminals (`amenity=bus_station`).

These categories were chosen because they have been shown to have measurable traffic demand impacts [23]. The geodesic distance from the extracted POI geometries to the

Table 3.2: OpenStreetMap tags used to identify relevant points of interest

OSM Tag	Meaning
`railway=station`	Railway stations — locations tagged in OSM with the key `railway` and value `station`, meaning a stop on a rail network
`leisure=stadium`	Sports stadiums — venues tagged with key `leisure` and value `stadium`
`amenity=hospital`	Hospitals — facilities tagged with key `amenity` and value `hospital`
`amenity=university`	Universities — campuses tagged with key `amenity` and value `university`
`shop=mall`	Shopping centres — commercial facilities tagged with key `shop` and value `mall`
`amenity=bus_station`	Bus terminals — major stops tagged with key `amenity` and value `bus_station`

nearest POI in each category was calculated for each sensor with the haversine formula using the centroid coordinates of the extracted geometries.

The complete hourly-aggregated dataset occupies approximately 3.9 GB on disk and requires more RAM to load fully into memory as a pandas DataFrame — exceeding the available memory on a standard laptop. For this reason, all data loading in this thesis uses DuckDB, a columnar query engine that reads directly from Parquet files on disk without loading the full file into memory.

3.2 Road Classification

The sensor measurement locations in Madrid are classified into three administrative groups, as detailed in the ELEMENT_TYPE column. The other category was present in just 2 observations collected by one sensor and was not used in any analysis. Table 3.3 summarises the distribution of sensors and observations into the three categories that have been retained.

Table 3.3: Sensor distribution by road class

Road class	Sensors	Observations	Mean intensity (veh/h)	Median intensity (veh/h)
URB	4,508	123,202,136	268.5	143.5
M30	315	8,495,730	1,260.3	798.3
C30	149	3,358,415	960.0	480.0

The URB (Urban) category represents sensors deployed on the municipal roadway network, including residential streets, secondary roads, arterials, and access connectors. The M30 classification refers to sensors installed on the M-30 ring motorway, the main express motorway of the Madrid metropolitan area, which is predominantly an open-air installation. The C30 category refers to "Calle 30", with a dedicated sensor network deployed only in the underground sections of the M-30 ring road. Although Calle 30 and the M-30 are located within the same physical corridor, the C30 sensor network is a separate monitoring system for the underground sections and is also published as a distinct data feed on the open data portal.

These three classes were found to be behaviorally distinct in the experiments. The average vehicle speed was 65.7 km/h for the M30 class, compared with 48.3 km/h for the C30 class, representing a difference of approximately 26%. This difference may reflect the physical constraints of tunnel driving, such as lower speed limits, limited visibility, and narrow tunnel geometries [3]. Despite similar tunnel occupancy rates (5.61% for M30 and 5.72% for C30), the M30 class showed a higher mean traffic intensity (1,260 veh/h) than the C30 class (960 veh/h). This suggests lower throughput per unit of occupation time in the tunnel sections, consistent with their lower operating speeds.

3.2.1 Urban Sub-Classification

An analysis of whether sensors were physically installed on motorway connectors or trunk road segments within the URB category found that a subset of sensors were located on such road types, despite being administratively classified as urban sensors. In particular, 145 URB sensors were matched to the OSM highway categories `motorway`, `motorway_link`, `trunk`, or `trunk_link`. These sensors also exhibited significantly different traffic characteristics, with a mean traffic intensity of approximately 611 veh/h and mean occupancy of 7.65%, compared with 257 veh/h and 5.20%, respectively, for the remaining URB sensors. This pattern is consistent with the typical bottleneck and merging behaviour of motorway ramp connectors, where multiple urban traffic streams merge into a single access lane [12]. Therefore, these sensors were reclassified as `URB_ramp`, while the remaining 4,295 sensors were classified as `URB_normal`. The resulting taxonomy of the four road classes used in this study is summarized in Table 3.4.

Note that the average speed is always 0 for all URB sensors, irrespective of the traffic conditions, meaning that the speed measurement capability is not working on urban loop detector hardware in the Madrid data set. This was validated for all 4,440 URB sensors and is listed as a data limitation. For M30 and C30 sensors, therefore, only the

Table 3.4: Final road sub-classification

Sub-class	Sensors	Mean intensity (veh/h)	Mean occupancy (%)	Speed available
URB_normal	4,295	257.0	5.20	No (always 0)
URB_ramp	145	610.8	7.65	No (always 0)
M30	304	1,273.7	5.61	Yes
C30	143	961.4	5.72	Yes

speed data were retained as a feature since only valid measurements were available for these sensors.

3.2.2 Main Artery Identification

For the identification of main arteries as a meaningful spatial division, sensors were classified according to the OSM `highway` category of their matched road segment. Main artery sensors were defined as those matched to the highway types `motorway`, `motorway_link`, `trunk`, `trunk_link`, `primary`, `secondary`, `primary_link`, or `secondary_link`, irrespective of their administrative class (URB, M30, or C30). A total of 2,130 sensors, corresponding to 44.0% of the 4,841 sensors with valid geographic coordinates, were classified as main artery sensors. Their distribution across the OSM highway types is summarized in Table 3.5.

Table 3.5: OSM highway types among main artery sensors

Highway type	Sensor count
primary	822
secondary	715
motorway_link	278
motorway	212
trunk	59
primary_link	28
secondary_link	15
trunk_link	1

3.3 Geographic Distribution of Sensors

Geographic coordinates joined from the Control Stations dataset were used to investigate the spatial distribution of the sensor network. The sensors cover the entire area of Madrid, with the URB network distributed throughout the interior road network and M30 and C30 sensors concentrated along the ring motorway corridor. The C30 sensors

were identified in three geographically concentrated areas in the southern part of the M-30 ring road, approximately between latitudes 40.39°N–40.42°N and longitudes 3.71°W–3.65°W. This distribution is consistent with the location of the M-30's main tunnel sections along the Manzanares river corridor. The observed geographic concentration was subsequently found to be associated with the higher data-quality problems reported for the C30 class (Section 4.1).

The high traffic intensity readings of sensor 7023 observed during the initial exploratory analysis were investigated further. The sensor was located approximately 449 m from Méndez Álvaro railway and bus station, one of Madrid's key multimodal transport hubs. This proximity is consistent with traffic demand effects around transport interchange locations [24]. As the sensor was not excluded from the dataset, its high intensity readings were considered to represent a real traffic phenomenon, partly influenced by interchange-related demand. This case motivated the addition of POI proximity as an explicit static covariate to the TFT model and as a variable to be included in the SHAP explainability analysis.

3.4 Data Splits

As usual for time series, the data set was divided chronologically into a training set, a validation set and a test set (Hyndman and Athanasopoulos, 2021 [25]; Joseph, 2022 [26]). Random split-up was not carried out because it would give an overestimation of the performance measures that would not be the performance of the model in real-life situations (Vélez-Serrano et al, 2021 [9]; Priya et al, 2025 [17]).

The time periods used in this study are:

Training: 1st January 2023 to 31st May 2025 (∼70%)
Validation: 1st June 2025 to 30th November 2025 (∼15%)
Test: 1st December 2025 onwards (∼15%)

All pre-processing transformers used (normalisation statistics, per-sensor outlier caps, seasonal reference profiles) and the model weights were tuned using the training data set. The validation set was used for hyper-parameter optimization and for early stopping. The test set was not utilized in any of the steps of parameter estimation but rather only used at the end of the model evaluation.

A buffer zone of encoder_length hours was added to the beginning of the validation and test data frames for models with sliding windows (N-BEATS, TFT) to ensure the full context of the encoder for prediction windows at the start. It was set with the aim of

not having any target in the decoder inside this buffer zone, which was confirmed by hard assertions before training was started.

3.5 Sensor Sample for Deep Learning Training

The complete network of 4,886 active sensors could not be directly utilized for training the deep-learning model in the GPU memory of the experimental server (NVIDIA RTX A2000, 12 GB VRAM). A representative sample of 200 sensors was thus chosen (50 for each road sub-class) for all deep learning experiments. This sample size was demonstrated to be suitable and to maintain sub-class proportionality in a computational tractable size.

In each sub-class, the sensors were ranked from high to low based on the number of valid training observations (VTO) where 'valid' is defined as the target is not null, the lag_168h is not null and the number of training observations is at least 8760 observations (one year of coverage). For each sub-class, the 50 sensors most complete in terms of data were chosen. The criterion is based on the notion of completeness, meaning the chosen sensors should have the most credible and persistent records; this is important because of the data quality problems that were found in Section 4.1 All 200 sensors are the same in all three modelling scenarios (CNN+LSTM, N-BEATS, and TFT), which allows us to compare the models without being influenced by variations in the number of sensors.

It is the same as Gómez and Ilarri (2025) [18] did in their deep learning exercises with a representative set of sensors in Madrid, making the exercises computationally feasible.

Chapter 4

Data Preparation and Preprocessing

Transforming raw sensor readings into a dataset appropriate for deep learning forecasting is often not a trivial process, and for the traffic network in Madrid there are multiple non-trivial decisions to make between the raw data and a feature set suitable for a model to be fed. The unprocessed observations include hardware errors up to values that correspond to physically impossible readings, sensors that supply static readings for several weeks in succession, gaps of different length due to maintenance or communication failure. These are all deliberate problems and the actions taken in response to one will affect the choices in the next.

This chapter explains all the preprocessing steps and features engineering that were implemented for this study. The decisions made here apply to all three modelling stages (CNN+LSTM, N-BEATS and TFT), although the features fed into each model vary, as outlined at the end of this chapter. The pipeline uses Python and is powered by an on-disk query processor called DuckDB (Raasveldt and Mühleisen, 2019) [27]. This way we make sure that our 135 million rows dataset will not be loaded into the main memory and the same pre-processing procedure will be applied every time.

4.1 Data Quality Assessment

4.1.1 Error Flag Analysis

The Madrid traffic data set includes the field `ERROR_FLAG`, which is used to denote if the sample in a certain measurement interval was erroneous or substituted. In the complete

data set, 134,712,264 records (99.75%) had N (no error) value, whereas 344,019 records (0.25%) had null value of the flag. No positive error codes were found. Because of the near-universal N value, the field will be able to respond with the same flag when it receives either physically impossible data or a data from a sensor that has frozen. This restriction does not apply to Madrid only. Turner et al. (2004) [28] point out that many hardware error flags in inductive loop detector networks do not detect fabrication type errors, or errors that occur when a counter yields a seemingly valid but actually invalid (non-null, non-negative) output. With no useful error flag available, this was an incentive to incorporate a physics-based quality assessment to motivate.

4.1.2 Physically Impossible Readings Detection

The maximum traffic intensity values were inspected, and a few data points were found where the intensity was more than 20,000 vehicles per hour. The maximum recorded was 91,368 veh/h, from sensor 7023. This speed is not possible for any road segment in Madrid's infrastructure, no matter how many lanes it has.

These extreme values were cross-referenced with `AVERAGE_SPEED` to find a commonality: many sensors reporting these extreme values at the same time showed that `AVERAGE_SPEED` = 0, which is inconsistent with the actual traffic state in which vehicles are moving. This is one of the most common problems associated with inductive loop detector systems according to Turner et al. (2004) [28]. This problem includes high intensity with zero speed. In all the cases, the `ERROR_FLAG` was set to N which means that no fault was detected.

There is not just one global threshold in response to these readings; it is a per-sensor cap. The valid traffic intensity range depends fundamentally on the road class of the sensors: Traffic intensities above 1,000 veh/h are common at the M30 motorway sensors during the peak period, but at the `URB_normal` sensors in the residential zone, traffic intensities rarely exceed 400 veh/h even on a busy day. A global limit of, say, 5,000 veh/h would probably leave most "real" peaks on the M30 unaffected and keep large manufactured readings on residential sensors going. On the other hand, a world-wide cap designed to safeguard residential sensors would falsely cut traffic from the motorway.

The method used here is based on the same underlying Madrid data set used by Vélez-Serrano et al. (2021) [9], but here a mean $\pm$ 2 standard deviation clip is applied to the per-sensor statistics with a 99.9th percentile threshold applied prior to the computation of the normalisation parameters. This particular approach is described in Section 4.4 and involves one step combining normalisation and quality-based filtering. This resulted in the removal of 4,366 observations from the training period (0.103% of the training

dataset). They remain within the stored dataset because the training loss function (MAE) does not respond to their existence as described in Section 5.1.3.

4.1.3 Detection of Stuck-Sensor Episodes

The other more serious quality problem is the existence of sensors that seem to be working but give a fixed value for a long period of time. Frozen-sensor readings are not null values and pass all common missingness tests, as opposed to missing values, which are clearly missing. They can only be detected by run-length analysis – recognizing long sequences of equal successive readings in one time series from one sensor.

A 'run-length encoding' approach, as a vectorized pandas operation, was used to identify stuck-sensor events. The number of consecutive hours for runs of three or more identical non-zero readings were set to three for C30 sensors, and six for both URB and M30 sensors. The various thresholds are related to a physical difference – for C30 tunnel sensors, a duration of stuck runs of about 63 hours was determined for the mean to be almost a sure sign of a hardware problem, even if it's only 3 hours. For URB sensors, especially on quiet residential streets with a low number of vehicles on the road at night, a series of 3 to 4 identical low intensity readings is physically reasonable and should not be flagged.

The percentage of stuck sensors was not similar for different road classes as evident from Table 4.1. The C30 category is greatly affected by the problem because 22.38% of the total number of records was affected by the stuck sensors, and this clearly demonstrates the maintenance problems of the underground sensors, especially weak signal strength and difficult access (Asadi and Regan, 2020) [3].

TABLE 4.1: Stuck-sensor statistics by road class

Road class	**Stuck runs**	**Total stuck hours**	**Mean run length (h)**	**Repeated-value rate (%)**
URB	57,381	334,313	5.8	0.99
M30	142	523	3.7	4.86
C30	6,709	424,989	63.4	22.38

Observations classified as "stuck" were treated as missing data and subsequently processed during the imputation stage. This prevented artificially repeated values from being interpreted as genuine traffic observations.

4.2 Missing Value Imputation

4.2.1 Overview of Missingness

The percentage of missing observations for each road class after the reclassification of stuck-sensor readings is presented in Table 4.2. C30 is disproportionately affected, with 8.78% of observations requiring imputation. This is more than five times the missingness rate of URB and M30.

TABLE 4.2: Missingness by road class after stuck-sensor reclassification

Road class	Missing (%)	Sensors affected
URB	2.56	4,379
M30	1.64	302
C30	8.78	130

Before selecting an imputation technique, the distribution of missing-gap lengths was examined, as short and long gaps require different treatment. For short gaps of one or two hours, the observed values immediately before and after the gap provide sufficient information for linear interpolation. In contrast, interpolating across long gaps spanning several days could produce an approximately linear trend that ignores daily traffic cycles and peak periods, which are important for forecasting.

The median gap length was below six hours, while the 95th percentile was no greater than 20 hours across all three road classes. This indicates that most missing intervals are sufficiently short for the surrounding observations to support linear interpolation. This finding is consistent with Gómez and Ilarri (2025) [18], who reported a similar gap-length distribution for an associated Madrid traffic dataset. Before imputation, 81 sensors with less than 50% data availability were classified as unreliable and removed entirely from the dataset.

4.2.2 Imputation Approaches in the Literature

Before describing the specific method used in this thesis, I have figured it out what other studies on the same Madrid dataset have done. This makes the methodological contribution explicit and allows the results to be interpreted in the context of established practice.

TABLE 4.3: Imputation approaches across studies using Madrid traffic data

Study	**Missing data strategy**	**Gap threshold**	**Comparable to our study?**
Vélez-Serrano et al. (2021) [9]	Linear interpolation for gaps $\leq$ 3h; sensor excluded if longer gaps present	3 hours	Yes – this study extends the interpolation threshold to 24h and adds a seasonal fallback rather than discarding sensors
Gómez and Ilarri (2025) [18]	Completeness threshold ($\geq$ 90% valid observations); no explicit imputation	N/A – exclusion based on completeness	Yes – the 30% fabrication cap used here is equivalent in spirit: sensors with excessive synthetic data are excluded
This study	Tiered: linear interpolation $\leq$ 24h; seasonal-naive median fill $>$ 24h; fabrication cap at 30%	24 hours	–

4.2.3 Tiered Imputation Strategy

Two separate imputation pipelines were applied according to road class. Motorway sensors exhibit different speed–flow–density relationships from urban street sensors; therefore, applying a common imputation model across classes could introduce inappropriate assumptions about their traffic behaviour. In particular, information from motorway or tunnel sensors may not provide a reliable basis for reconstructing missing observations at residential or urban street sensors, and vice versa. Separating the pipelines therefore reduces the risk of transferring traffic patterns between fundamentally different road environments.

Each pipeline processes sensors individually. For every sensor, the first step is to reindex its time series to a complete and continuous hourly timeline. Any missing timestamps in the raw data are explicitly inserted as `NaN` rows before the imputation procedures are applied:

CODE 4.1: Reindexing sensor observations to a continuous hourly timeline

```
for sid in sensor_ids:

    # Load all observations for this sensor
    df = con.execute(f"""
```

```
        SELECT *
        FROM read_parquet('{clean_base}')
        WHERE SENSOR_ID = {sid}
        ORDER BY DATETIME
    """).df()

    df["DATETIME"] = pd.to_datetime(df["DATETIME"])

    # Reindex to a complete hourly timeline.
    # Missing hours become explicit NaN rows.
    full_index = pd.date_range(
        start=df["DATETIME"].min(),
        end=df["DATETIME"].max(),
        freq="h"
    )

    df = df.set_index("DATETIME").reindex(full_index)
    df.index.name = "DATETIME"

    df["SENSOR_ID"] = sid

    # Mark rows that were originally missing
    df["WAS_MISSING_ROW"] = (
        df["TRAFFIC_INTENSITY"].isna().astype("int8")
    )
```

The raw sensor data does not always contain one observation per hour. During communication failures or maintenance periods, the data feed may stop completely, resulting in missing rows rather than rows containing null values. Reindexing each sensor to a complete hourly timeline converts these otherwise invisible gaps into explicit `NaN` values, which can then be detected and processed during the imputation stage.

4.2.3.1 Tier 1 — linear interpolation for gaps 24 hours or less

If the intervals of missing data were contiguous and no more than 24 hours long, missing data were replaced by linear interpolation between the adjacent observed data:

$$\hat{x}_{i,t} = x_{i,t_1} + \frac{t - t_1}{t_2 - t_1} \left(x_{i,t_2} - x_{i,t_1} \right) \tag{4.1}$$

where x_{i,t_1} and x_{i,t_2} are the values at the previous observation to the gap and the next observation after the gap. The `limit_area="inside"` parameter was used to prevent

the linear/quadratic extrapolation out of the range of the sensor's actually measured data. The assumption of smooth continuous change is reasonable within a range of up to 24 hours: the traffic doesn't teleport between radically different states within one day.

For each sensor, the length of every contiguous missing run is computed using a run-length encoding approach — a vectorised operation that groups consecutive missing values and counts how many hours each gap spans:

Code 4.2: Computing the length of contiguous missing-data runs

```
# "Vectorised" means this operates on the entire column at once
# using optimised compiled code -- not a Python loop over
# individual rows.

missing = df["TRAFFIC_INTENSITY"].isna()

# group_id increments each time the missing/observed state
# changes.
# All rows belonging to the same contiguous gap share the same
# group_id.

group_id = (missing != missing.shift()).cumsum()

# transform("sum") fills each row in a gap with the
# total length of that gap.

gap_len = missing.groupby(group_id).transform("sum")

df["GAP_LENGTH_HOURS"] = np.where(
    missing, gap_len, 0
).astype("int32")
```

Missing values in gaps of 24 hours or fewer are then filled using **time-aware interpolation** (`method="time"`). Unlike simple linear interpolation which assumes equally-spaced data points, time-aware interpolation uses the actual datetime spacing of the index — producing more accurate fill values when observations are not uniformly spaced:

Code 4.3: Time-aware interpolation for short missing-data gaps

```
# Interpolate traffic columns using the datetime index for spacing
    .
# method="time": uses actual timestamp differences, not row index
    positions.
```

```
# limit_area="inside": prevents extrapolation beyond the first and last
# observed values.

traffic_cols = [
    "TRAFFIC_INTENSITY", "OCCUPANCY", "LOAD",
    "AVERAGE_SPEED", "INTEGRATION_PERIOD"
]

for col in traffic_cols:
    if col in df.columns:
        interpolated = df[col].interpolate(
            method="time",
            limit_area="inside"   # no extrapolation beyond observed range
        )

        # Apply only to rows that are: (a) missing AND
        # (b) part of a gap of 24 hours or fewer
        mask = df[col].isna() & (df["GAP_LENGTH_HOURS"] <= 24)
        df.loc[mask, col] = interpolated.loc[mask]

# Track which rows were filled by Tier 1
df["IS_IMPUTED"] = (
    (df["WAS_MISSING_ROW"] == 1) &
    (df["TRAFFIC_INTENSITY"].notna())
).astype("int8")
```

4.2.3.2 Tier 2 — Seasonal-naive fill for holes > 24 hours

For an interval greater than 24 hours, linear interpolation would generate a straight-line solution that would cover several daily cycles, resulting in a loss of the morning and evening peaks which are the most prominent feature of traffic time series. Rather, each missing observation was replaced by the median value of the historical observations made on that sensor on that particular hour and day of the week using only those observations that were recorded during the training period:

$$\hat{x}_{i,t} = \text{median}\left\{x_{i,t'} : \text{hour}(t') = \text{hour}(t),\ \text{dow}(t') = \text{dow}(t),\ \neg\,\text{stuck}(i,t'),\ x^{\text{raw}}_{i,t'} \neq \text{null},\ t' \in T_{\text{train}}\right\}. \tag{4.2}$$

where dow(t) is the day of week of timestamp t (0 = Monday, 6 = Sunday) and $x_{i,t'}^{\text{raw}}$ refers to the original observed value before any imputation.

Critically, these reference profiles are built exclusively from observations that are:

- Not stuck readings (`~df["IS_STUCK"]`)
- Not already imputed (`df["TRAFFIC_INTENSITY_RAW"].notna()`)

This double filter prevents compounding of errors — the seasonal profiles reflect only genuinely observed traffic behaviour:

Code 4.4: Seasonal median filling of remaining missing traffic observations

```
def seasonal_fill(df):
    """
    Fills remaining NaNs using each sensor's own historical median
    for the same (day_of_week, hour) combination.
    """
    df = df.copy()
    df["DATETIME"] = pd.to_datetime(df["DATETIME"])
    df["hour"]     = df["DATETIME"].dt.hour
    df["dow"]      = df["DATETIME"].dt.dayofweek  # 0=Monday, 6=Sunday

    # Build per-sensor, per-(dow, hour) median profiles
    # from truly observed values only
    seasonal_lookup = (
        df[~df["IS_STUCK"] & df["TRAFFIC_INTENSITY_RAW"].notna()]
        .groupby(["SENSOR_ID", "dow", "hour"])["TRAFFIC_INTENSITY_RAW"]
        .median()
        .rename("SEASONAL_FILL_VALUE")
        .reset_index()
    )

    # Join the seasonal reference values onto the full dataset
    df = df.merge(
        seasonal_lookup,
        on=["SENSOR_ID", "dow", "hour"],
        how="left"
    )

    # Apply fill only where Tier 1 left NaN values remaining
    still_missing = df["TRAFFIC_INTENSITY_FINAL"].isna()
```

```
    df["IS_SEASONAL_FILLED"] = (
        still_missing & df["SEASONAL_FILL_VALUE"].notna()
    )

    df["TRAFFIC_INTENSITY_FINAL"] = np.where(
        df["IS_SEASONAL_FILLED"],
        df["SEASONAL_FILL_VALUE"],
        df["TRAFFIC_INTENSITY_FINAL"]
    )

    return df.drop(columns=["hour", "dow"])
```

This is similar to the approach used in the traffic imputation literature (Boquet et al., 2020 [29]), referred to as the Historical Average approach. The mean was more likely to be skewed by the remaining hardware-fault readings, as they would have no impact on the median, so the median was chosen over the mean. To avoid compounding of existing imputations, only truly observed values (non-stuck, non-interpolated values) were used in computing the seasonal reference.

4.2.4 Fabrication cap

To minimize the contribution of the seasonal fill to the overall amount of synthetic data for an effectively non-functional sensor, sensors for which more than 30% of the data were added during the seasonal fill were not used in further analyses. This threshold is based on a belief that a model trained on data where almost one third of the data has been artificially manufactured is not learning from a reliable signal. The total number of sensors excluded were 61 (16 from C30, 4 from M30, and 41 from URB). The 16 sensors removed from C30 were grouped together geographically in the three locations of problematic infrastructure underground tunnels discussed in Section 3.3, thus supporting the assertion of location specific structural failure and degradation in this infrastructure, and not random degradation.

4.2.5 Imputation Provenance

Three binary indicator columns were added to the final dataset to track the origin of each observation, enabling full reproducibility and transparency:

- `IS_IMPUTED` — set to 1 if the value was filled by Tier 1 time-aware interpolation
- `IS_SEASONAL_FILLED` — set to 1 if the value was filled by Tier 2 seasonal naive fill

- `WAS_MISSING_ROW` — set to 1 for any row that did not exist in the raw data before reindexing

The final data provenance is summarized in Table 4.4.

TABLE 4.4: Data provenance after imputation by road class

Road class	Sensors retained	Observed (%)	Linear interpolated (%)	Seasonal filled (%)	Still missing (%)
URB	4,399	97.30	0.09	2.26	0.35
M30	300	98.36	0.00	1.16	0.48
C30	127	79.74	0.90	14.55	4.81

The difference between URB/M30 and C30 is substantial. More than 97% of observations for URB and M30 were made directly and less than 2.5% needed any intervention. The amount of observations that were either linearly interpolated or seasonally filled was close to one-fifth of the total observations for C30, and 4.81% observations are still missing even after the data were imputed. This data limitation (as a residual "missingness" to C30) is documented and is expected to affect the performance of models trained on C30 sensors, as detailed in the results chapter.

4.3 Feature Engineering

4.3.1 Overview

Factors that affect traffic intensity at a particular sensor at a particular time of day affect a wide range of timescales — the momentum of traffic as it occurs (what happens over past one to three hours), the daily rhythm of human activity (the commute pattern, the lunch hour, the evening return to the home, the weekday/weekend cycle), the longer-term calendar (holidays, school terms), and the physical context of the sensor location (whether it's on a main artery, near a transport hub or on a quiet residential street). With good feature engineering, they all become factors that a neural network is able to learn from.

There are 28 variables grouped into six features. A summary is provided in Table 4.5, and the next sections outline the rules and reasons for each group.

TABLE 4.5: Final feature set (28 features)

Group	Features	Count
Cyclical temporal	`hour_sin`, `hour_cos`, `dow_sin`, `dow_cos`, `month_sin`, `month_cos`	6
Calendar flags	`is_weekend`, `is_holiday`, `is_rush_hour`, `season`	4
Lag features	`lag_1h`, `lag_2h`, `lag_3h`, `lag_24h`, `lag_168h`	5
Rolling means	`roll_mean_3h`, `roll_mean_24h`	2
Road static	`road_subclass_enc`, `longitude`, `latitude`, `is_main_artery`	4
POI proximity	`dist_station_km`, `dist_stadium_km`, `dist_hospital_km`, `dist_university_km`, `dist_mall_km`, `dist_bus_station_km`, `near_major_poi`	7
Total		**28**

4.3.2 Temporal Features

The first question when encoding time is how to capture its circularity. For instance, hour of day: these are 23 units apart numerically, but 1 hour apart in traffic — a nice smooth transition between late night and midnight. A linear integer encoding then breaks this continuity such that any model will be considering 0:00 as a point of discontinuity, rather than part of a smooth low traffic dark time of the night. To overcome this, sine and cosine encodings map each time unit to a circle:

$$\sin_h = \sin\left(\frac{2\pi \cdot h}{24}\right), \qquad \cos_h = \cos\left(\frac{2\pi \cdot h}{24}\right) \tag{4.3}$$

The sine and cosine of any time value uniquely determine a position in the cycle and also maintain the circular distance relationship. The same encoding was applied to day of week (period 7) and month (period 12). The approach is frequently used in traffic prediction models (see Chauhan et al., 2025 [8]; Gómez and Ilarri, 2025 [18]) and is suggested by Joseph (Chapter 6, 2022 [26]) as the best one for cyclical time attributes.

Three calendar indicators have been derived. Saturday and Sunday are indicated with the flag `is_weekend`. Spanish national and Madrid regional public holidays are shown

with `is_holiday` flag, using the holidays Python library with holidays.The Spanish holiday calendar (subdiv='MD', years=range(2023, 2027)), which includes the Community of Madrid's holiday calendar, as the regional holidays (e.g. Almudena Day, 9 November) generate significant traffic anomalies that are not reflected in the national holiday calendar. The `is_rush_hour` flag signals the mornings peak (06:00 - 09:00) and evenings peak (17:00 - 20:00) which were decided based on exploratory analysis described in Chapter 6.

Slow seasonality, which is not captured by the monthly cyclical encoding, was encoded as a meteorological season variable (December–February: winter; March–May: spring; June–August: summer; and September–November: autumn).

4.3.3 Lag and Rolling Window Features

A lagged feature is a feature that is the most direct representation of the time series autocorrelation structure from a tabular feature set. The idea is very simple, if the traffic intensity one hour ago is a good predictor of the traffic intensity now, then the lag-1h value should be included in the feature vector. In the case of traffic time series, the structure of the autocorrelation is applied at three different time scales.

The intensity values of `lag_1h`, `lag_2h` and `lag_3h` are indicators of short-term momentum, representing the intensity of the previous hour, the previous two hours and the previous three hours respectively. They reflect the fact that often the traffic conditions change gradually from one hour to the next under normal conditions: if the sensor is congested one hour, it's likely to be congested the next hour, and if it's quiet one hour, it's likely to be quiet the hour after.

`lag_24h` — traffic intensity during the same hour yesterday, reflecting the hourly pattern. So, for instance, traffic intensity at 08:00 on Tuesday is closer to traffic intensity at 08:00 on Monday than at 08:00 on Sunday.

This is what `lag_168h` represents: The intensity at the same hour one week ago. Through Kruskal-Wallis testing,Vélez-Serrano et al. (2021) [9] proved that, for this type of sensor data in Madrid, the weekly seasonal correlation (lag 168) is statistically stronger than the daily one (lag 24), therefore both should be included and the 168 hours should be the most important reference point for this seasonal correlation.

All lag features were calculated in each time series of the respective sensor (with window functions, where applicable, divided by the `SENSOR_ID` and ordered by the `DATETIME`) with no cross-sensor leakage in the calculation step. To ensure there is no forward

leakage, two rolling mean features were also calculated; the mean of the previous 3 hrs (`roll_mean_3h`), and the mean of the previous 24 hrs (`roll_mean_24h`).

4.3.4 Road Static Features

Road sub-class was encoded as an ordinal integer: `URB_normal` = 0, `URB_ramp` = 1, `M30` = 2, `C30` = 3. This encoding enables the global forecasting model paradigm used by Joseph (2022, Chapter 10 [26]) for traffic data by Gómez and Ilarri (2025) [18] to learn class-specific patterns without having to learn a separate model for each road type. Geographic location (longitude and latitude) of each sensor is provided as continuous feature, so the model can learn the baseline intensity for each sensor no matter they are in the same road sub-class, or have different baseline traffic intensity due to their geographic locations. The concept of a principal arterial road and a local street has been differentiated with the introduction of a boolean attribute called `is_main_artery` where, according to the OSM classification of the road (see Section 3.2.2), this attribute is set to true if the road is a principal arterial road, and to false if it is a local street.

4.3.5 Point of Interest Proximity Features

Six distance features measure the geodesic distance in kilometers from each sensor to the nearest Point of Interest in six categories of points: railway stations, metro stations, sports stadium, hospital, university, shopping center and bus terminal. The haversine formula was used to calculate these distances:

$$d = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)}\right) \tag{4.4}$$

Here ϕ_1, ϕ_2 are the latitudes and λ_1, λ_2 the longitudes of two points in radians and its difference shows $\Delta\lambda$, and the mean radius of the earth, $R = 6,371$ km. A new binary `near_major_poi` (NMP) flag was added with a value of 1 if the sensor was located within 500 meters of any of the 6 categories of POI.

Code 4.5: Adding the binary POI proximity flag

```
# Add binary POI flag + save

poi_cols = [f"dist_{cat}_km" for cat
in poi_categories]

results["near_major_poi"] = \
```

```
(results[poi_cols].min(axis=1) < 0.5).astype(int)
```

where `cat` is type of poi whether station, stadium or hospital etc,.

The density of traffic generating facilities in the Madrid urban area is shown by 3,944 out of 4,841 sensors (81.5%) within 500 meters of at least one major POI.

POI motivation is directly from the data. As stated in Section 3.3, the sensor 7023 located at Méndez Álvaro station shows the influence of POI distance on traffic intensity – this is the reason why features related to POI distance were introduced. If there is no POI proximity feature, a model would not be able to differentiate this sensor from other sensors with the same type of POI. By using it, the model can be trained that the sensors located near transport hubs have a different demand profile, one that is based on the departure time of trains and buses from the hubs, instead of the more traditional commuting profile. POI proximity features are also interesting for the explainability analysis that will be performed in respective chapter as they represent a spatial influence which is not visible with purely temporal models.

4.3.6 Weather Features

The joined dataset contained data for 7 weather parameters: temperature, precipitation, wind speed, relative humidity, solar radiation, barometric pressure and wind direction. Their investigation showed them a difficult situation; with 62.8% of the precipitations not observed in the training records, the wind speed was absent in 68.8% of the training records, and solar radiation was absent in 74.5% of the training records. The NULL pattern was seen at the same time in all three variables, and in fact, when one variable is missing, the others are usually missing as well; this is not the result of individual sensor failures, but rather of intermittent weather station downtime.

Weather features were not excluded from the full feature set investigation because of this missingness but instead were incorporated into the investigation. The lowest missingness rate was for temperature (12.5%) which was used to look at how it correlated with road class traffic intensity. The correlation observed in each sub-class was almost zero, similar to Gu et al. (2026) [15]], who found that temporal features were significantly more important than meteorological inputs for traffic intensity prediction, in their experiments on highway traffic data after ablating each feature. There is moderate correlation between precipitation and traffic, however, and the data isn't very useful due to the missingness.

Weather variables are stored and fed into the complete feature vector that is explored in the CNN+LSTM and TFT stages. This is not the case for the N-BEATS model, since

it is a model for only one variable, namely traffic intensity history and no other external covariates such as weather.

4.4 Prevention of Data Leakage

The data leakage issue, where information from the future creeps into the model training, is subtler in time series than using the test labels during training. In this pipeline, there were specific leakage risks that were identified and mitigated.

4.4.1 Normalisation statistics

The scaler parameters (Per-sensor Z-score means and standard deviations, 99.9th percentile caps) were only calculated during the training period. If the scaler was applied to the entire dataset after initial fitting, then subsequent traffic distribution data could be used to modify the representation of the data for the period of the fitting.

4.4.2 Lag features near split boundaries

It includes the following features: lag. The lag_168h variable at the first validation step (June 2025) is referring to a training-period observation (May 2025), as expected and not a value from the future. This was confirmed by examining the lag_source_time for the boundary rows, making sure that all lag values at validation and test time steps relate to only previous observations.

4.4.3 Rolling means

Rolling window features are calculated with the range BETWEEN N PRECEDING AND 1 PRECEDING; this is an explicit range that excludes the current timestep (and all future timesteps). No moving averages are present in the feature set that look forward.

4.4.4 Seasonal fill reference profiles

Only the truly observed (non-stuck, non-interpolated) values for the training period were used in the computation of the historical median profiles used in Tier 2 imputation. The same profiles used for filling validation or test gaps are not considered leakage as they have no information about the imputed value, but only the typical value for the given hour and the given day of week.

4.4.5 Dataset boundary assertions

The parameter min_prediction_idx was set so that none of the decoder targets is in the buffer zone set for the encoder context for the sliding-window datasets used in N-BEATS and TFT. Before any training of the model was started, the validation and test boundary conditions were checked by hard assertions, failing which, the error was displayed instead of silently failing.

4.5 Final Dataset Characteristics

The feature-engineered data is stored in a single file called `traffic_features_v3.parquet`, which includes all sensors and all time periods covered by features. The key properties are summarized in Table 4.6.

TABLE 4.6: Final dataset summary (all sensors)

Property	**Value**
Total observations	135,056,283
Unique sensors	4,886
Features (input)	28
Target variable	`target` (vehicles per hour, post-imputation)
Target range (raw)	0–91,368 veh/h (including faults)
Target range (after p99.9 cap)	0–∼8,800 veh/h
Road sub-classes	4 (URB_normal, URB_ramp, M30, C30)
Date range	2023-01-01 to 2026-05-31

The dataset after stratified selection for the 200-sensor sample set for all the deep learning experiments has 5981200 rows corresponding to the sensor level.

4.6 Feature Set by Model

All 28 features are not used by all models. It's no accident. It is not an automatic feature of the design.

CNN+LSTM employs all 28 features. It is a multivariate sequence model, meaning that it takes all features simultaneously and processes the entire feature matrix over the length of the sequence in the encoder. This enables the CNN+LSTM to take advantage of all the covariate information and provides the multivariate baseline for the thesis.

The N-beats model is strictly univariate where only the normalized history of traffic intensity measurements (`target_norm`) from a single sensor are used as input. This restriction is not a restriction, but a conscious decision. N-BEATS is the interpretable baseline model in the thesis, which addresses the question of how much predictive information can be gained from the temporal signal alone? It is competitive performance that sets a univariate lower bound and can be used to measure the multivariate advantage of CNN+LSTM and TFT. The N-BEATS model is able to be configured to operate with `target_normalizer=None` in PyTorch Forecasting and the manually calculated per sensor z-scores are fed directly into the pipeline without modification.

The TFT includes a subset of 28 features which are partitioned into time varying and static covariates as is needed by the TFT architecture. Time-varying known inputs include the cyclical temporal features, calendar flags, and rolling means. Time-varying unknown inputs are: Lag features and normalized traffic intensity. Road sub-class, geographic coordinate and proximity distances to POIs are static covariates. Weather is available in the time-varying known set to the extent that it is available and is evaluated empirically as part of TFT's variable selection mechanism.

The aim of this successive introduction (univariate N-BEATS, multivariate CNN+LSTM, interpretable multivariate TFT) is to enable the contribution of each additional information source to be measured and attributed, thus directly supporting explainability goals of the thesis.

Chapter 5

Methodology

5.1 Normalisation

5.1.1 The Problem with Global and Per-Class Normalisation

Normalisation is required since neural networks are sensitive to the scales of their inputs. A feature with values in the range [0, 8,000] will give updates of very different magnitudes on its parameters when training, and a feature with values in the range [0, 1] will give updates of a very similar magnitude. This variation in the sizes of the updates to the parameters during training can make it hard to find a learning rate that works well for all of them at the same time.

The problem in the sensor network of Madrid is that the valid operating range of the traffic intensity varies greatly between the different road sub-classes, so a global normalizer is not a good solution and a per-class normalizer is not either. The global MinMaxScaler based on the entire data set will scale the maximum of the M30 to be close to 1 within the [0, 1] interval but will squeeze all values of the URB_normal features, usually lying in the interval from 0 to 600 veh/h, close to zero. All of the URB_normal sensors would appear exactly the same in normalized space, as the model would be missing the exact difference it needs to know between the low traffic and high traffic hours at residential sensors.

A per-class normalizer is a better solution, but continues to group sensors into each class. URB_normal has intensity values from around 50 veh/h (low traffic volume) to more than 400 veh/h (high traffic volume) within road network sensors. A class level normalizer normalizes both, and effectively gives them the same level of traffic, and the same level of meaningful deviation from that level.

5.1.2 Per-Sensor Z-Score Normalisation

In this study, the normalisation process used is per sensor z-score normalisation. For each sensor i, the mean μ_i and standard deviation σ_i are computed from the observations gathered by the sensor during the training period, with each sensor's data capped at the 99.9th percentile and then averaged:

$$\mu_i = \frac{1}{|C_i|} \sum_{t \in C_i} x_{i,t}, \qquad \sigma_i = \mathrm{std}\left(\{x_{i,t} : t \in C_i\}\right) \tag{5.1}$$

where $C_i = \{t \in T_{\text{train}} : x_{i,t} \leq P_{99.9}(i)\}$ is the set of training observations at sensor i that are at or below the 99.9th percentile of sensor i's training period observations. The cap is used before adding the statistics to the data, not to the data set itself, which means that if hardware faults occur, they are still in the stored statistics, but they do not affect the normalisation parameters. Then the normalized target is:

$$\tilde{x}_{i,t} = \frac{x_{i,t} - \mu_i}{\sigma_i} \tag{5.2}$$

applied to all observations — training, validation and test — were normalized with the same statistics from the training period. Using validation or test statistics would constitute leakage.

The Python implementation processes each sensor individually, computing statistics only from its training-period clean population:

Code 5.1: Per-sensor z-score normalisation and application to target and lag features

```
sensor_stats_list = []

for sid in SENSOR_IDS:

    # Extract training-period observations for this sensor only
    sensor_train = df.loc[train_mask & (df["SENSOR_ID"] == sid), "
   target"]

    # Per-sensor 99.9th percentile cap -- adaptive to each sensor'
   s distribution
    p999    = float(sensor_train.quantile(0.999))
    clean   = sensor_train[sensor_train <= p999]

    # Compute $\mu_i$ and $\sigma_i$ from clean population only
    mu_i    = float(clean.mean())
```

```
    sigma_i = float(clean.std())

    # Guard against near-zero variance (e.g. very quiet overnight
    sensors)
    if sigma_i < 1e-6 or np.isnan(sigma_i):
        sigma_i = 1.0

    sensor_stats_list.append({
        "SENSOR_ID": sid, "mu": mu_i,
        "sigma": sigma_i, "p999": p999,
    })

SENSOR_STATS = pd.DataFrame(sensor_stats_list).set_index("
    SENSOR_ID")

# Apply normalisation to target and all lag features
# using the same $\mu_i$ and $\sigma_i$ for each sensor
df["mu_i"]    = df["SENSOR_ID"].map(SENSOR_STATS["mu"])
df["sigma_i"] = df["SENSOR_ID"].map(SENSOR_STATS["sigma"])

TARGET_COLS = [
    "target", "lag_1h", "lag_2h", "lag_3h",
    "lag_24h", "lag_168h", "roll_mean_3h", "roll_mean_24h"
]

NORM_COLS = [
    "target_norm", "lag_1h_norm", "lag_2h_norm", "lag_3h_norm",
    "lag_24h_norm", "lag_168h_norm", "roll_mean_3h_norm", "
    roll_mean_24h_norm"
]

for raw_col, norm_col in zip(TARGET_COLS, NORM_COLS):
    df[norm_col] = (df[raw_col] - df["mu_i"]) / df["sigma_i"]
```

Normalisation of lag features: The same μ_i and σ_i are applied to all lag and rolling mean features — not just the target. This is essential for consistency: `lag_168h_norm` must be on the same scale as `target_norm` so that the model can learn the relationship between past and future values without a scale discontinuity between them.

Empirical verification across all three training runs (CNN+LSTM, N-BEATS, TFT) confirmed that the normalisation was applied correctly. The clean training population (observations at or below the per-sensor 99.9th percentile) met the requirements mean = −0.0000 and standard deviation = 1.0000 across all 200 sensors (selected while applying

deep learning models), which were met by construction. The full training population (including retained upper-tail readings) had: mean = 0.0033; `std_dev` = 1.0071; max reading z-score = 58.94 (which is a true hardware-fault reading and is now correctly assigned as a very large z-score, not silently discarded).

Instead of an absolute number of standard deviations, the cap threshold was selected as the 99.9th percentile to adapt to the distribution of each sensor; it is an adaptive threshold that establishes the upper tail of each sensor's distribution as the cap without being too high, a criterion that is needed to avoid a large number of false alarms. In comparison, the cap removed 4,366 (0.103%) normalisation observations from 200 sensors during training.

This same normalisation pipeline — same code, same `SENSOR_STATS` file, same 200 sensors — was applied identically to all three deep learning models (CNN+LSTM, N-BEATS, and TFT). The `sensor_stats.csv` file produced during each training run was reloaded during evaluation to perform the inverse transformation, ensuring that predictions are converted back to physical units using exactly the same statistics used during training.

Why Z-Score Over Min-Max for This Dataset

It is evident that z-score normalisation is much more suitable than the Min-Max Scaler for this application because of the following two properties of the data set. First, by construction Min-Max Scaler is sensitive to outlier data: if a single reading from the hardware were corrupted by a fault, such as reading 91,368 veh/h on a residential sensor, all of the real readings would occur at the bottom 0.5% of the normalized range and lose their signal. Z-score normalisation is based on the mean and the standard deviation of the distribution, calculated from the capped distribution, and thus not affected by one extreme value. This outlier sensitivity is the main limitation of the Min-Max normalisation technique in the presence of anomalous readings, as pointed out by Patro and Sahu (2015) [30]. Second, the same applies to Min-Max Scaler across splits: The maximum of the test set has to be less than or equal to the maximum of the training set, which is not true for traffic data when the test period might have a higher traffic event than the training period. This is not required with the z-score normalisation: if a test observation is 3 standard deviations above the training mean, it gets a z-score of 3.0, which is a meaningful measurement of how extreme it is.

5.1.3 Loss Function Robustness and Inverse Transformation

All models trained on normalized data have a training target of Mean Absolute Error (MAE) measured in z-scores.

$$L = \frac{1}{NF} \sum_{i=1}^{N} \sum_{h=1}^{F} \left| \hat{\tilde{y}}_{i,h} - \tilde{y}_{i,h} \right| \tag{5.3}$$

where N is the number of prediction windows in a batch, F is the number of forecast horizons (6 in this study), $\hat{\tilde{y}}_{i,h}$ is the model's z-score prediction for window i at horizon h, and $\tilde{y}_{i,h}$ is the corresponding z-score target.

The hardware-fault readings were included in the data set and for this reason MAE was chosen rather than Root Mean Squared Error (RMSE). Under RMSE, the amount of gradient pressure is proportional to the error of the prediction — so a 58.94σ fault observation creates ~ 59 times the gradient pressure of a typical normal observation, meaning that a small number of readings in the "abnormal" range can dominate the weight updates. With MAE, no matter how large the error, the contribution of each observation to learning's direction is constant at $1/N$, with each observation — even retained fault readings — having the same influence on the direction of learning. In the context of operational forecasting applications where there is sensor noise and outlier readings, MAE is recommended rather than RMSE according to Joseph (2022, Chapter 15 [26]).

The model outputs are converted back to physical units via the inverse transformation:

$$\hat{y}_{i,t} = \hat{\tilde{y}}_{i,t} \times \sigma_i + \mu_i \tag{5.4}$$

Each prediction is then capped at zero, reflecting the fact that traffic can't be negative. Small negative predictions occur during nights with low traffic levels with the polynomial and Fourier basis functions employed in N-BEATS, which is physically unfeasible.

5.1.4 Comparable Training and Validation Loss

There is a minor methodological issue when comparing training loss and validation loss in the course of neural network training. In this thesis, we use PyTorch Lightning, a training framework that calculates the loss function for training during an epoch as an average of all the training batches in that epoch. The weight updates are done sequentially: Every batch processed with the intermediate model state after the first

epoch weight update, after the second, etc. Validation loss is computed, on the other hand, after finishing all the weight updates for that epoch, with the final model state.

These two quantities are thus not calculated in the same manner. To alleviate this issue, a FrozenTrainEval procedure is carried out for CNN+LSTM and N-BEATS models. Once the epoch's weights are updated, the model is put in evaluation mode (`model.eval()`) with `torch.no_grad()`, thus disabling the computation of the gradients and dropout. The final state after completing a forward pass of all the training data is then passed through a complete forward pass, with this final state used to compute validation loss, and the resulting loss is logged as `train_loss_frozen`.

This results in the generalisation ratio being directly interpretable:

$$R = \frac{\text{val_loss}}{\text{train_loss_frozen}} \tag{5.5}$$

The model state, the evaluation procedure (`model.eval()`, no dropout), and the loss function (MAE on z-score targets) are the same for both the numerator and the denominator. $R = 1.0$ indicates that the training and validation sets are similarly challenging for the model. If the value is < 1.0, it is NOT a sign of overfitting, it is a property of the data, and the validation period is just easier to predict than the training period. A value greater than 1.0 indicates that the model generally is more difficult to validate than train, which is generally the case when dropout is applied to the model; artificially making training harder (randomly disabling neurons) and not making validation harder results in a higher value for it.

The FrozenTrainEval procedure is not possible to do in a reasonable amount of time for TFT. There are a total of about 4.2 million windows in TFT's training set. First, the batch size is 128, which would take about 5–6 hours to freeze for the entire training process, let alone run every epoch. Second, LSTM, multi-head attention and the selection network are all part of the forward pass of the TFT, which is very complex. As such, TFT takes the default running average training loss from PyTorch Lightning. It is a useful R ratio for observing convergence, but is not directly comparable to CNN+LSTM and N-BEATS R ratios, and this is emphasised wherever TFT training results are reported in Chapter 6.

5.1.5 The Generalisation Ratio and Data Difficulty

Throughout Chapter 7, the generalisation ratio R is used as a diagnostic tool for training. There are two systematic effects which cause R to differ from 1.0 and which are expected, and are not evidence of modelling problems:

Data difficulty difference. The validation period (June–November 2025) has a seasonal naive MAE of 105.63 veh/h. The training period naive MAE is roughly 116.8 veh/h – more difficult to predict due to the wider range of seasonal conditions experienced across 29 months. The validation loss will be lower (naturally) when also training and validation losses are computed on the final frozen model state, as the validation data is actually more regular. This results in R values less than 1.0, often in the range of 0.91–0.96 during the initial epochs of training. This has nothing at all to do with the model but rather the time periods chosen.

Dropout effect. When training, 10% of the neurons are randomly disabled at every forward propagation, which reduces the size of the network and therefore increases the training loss. Full network is used and dropout is disabled during validation and FrozenTrainEval. As the model approaches convergence and the difference in the difficulty of the data diminishes, this leads to $R > 1.0$ in later epochs typically 1.01–1.05. This corresponds to a good level of regularisation and is not overfitting.

Any R value significantly greater than 1.5 would suggest that the model is truly overfitting, that is, memorising the training data, at the cost of not generalising. None of these were found in any of the models in this thesis.

5.2 Model Architectures

5.2.1 CNN+LSTM

Input Representation. At every encoder time step t, the model gets a feature vector $x_t \in \mathbb{R}^{29}$ which consists of all 29 normalized features. In other words, the entire 48 hours window of the encoder is:

$$X = [x_{t-47}, x_{t-46}, \ldots, x_t] \in \mathbb{R}^{48\times 29} \tag{5.6}$$

Where t is the origin of forecast — the last hour before making predictions.

Figure 5.1 depicts the entire architectural structure of the proposed CNN+LSTM model by Asadi & Regan (2020) [3] used in this thesis for six-horizon MIMO forecasting with 48 hours encoder and 29 input variables.

1D Convolutional Block. For this 1-D CN operation, we have one signal dimension—time—and we are moving a 3-timestep kernel (k) containing 64 filters (C) along the 48 timesteps of the signal. Notably, the activation of each of these filters is the interaction of

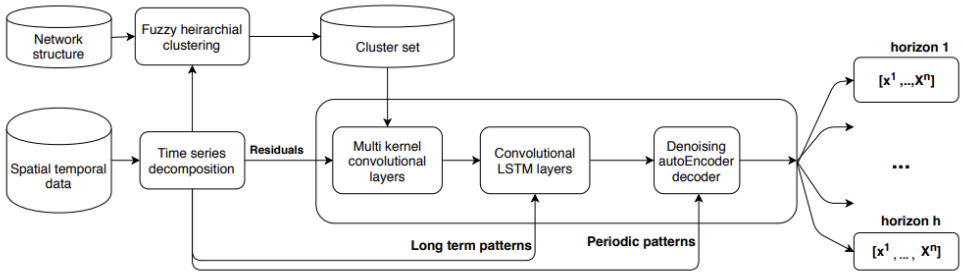


FIGURE 5.1: CNN + LSTM Architecture

some features in the 3 hours long local window, which is a "short term" traffic momentum of sorts, and the state of things at 08:00 depends on the state at 07:00 and 06:00. To perform the convolution, we transpose our input matrix, and set 29 as the dimension of channel:

$$Z_{\text{conv}} = \text{ReLU}\left(\text{BN}\left(W_{\text{conv}} *_1 X^\top + b_{\text{conv}}\right)\right) \in \mathbb{R}^{64\times 48} \tag{5.7}$$

The trained filter weights are denoted by $W_{\text{conv}} \in \mathbb{R}^{64\times 29\times 3}$, batch normalisation (Ioffe and Szegedy, 2015 [31]) by BN, and ReLU replaces negative values with zeros. The padding of 1 keeps the sequence length constant at 48. Batch normalisation normalises the convolution outputs for each time step as follows:

$$\text{BN}(z) = \gamma \cdot \frac{z - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta \tag{5.8}$$

where μ_B and σ_B^2 are the current batch mean and variance, and γ and β are the parameters learned from the current batch that can be used to scale and shift the resulting distribution. This will help stop the convolutional activations from getting too large or too small in magnitude, which will help prevent the LSTM dynamics in the following layer from becoming unstable. The output from the convolutional network is then transformed back to have time first, resulting in $H_{\text{conv}} \in \mathbb{R}^{48\times 64}$.

LSTM block. The convolutional representations of 48 hours in sequence are fed into the Long Short-Term Memory network (LSTM) (Hochreiter and Schmidhuber, 1997 [2]). It has two states: hidden state $h_s \in \mathbb{R}^{128}$ and cell state $c_s \in \mathbb{R}^{128}$ at each timestep s. The state of the cell is what constitutes the memory of the network, and its memory can hold the information about hour $t-47$ up until hour t, whereas the main advantage

of using an LSTM network is that it does not experience vanishing gradient problem when being trained with long sequences, unlike simple recurrent networks (Pascanu et al., 2013 [32]). The update equations for each timestep are:

$$f_s = \sigma(W_f h_s^{\text{conv}} + U_f h_{s-1} + b_f) \qquad \text{(forget gate — how much past memory to retain)} \tag{5.9}$$

$$i_s = \sigma(W_i h_s^{\text{conv}} + U_i h_{s-1} + b_i) \qquad \text{(input gate — how much new information to write)} \tag{5.10}$$

$$g_s = \tanh(W_g h_s^{\text{conv}} + U_g h_{s-1} + b_g) \qquad \text{(candidate cell content)} \tag{5.11}$$

$$o_s = \sigma(W_o h_s^{\text{conv}} + U_o h_{s-1} + b_o) \qquad \text{(output gate — what to expose)} \tag{5.12}$$

$$c_s = f_s \odot c_{s-1} + i_s \odot g_s \qquad \text{(cell state update)} \tag{5.13}$$

$$h_s = o_s \odot \tanh(c_s) \qquad \text{(hidden state output)} \tag{5.14}$$

where σ is the sigmoid function, $\odot$ is the element-wise multiplication, and W, U, b are weight matrices. The forget gate f_s learns, for example, when the sensor was in a certain "incident" at the start of the hour, how much of the previous state of the cell to forget, that is, how much of the traffic state to forget. The input gate i_s learns the amount of the current state to take in. LSTM network comprises of two layers; the output of the first layer is fed to the second layer.

MIMO forecast head. All 48 timesteps of the encoder are processed, and the last hidden state of the last LSTM layer, $h_{48} \in \mathbb{R}^{128}$, encodes the entire sequence of encoder states. It goes through a two-layer fully-connected head:

$$\hat{y} = W_2 \cdot \text{ReLU}(W_1 h_{48} + b_1) + b_2 \in \mathbb{R}^6 \tag{5.15}$$

where $W_1 \in \mathbb{R}^{64\times128}$ and $W_2 \in \mathbb{R}^{6\times64}$.

The output $\hat{y} = [\hat{y}_{t+1}, \hat{y}_{t+2}, \ldots, \hat{y}_{t+6}]$ is a set of predictions for the six z-horizons, all computed in a single forward pass. This is called MIMO (Multiple Input Multiple Output) approach which does not have the problem of cumulative error that happens in autoregressive methods, where the prediction at $t + 2h$ is based on the prediction at $t + 1h$, so if there is an error at $t + 1h$, then it carries through the rest of the horizons. The complete PyTorch implementation:

Code 5.2: CNN-LSTM forecasting model architecture

```
class CNNLSTMForecaster(nn.Module):
    def __init__(self, n_features, pred_len, cfg):
        super().__init__()
        # 1D CNN: extracts local 3h temporal patterns across 29 features
        self.cnn = nn.Sequential(
            nn.Conv1d(
                in_channels  = n_features,          # 29
                out_channels = cfg["cnn_filters"],  # 64 learned filters
                kernel_size  = cfg["cnn_kernel"],   # 3h local window
                padding      = cfg["cnn_kernel"] // 2,  # preserve length
            ),
            nn.BatchNorm1d(cfg["cnn_filters"]),
            nn.ReLU(),
            nn.Dropout(cfg["dropout"]),
        )

        # 2-layer LSTM: captures long-range sequential dependencies
        self.lstm = nn.LSTM(
            input_size  = cfg["cnn_filters"],  # 64 from CNN output
            hidden_size = cfg["lstm_hidden"],  # 128 hidden units
            num_layers  = cfg["lstm_layers"],  # 2 stacked layers
            batch_first = True,
            dropout     = cfg["dropout"] if cfg["lstm_layers"] > 1 else 0.0,
        )

        # MIMO head: 128 -> 64 -> 6 simultaneous horizon predictions
        self.head = nn.Sequential(
```

```
            nn.Dropout(cfg["dropout"]),
            nn.Linear(cfg["lstm_hidden"], 64),
            nn.ReLU(),
            nn.Linear(64, pred_len),  # pred_len = 6
        )

    def forward(self, x):
        # x: (batch, 48, 29) -- batch of encoder windows
        x = x.permute(0, 2, 1)       # (batch, 29, 48)
        x = self.cnn(x)              # (batch, 64, 48)
        x = x.permute(0, 2, 1)       # (batch, 48, 64)
        _, (h_n, _) = self.lstm(x)   # h_n: (2, batch, 128)
        x = h_n[-1]                  # (batch, 128)
        return self.head(x)          # (batch, 6)
```

5.2.2 N-Beats

Doubly Residual Topology. N-BEATS' core architectural principle is doubly residual stack. The structure is crucial for understanding the model's performance and the properties it possesses for interpretation.

Figure 5.2 illustrates the N-BEATS architecture as introduced by Oreshkin et al. (2020) [33].

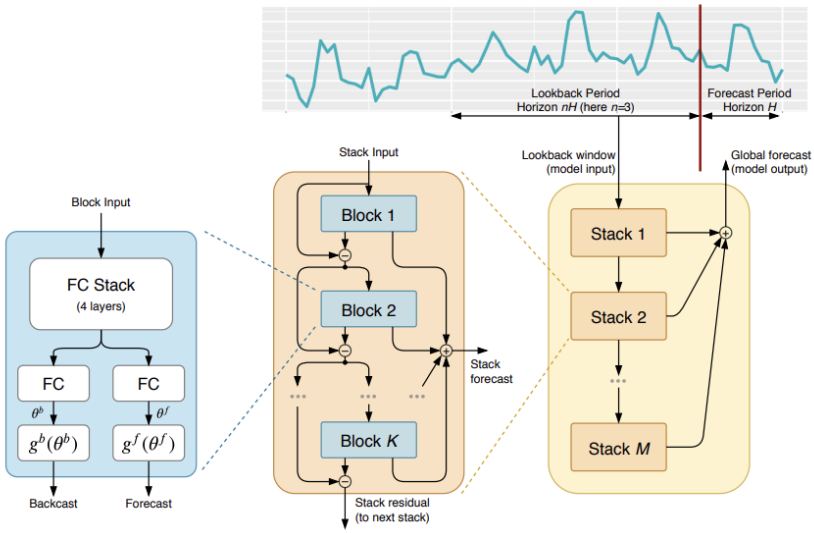


FIGURE 5.2: N-BEATS Architecture

There are 4 blocks in the trend stack and 4 blocks in the seasonality stack, and they are stacked one after the other. Each block works the same way: It gets an input signal (at first the complete encoder window, later a residual), tries to reproduce as much of this signal as it can and transmits the part it could not reproduce to the next block. This means that the first block deals with the dominant pattern, the second block with the remaining, etc. and so forth, which is hence a progressive simplification.

There are two outputs from each block. The backcast $\hat{x}$ is the reconstruction of the block's input, which is what it has learned to explain about the incoming signal. The final prediction made by the block is the result of the contribution made by the forecast $\hat{y}$. The output of the previous block is:

$$x_{\text{res}} = x - \hat{x} \tag{5.16}$$

This connection is residual, where each subsequent block receives a signal that is increasingly simpler, as the most dominant structure gets removed in the preceding block. The last prediction is a total of all the contributions to the forecast from all blocks and both stacks:

$$\hat{y}_{\text{total}} = \sum_{s \in \{\text{trend, season}\}} \sum_{b=1}^{4} \hat{y}^{(s,b)} \tag{5.17}$$

The additive structure is an exact structure, and the trend and seasonal components of the resulting forecast can be independently added together to obtain the final forecast. N-BEATS' decomposition is direct and not an approximation as would be done in a post-hoc method.

Block internal structure. These four fully connected layers with ReLU activation are applied to an input in each block:

$$h_1 = \text{ReLU}(W_1 x + b_1), \quad h_2 = \text{ReLU}(W_2 h_1 + b_2), \ldots \tag{5.18}$$

Then, two basis expansion coefficient vectors θ_b (backcast) and θ_f (forecast) are projected using linear layers onto the final hidden state h_4. These coefficients are used with fixed basis matrices for analytical work, generating outputs of backcast and forecast.

Trend stack, from polynomial basis. The trend stack's basis matrices are constructed from polynomial functions. The entries of the forecast basis matrix $V^f_{\text{trend}} \in \mathbb{R}^{F\times 3}$ are:

$$[V^f_{\text{trend}}]_{i,j} = (F_i)^j, \quad j = 0, 1, 2, \quad i = 0, 1, \ldots, F-1 \tag{5.19}$$

where $F = 6$ is the length of the prediction. The contribution of the trend block to the forecast is thus limited to a quadratic curve over the 6 hour time horizon: it can model the background traffic level (the constant term), a linear trend (the degree-1 term), but cannot model sharp curves (the degree-2 term). The constraint is not a limitation in the field of trend modelling because the background traffic level changes are generally slow but it is actually a structural regulariser that prevents the learning of arbitrary patterns by a single block.

A seasonal stack using the Fourier basis. The seasonality stack is built based on 7 harmonic pairs of the Fourier basis. The following entries are found in the forecast basis matrix, $V^f_{\text{season}} \in \mathbb{R}^{F\times 15}$:

$$[V^f_{\text{season}}]_{i,k} = \begin{cases} 1, & k = 0, \\ \cos\left(\frac{2\pi k t_i}{F}\right), & k \text{ odd}, \\ \sin\left(\frac{2\pi k t_i}{F}\right), & k \text{ even}. \end{cases} \tag{5.20}$$

There is enough flexibility in 7 harmonics to capture the asymmetric double-peaked diurnal traffic profile of Madrid, which is sharper in the mornings and broader in the evenings. Periodic patterns are only allowed to be produced by the seasonality blocks as enforced in the Fourier constraint, which keeps the trend-like structures in the trend stack from being absorbed by the seasonality blocks.

Weight sharing. Each stack consists of four blocks, and each of these blocks has the same basis matrices V_b and V_f, but has different fully connected weight matrices. This weight sharing works as an extra structural regulariser, which helps the individual blocks to not "cheat" on the basis constraint by learning different functional families for different blocks. Oreshkin et al. (2020) [33] demonstrated that weight sharing generally led to better generalisation on various forecasting tasks.

Univariate input. N-BEATS only takes the sequence:

$$x = [\tilde{x}_{t-71}, \tilde{x}_{t-70}, \ldots, \tilde{x}_t] \in \mathbb{R}^{72} \tag{5.21}$$

where $\tilde{x}_t$ is the z-score normalised traffic intensity per sensor at hour t. The first fully-connected layer maps this 72-dimensional vector to the block hidden dimension. The total number of trainable parameters is 51,870,720, and is intentionally asymmetric (Fourier approximation requires more capacity than polynomial trend: trend width is 256, seasonality width is 2,048). This is a large number due to the width of the Fourier-basis seasonality stack, not architectural depth — more parameters can't make up for missing input information, which explains why N-BEATS does the worst on the test.

5.2.3 TFT

TFT's architecture could be seen as four nested mechanisms, each of which is solving a modest problem that simpler architectures are having to deal with.

The architecture of the Temporal Fusion Transformer is shown in Figure 5.3, and it has been taken from Lim et al. (2021) [5]. The architecture shows how static, time-varying known, and time-varying unknown features have been processed using variable selection networks, an LSTM encoder-decoder, and interpretable multi-head attention.

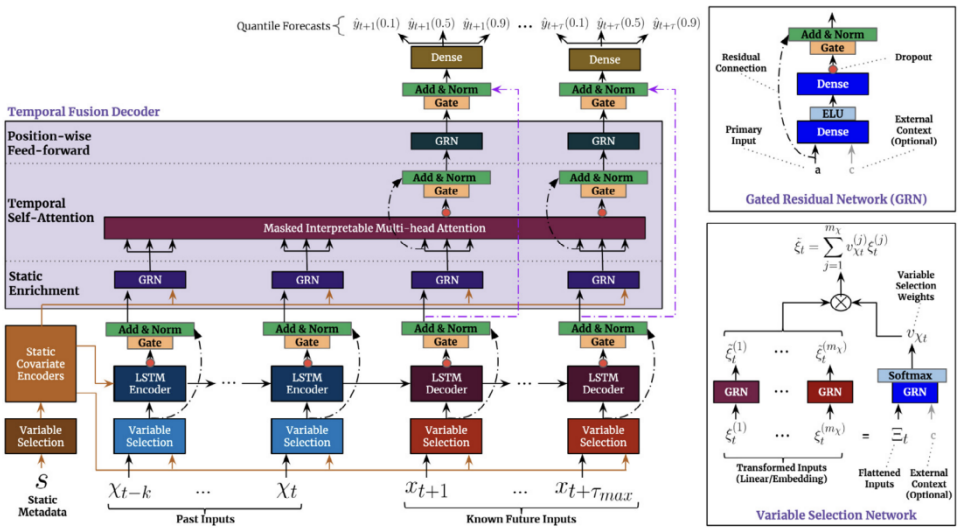


FIGURE 5.3: Temporal Fusion Transformer (TFT) Architecture

Variable Selection Networks. TFT first applies a learned gating mechanism to each feature of the input at every timestep before any sequence processing. A set of weights $\alpha \in [0, 1]^d$ are produced by a small neural network for each input position which

sum to 1 and d is the number of features. These weights are used to give weight to the features in the current calculation. The VSN is trained end-to-end with the rest of the model, and learns feature relevance from the data instead of it being programmed by the researcher. For a set of d inputs $\{x^{(1)}, \ldots, x^{(d)}\}$ each passed through a Gated Residual Network (GRN), the weights of the variable selection are:

$$\alpha = \text{Softmax}\left(W_{\text{vsn}} \cdot [x^{(1)} \| \cdots \| x^{(d)}] + b_{\text{vsn}}\right) \tag{5.22}$$

The selected representation is the weighted sum:

$$\tilde{x} = \sum_{j=1}^{d} \alpha^{(j)} \cdot \text{GRN}_j(x^{(j)}) \tag{5.23}$$

The three different VSNs used in the TFT is one for the static covariates (road sub-class, geographical coordinates, POI distance), another for the time-varying unknown inputs while the encoder (lag features, past traffic intensity), and a third for the time-varying known inputs while the decoder (calendar features, holiday flags). The Chapter 8 results for these weights are consistent: VSN has learned a meaningful ranking: The top two static features are distance to railway station and the road sub-class and the top two encoder features are relative position in the encoder window and day-of-week encoding.

Gated Residual Networks. All features are fed into a VSN after processing with a GRN. A GRN uses a two-stage feed forward transformation and then a learned gating mechanism:

$$\text{GRN}(x) = \text{LayerNorm}\left(x + \text{GLU}\left(W_2 \cdot \text{ELU}(W_1 x + b_1) + b_2\right)\right) \tag{5.24}$$

where GLU is a Gated Linear Unit — a sigmoid gate which can suppress the whole output of the transformation to near zero, when no information is relevant:

$$\text{GLU}(h) = \sigma(W_3 h + b_3) \odot (W_4 h + b_4) \tag{5.25}$$

The gating is crucial because it enables the GRN to fully block a feature's contribution when it does not help to make the prediction, not just downplay its contribution. This is a stronger regularisation than dropout or L2 regularisation which lower the weight of all features equally.

Sequence-to-sequence LSTM encoder-decoder. Once the variable selection process is done, the encoder and the decoder representations are passed to an ordinary LSTM encoder and decoder. The 48-hour history window is fed into the encoder which outputs a context state, while the decoder takes in the selected features known in the future and processes the 6 decoder timesteps using that state. The LSTM is essentially the same as the CNN+LSTM's recurrent layer, but uses a subset of the features instead of the entire feature set.

Interpretable Multi-Head Attention. At each time step into the future, LSTM decoder processes the data, and then TFT uses a multi-head attention mechanism to enable LSTM decoder to revisit the entire encoder history of 48 hours. An attention weight is calculated for every time step of the decoder and position of the encoder:

$$A = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \tag{5.26}$$

where the key dimension is $d_k = 16$, Q (query) is sent from the decoder and K (key) and V (value) are sent from the encoder. Shared attention head is the design choice that makes the weights readable as a single average attention profile over the encoder and why the Chapter 9 attention analysis is possible. The results in Chapter 8 validate that TFT found the daily periodicity independently with attention, that is, the most attended encoder positions are $t - 23h$ and $t - 24h$ (yesterday same hour) irrespective of road sub-class.

Input categories. TFT identifies three classes of input, and treats them as such:

Static covariates: Properties of the sensor that do not vary with time (road sub-class, coordinates, distances to POIs). They are stored in one place and shared across all the encoder and decoder timesteps through a 'context vector'.

Known inputs that vary with time — features that are known, but change as a function of time (calendar encodings, `is_holiday`, `is_rush_hour`). These go in to the decoder, and enable TFT to utilize future calendar knowledge for prediction.

Time varying unknown inputs — only features that have been available in the past (lag features, past traffic intensity etc.). These are only allowed in the encoder and will not leak in the future.

This separation is categorical and is enforced in the architecture, i.e., the decoder cannot have access to any unknown future input by construction, thus no risk of unintended leakage via the attention scheme.

5.3 Evaluation Metrics and Seasonal Naive Baseline

5.3.1 Evaluation Metrics

5.3.1.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) is defined in Section 5.1.3 as the training loss function.

$$MAE = \frac{1}{NF} \sum_{i=1}^{N} \sum_{h=1}^{F} \left| \hat{\tilde{y}}_{i,h} - \tilde{y}_{i,h} \right| \tag{5.27}$$

It serves the same role as the primary reported evaluation metric, computed in physical units (veh/h) after inverse transformation. An MAE of 75 veh/h means the model's predictions are wrong by 75 vehicles per hour on average.

5.3.1.2 Mean Absolute Scaled Error (MASE)

Mean Absolute Scaled Error (MASE) normalises MAE by the seasonal naive MAE for the same period and the same set of prediction windows:

$$\text{MASE} = \frac{\text{MAE}_{\text{model}}}{\text{MAE}_{\text{naive}}} \tag{5.28}$$

A MASE below 1.0 means the model beats the naive baseline. A MASE of 0.50 means the model's average error is exactly half the naive baseline's — a 50% improvement. A MASE above 1.0 means the model is outperformed by simply repeating last week's value, indicating the model has failed to learn anything useful from the data. MASE is the primary comparison metric throughout this thesis because it is scale-free, period-independent, and directly interpretable as a multiple of the simplest reasonable baseline.

5.3.1.3 Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) penalises large prediction errors more heavily than small ones by squaring the errors before averaging:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2} \tag{5.29}$$

RMSE is reported alongside MAE as a secondary metric. A large gap between RMSE and MAE for the same model indicates the presence of occasional large prediction errors — useful for diagnosing which conditions the model struggles with most.

5.3.1.4 Coefficient of Determination (R^2)

Coefficient of Determination (R^2) measures the proportion of variance in actual values explained by the model's predictions:

$$R^2 = 1 - \frac{\sum_{i=1}^{N} (\hat{y}_i - y_i)^2}{\sum_{i=1}^{N} (y_i - \bar{y})^2} \tag{5.30}$$

Where N stands for the total number of forecasts in all windows and forecast horizons, $\hat{y}_i$ is the traffic intensity in veh/h forecasted by the model for the i-th forecast, y_i stands for the traffic intensity in veh/h observed for the i-th forecast, and

$$\bar{y} = \frac{1}{N} \sum_{i=1}^{N} y_i \tag{5.31}$$

is the mean of all observations in the test dataset. The numerator is the sum of squared residuals between predictions and observations; the denominator is the total sum of squares representing the variance of the observations around their mean. R^2 ranges from 0 (model explains none of the variance — equivalent to always predicting the mean) to 1.0 (perfect prediction). Values above 0.95 indicate that the model captures the dominant temporal patterns of traffic intensity.

5.3.1.5 Symmetric Mean Absolute Percentage Error (SMAPE)

Symmetric Mean Absolute Percentage Error (SMAPE) expresses the error as a percentage of the average of predicted and actual values:

$$\text{SMAPE} = \frac{100}{N} \sum_{i=1}^{N} \frac{2\,|\hat{y}_i - y_i|}{|\hat{y}_i| + |y_i|} \tag{5.32}$$

SMAPE is reported for completeness and cross-study comparability. However it is treated with caution for C30 tunnel sensors, where near-zero traffic during scheduled closures makes the denominator $|\hat{y}_i| + |y_i|$ unstable — a prediction of 5 veh/h against an actual of 2 veh/h gives SMAPE of 85% even though the absolute error is only 3 veh/h. For C30 sensors, MAE and MASE are the reliable primary metrics.

5.3.2 The Seasonal Naive Baseline

In order to present model results, a reference baseline is required which can be compared with the value added by deep learning. The simplest forecasting method that takes into account the dominant temporal structure in traffic data is the seasonal naive baseline: to predict traffic at any future hour t for sensor i, simply use the observed value from exactly one week earlier:

$$\hat{y}_{i,t}^{\text{naive}} = x_{i,t-168} \tag{5.33}$$

For example, The baseline prediction for Tuesday 10 March 2026 at 08:00 is for an observation made at Tuesday 3 March 2026 at 08:00 at the same sensor. This takes advantage of the weekly seasonality pattern as established in Section 4.3.3, stronger than the daily (lag-24h) pattern.

There are two uses of seasonal naive baseline. First, it is a minimum competence threshold: any deep learning model that is not able to beat this trivially simple method has learned nothing that is valuable. Second, it provides the denominator for the primary scaled evaluation metric used throughout this thesis.

The naive baseline MAE was computed over the exact same prediction timestamps as each model's predictions — not over a separate or approximated population. This exact-population computation is required for the scaled metric to be mathematically valid (Hyndman and Koehler, 2006). The resulting values are:

- Validation period (June – November 2025): naive MAE = 105.63 veh/h
- Test period (December 2025 – May 2026): naive MAE = 139.20 veh/h

The 32% increase from validation to test naive MAE reflects the greater irregularity of the test period. December 2025 through May 2026 includes Christmas, New Year, and several Spanish public holidays during which weekly traffic patterns break down temporarily — the same-hour-last-week prediction performs substantially worse when the weekly rhythm is disrupted. This difference is a characteristic of the selected time periods, not of any model, and it means validation and test metrics cannot be compared in absolute terms across periods. All model comparisons throughout this thesis therefore use the period-specific MASE rather than raw MAE.

5.3.3 Per-Horizon Reporting

All metrics are reported at two levels of aggregation:

Per-horizon: separately for each forecast step $t + 1h$ through $t + 6h$. This shows how prediction accuracy degrades as the forecast horizon extends — a pattern expected in all models because uncertainty compounds over longer lead times.

Overall: the arithmetic mean of the six per-horizon values. This single-number summary enables direct model comparison but should always be read alongside the per-horizon breakdown, since two models with the same overall MASE can differ substantially in their relative accuracy at short versus long horizons.

Sub-class and peak/off-peak breakdowns are reported additionally to characterise which road types and time periods each model handles well or poorly.

Chapter 6

Exploratory Data Analysis

6.1 Purpose and Approach

It is crucial to have an understanding of the data being modeled prior to constructing any forecasting model. The patterns found as a result of the exploratory analysis actually led to the features engineered in Chapter 4, the lengths of the encoders in Chapter 7, and the interval of when the patterns were evaluated to test the model. Exploratory data analysis here has three objectives: characterize the statistical properties and the temporal structure of the traffic intensity according to the four road sub-classes; estimate the strength and periodicity of the seasonal patterns to support the model lag features and season length; and explore the utility of the weather data as a possible covariate.

The analysis is conducted on the cleaned data set obtained from the pre-processing pipeline described in Chapter 4, following stuck-sensor detection and two-tier imputation procedure. All statistics and plots are therefore based on the final (model ready) data and not the raw sensor output. The entire data set was utilized to obtain general information of all the data for this analysis.

6.2 Traffic Intensity Distributions by Road Sub-Class

The main three road sub-classes are not just different by their average traffic levels, but by the full form of their traffic intensity distributions and these differences have significant meanings in modelling. Table 6.1 summarizes the key descriptive statistics of hourly traffic intensity (in vehicles per hour) for each sub-class.

Table 6.1: Traffic intensity descriptive statistics by road sub-class (Veh/h)

ELEMENT_TYPE	rows	avg_traffic	median_traffic	p95	p99	max_traffic
URB	127664838	268.348658	142.75	956.0	1678.25	91368.333333
C30	3678043	961.448929	480.00	3438.0	5034.00	19614.000000
M30	8634412	1273.656486	807.25	4058.5	5415.50	8215.250000

These distributions have a number of characteristics to note. The distribution of peak hours for URB has the largest difference between its mean (around 268 veh/h) and median (around 143 veh/h), suggesting a right skewed distribution with a small proportion of high intensity peak hours creating a larger than usual mean. This is the expected behavior for residential and secondary urban roads with low levels of traffic throughout the majority of the day, but high levels of traffic during the morning and evening rush hour. The M30 distribution is not as skewed, with a mean of 1,274 veh/h, which is closer to the median of 807 veh/h, representative of the motorway's higher baseline level of usage throughout the day, even outside peak hours.

The median of 480 veh/h for the C30 is significantly lower than the mean of 961 veh/h. These differences partly reflect the operating characteristics of the tunnel (speed limits, merge points, ventilation constraints) in reducing the throughput but are also because of the episodic nature of C30 traffic which can be near zero when a tunnel is closed or undergoing maintenance work, and because, as described in Chapter 4, a larger proportion of values are imputed than observed. These near or zero observations for closure will skew the mean and median results up for C30 and will result in the very high SMAPE results obtained in the results for the modelling runs: error metrics based on percentages are unstable when they approach zero.

6.3 Temporal Patterns

6.3.1 The Diurnal Cycle

Madrid's traffic data is dominated by the daily cycle, the most prominent short-term structure, which is a repeating 24-hour structure that is due to the rhythm of human activity. In all four road sub-classes, the traffic intensity during the weekdays has a characteristic double-peaked pattern, with a morning peak related to commuting and school travel and an evening peak related to the return trip and leisure activity.

The morning peak time is seen to have a strong concentration between approximately 06:00 and 09:00 across sensors URB_normal, M30 and C30, which is the time when most workers, students and commuters are travelling to work. The evening peak is

concentrated between around 17:00 and 20:00 and is the return journey. It is a common feature in European urban traffic networks (Asadi and Regan, 2020 [3]; Vélez-Serrano et al., 2021 [9]) but the exact times of the two peaks, and their relative heights, are specific to the road type within the Madrid network.

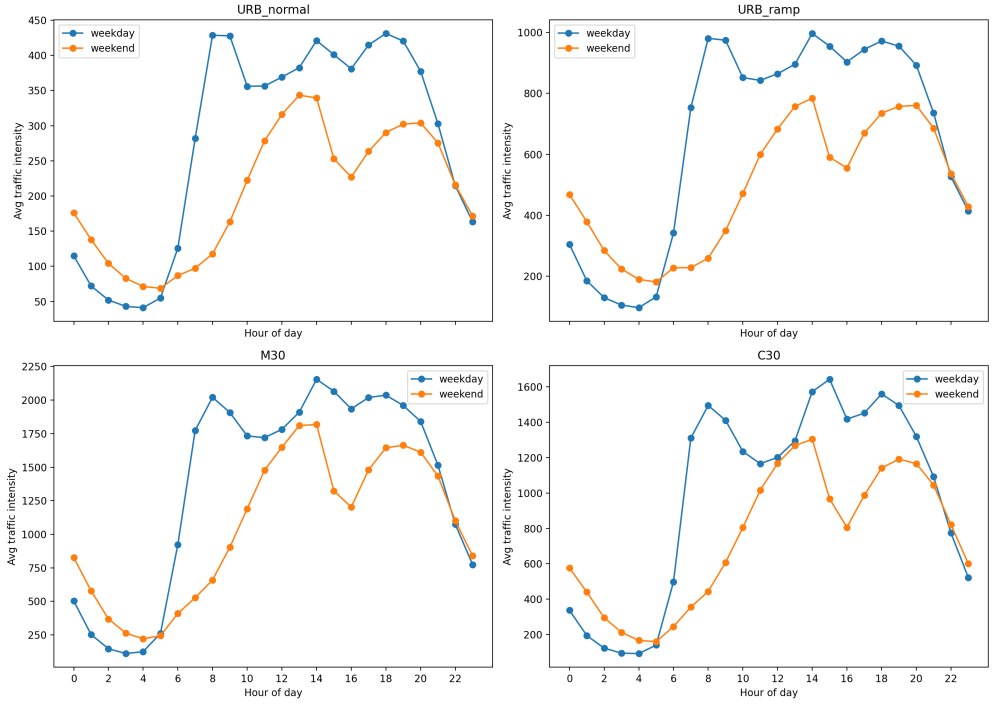


FIGURE 6.1: Average hourly traffic intensity profiles by road sub-class and day type

For URB_normal sensors, the peak in the morning is typically sharper and narrower compared to the evening peak. This asymmetry can be physically interpreted as the fact that when people go to work in the morning, they tend to go at given times (depending on the work and school routines), while at the evening they tend to go at different times giving a longer window of the journey. Patterns are very different on the weekend than they are during the week. The morning peak almost vanishes — or becomes later to about 10:00–12:00 — while the evening peak becomes more diffuse and is spread across the night. M30 traffic is generally more fragmented during the weekends, with more journeys occurring in the midday and early afternoon, which is likely to be for leisure or shopping instead of commuting. Based on these observed time windows, the `is_rush_hour` flag was defined as an example feature (Section 4.3.2) that was set only during the weekday time window, which include the time periods 06:00–09:00 and 17:00–20:00. The exploratory analysis verified that these windows were able to correctly locate the major proportion of peak-period observations in each of the four sub-classes.

6.4 Weekly Patterns

Weekly periodicity is also strong for traffic intensity, in addition to the daily cycle. The 24-hour profile is repeated each day, but with some changes that have a systematic pattern; and weekdays differ from weekends in systematic ways and weekends differ from weekdays in systematic ways. This structure can be visualized in one graphic by a heatmap of traffic intensity, averaged over time, on an hourly and daily basis.

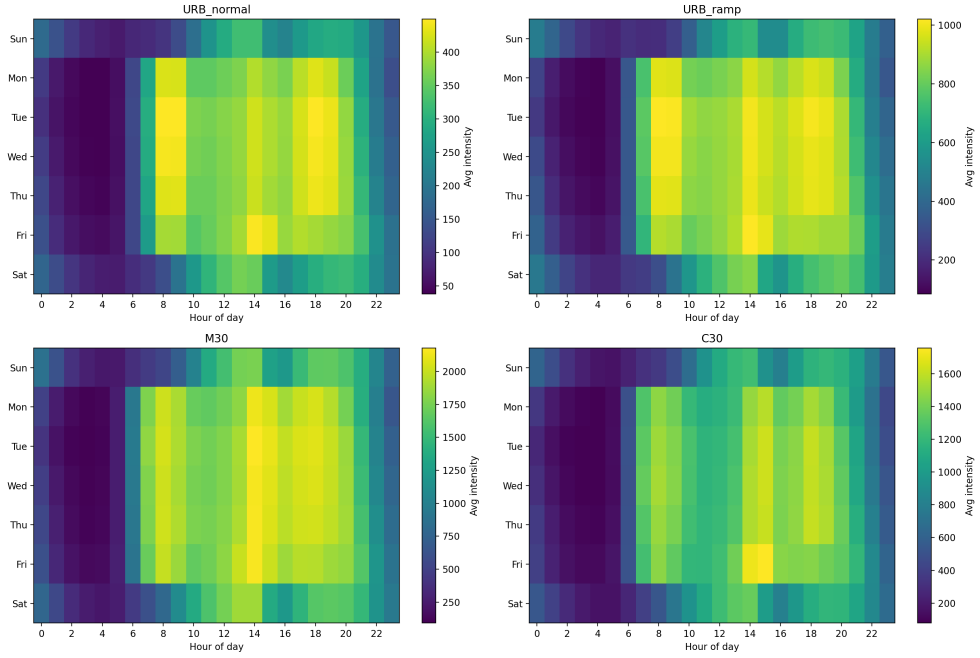


FIGURE 6.2: Weekly heatmap of average traffic intensity by road sub-class

The heatmaps show temporal patterns which would not be apparent from statistics alone. For all sensor subclasses, overall traffic is at the lowest level in the night and first few hours of the morning, with a distinct rise from approximately 06:00–08:00. Overall, the intensities are consistently higher on weekdays than on weekends, and the lowest overall intensity is found on Sunday. Patterns are quite similar on the weekdays, with a very broad and strong peak on Friday afternoon and evening in the case of M30. The heatmaps don't show a steady increase between Monday and Thursday, they seem to show a consistent difference within the weekly trend.

Having a strong weekly pattern in the heatmaps is directly affecting the model design. This explains the reason behind the inclusion of `lag_168h` (same hour from the previous week) as an input and explains the findings of Vélez-Serrano et al. (2021) [9] who performed Kruskal-Wallis test on the weekly/daily seasonality of the traffic data in Madrid and found out that the weekly seasonality was statistically significant. The

model with the same-hour-last-week observation has a strong prior for what the traffic is likely to be at a given time and the seasonal naive baseline (a model that simply predicts $y(t-168)$) turns out to be a remarkably competitive reference, with a validation-period MAE of 105.63 veh/h and a test-period MAE of 139.20 veh/h without any learning.

6.5 Public Holiday Effects

The pattern of holidays is systematic and shows a departure from the regular weekly pattern during public holidays. Spanish national holidays and Madrid regional holidays, in general, exhibit traffic levels quite similar to those on a Sunday and not like those on a weekday, although the amount of suppression depends on the type of holiday.

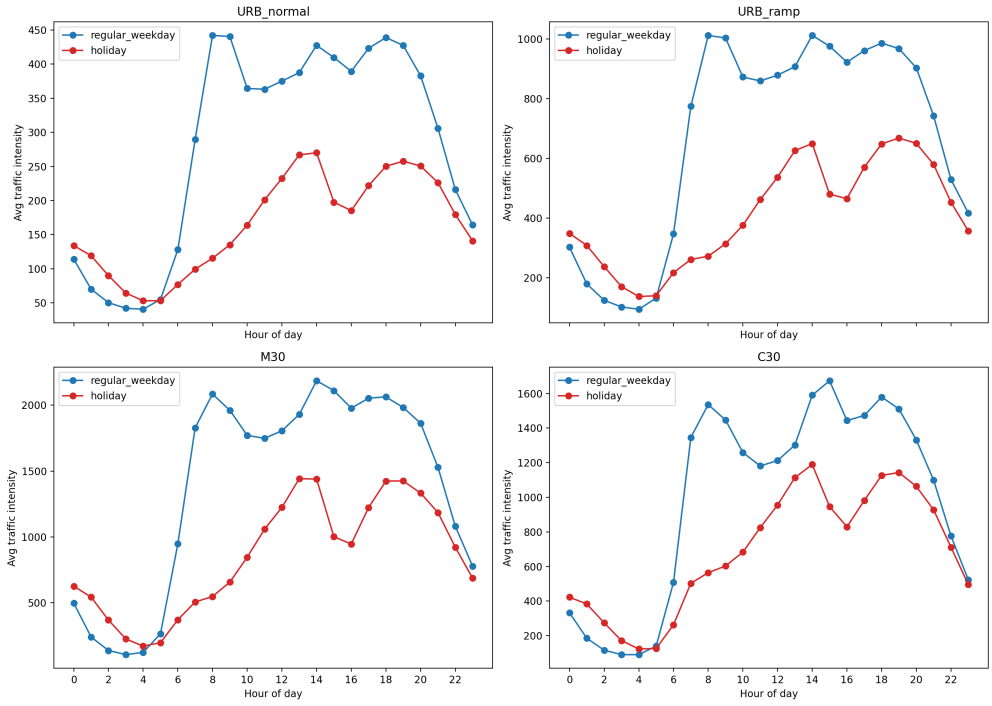


FIGURE 6.3: Average traffic intensity comparison between public holidays and regular days by road sub-class

Despite this systematic deviation, the model feature set included the binary feature `is_holiday` (Section 4.3.2) to capture this systematic deviation. A drawback of this is that it applies the same rules to all the holidays, whether it's a "rush" public holiday or a festive religious day.

Chapter 7

Experimental Results

This chapter will discuss the process of training of three deep learning models, the architecture of these models, and how they are evaluated on the test data; these three models are CNN + LSTM, N-BEATS, and the Temporal Fusion Transformer (TFT). The order in which the models have been presented is chronological to their development, with CNN + LSTM being the first one to be developed, followed by N-BEATS and TFT.

They were all trained on the same stratified sample of 200 sensors, described in Chapter 3, and normalised with the same per-sensor z score pipeline, described in Chapter 5. They were also tested during the same time period (December 2025 - May 2026).

7.1 CNN+LSTM

7.1.1 Context and Motivation

The traffic intensity at a given sensor will change over at least two different time periods. The number of cars passing an area over a 1-3 hour period will typically be a gradient. For example, if a sensor has a traffic intensity of 600 veh/h at 08:00, then it's extremely improbable that it will drop to 200 veh/h at 08:30, unless there was something abnormal that occurred. This is because of the inertia in traffic flow since vehicles already on the road will take some time to gather or spread out. Traffic volume is known to have a predictable daily pattern, with 24 to 168 hours.

Until now, the methods employed for traffic forecast in deep learning focused on the short term. The recurrent networks like Long Short-Term Memory (LSTM) introduced by Hochreiter and Schmidhuber (1997) [2] used to be good for dealing with sequences sequentially and were able to deal with long terms of time relations, but were not

able to consider the local interaction within the input. Even though the convolutional models, such as ResNet of Vélez-Serrano et al. (2021) [9], might be able to effectively learn some local temporal and spatial relations, they failed to acquire the sequence information retention effectively. Hence, it was essential to fuse the CNN and LSTM into one network, as each method stood alone had a few weaknesses. Asadi and Regan (2020) [3] have found that if the convolution operation is applied to the inputs and then the LSTM network is applied to it, the performance is better than each model used for traffic forecast alone.

The baseline of the first architecture in our experimental pipeline is the CNN+LSTM in this thesis. It takes all 29 of the input features for each time step in the encoder; that is, the 28 engineered features mentioned in chapter 4 (Table 4.5) along with the normalized target (target_norm) representing the traffic intensity, which serves as the autoregressive input in the encoder sequence. This includes both static road characteristics, calendar features, lagged inputs, moving averages, POI proximity distances, and traffic state.The task of this architecture is to demonstrate how good the forecasting results would be using an ordinary convolutional-recurrent architecture.

The CNN+LSTM architecture is described in Section 5.2.1.

7.1.2 Training Setup and Loss Function

The network is trained to minimise MAE between the predictions and the z-score of the target value across all 6 horizons at once:

$$L = \frac{1}{N \cdot 6} \sum_{n=1}^{N} \sum_{h=1}^{6} \left| \hat{\tilde{y}}_{n,h} - \tilde{y}_{n,h} \right| \tag{7.1}$$

where N is the batch size, $\hat{\tilde{y}}_{n,h}$ is the predicted z-score at prediction window n at horizon h and $\tilde{y}_{n,h}$ is the actual z-score.

Adam (Kingma and Ba, 2015 [34]) was used with initial learning rate of $1e-4$. ReduceLROnPlateau was used to decrease learning rate by half after 4 epochs no validation set improvement. Gradient clipping with maximum norm of 0.1 was used to avoid exploding gradients in the LSTM from back-propagating through multiple timesteps (Pascanu et al., 2013 [32]):

Code 7.1: Loss function, optimizer, learning-rate scheduler, and gradient clipping

```
criterion  = nn.L1Loss()    # MAE -- robust to hardware-fault
    outliers
```

```
optimizer  = torch.optim.Adam(model.parameters(), lr=1e-4)
scheduler  = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=4
)
# Applied after each backward pass:
nn.utils.clip_grad_norm_(model.parameters(), max_norm=0.1)
```

The FrozenTrainEval algorithm presented in Section 5.1.4 is used in the training loop. After performing weight updates for each epoch, the network switches to evaluation mode (no dropout), and a forward pass through the entire training dataset evaluates the frozen training loss in the exact same manner that the validation loss is evaluated. It results in the directly comparable generalisation ratio $R = val_loss/train_loss_frozen$, bringing CNN+LSTM's training monitoring in line with N-BEATS final version.

History of versions. Two training schemes were tested. With the learning rate of $1e-3$, Version 1 oscillated – with validation loss ranging from 0.163 to 0.181 over 19 epochs, with the lowest being 0.1632 at epoch 4, and no improvement after that. The Adam optimizer was making large steps that were too much to keep reaching the minimum of the loss function and land at a new point every epoch. This problem was completely resolved by reducing learning rate to $1e-4$ in Version 2.

Table 7.1 outlines the final configuration of CNN+LSTM network architecture used in this study.

Table 7.1: CNN+LSTM final configuration

Parameter	**Value**	**Justification**
Encoder length	48h	
Prediction length	6h	Matches N-BEATS v7 and TFT v3
CNN filters	64	Following Asadi and Regan (2020)
CNN kernel size	3h	Local 3-hour temporal window
LSTM hidden size	128	Following Asadi and Regan (2020)
LSTM layers	2	Hierarchical temporal representation
Dropout	0.1	Matches TFT v3 for consistent regularisation
Learning rate	1e-4	Corrected from v1 (1e-3 caused oscillation)
Batch size	128	
Max epochs	150	Budget — early stopping triggered at epoch 44
Patience	15	Allows LR scheduler to act before stopping
Gradient clip	0.1	LSTM gradient explosion prevention
Loss function	MAE on z-score	Identical to N-BEATS v7 and TFT v3
Trainable parameters	245,830	—
Training time	212 min (3.5h)	NVIDIA RTX A2000 12GB

7.1.3 Training Behaviour

Training loss started off at 0.1902 and dropped down to 0.1388 by epoch 44. The validation loss went down from 0.1806 to 0.1460 at epoch 29. During the training, learning rate was decreased four times (epochs 19, 25, 34, 39) with improvement after each learning rate decrease and plateaus. Early stopping algorithm was invoked after 15 epochs, at epoch 44 the learning rate was $3.13e-6$ which is almost equal to zero, so it was not an early stopping.

The average for the whole period (44 epochs) was Generalisation ratio R value of 1.024. The difference in difficulties between training and validation epochs caused R to be slightly below 1.0 in the early epochs. From epoch 19 R started to increase to values that are just above 1.0, which is caused by the concept of drop-out in training. No signs of divergent validation loss with steady decreasing trend.

The plots of the training loss and the validation loss, as well as the generalisation ratio R are shown in Figure 7.1. The left hand plot shows the staircase decrease in response to the decrease in LR. The R ratio is seen to be increasing on the right side from below 1.0 to stay in a band around 1.02 from epoch 19 onwards, indicating good regularisation.

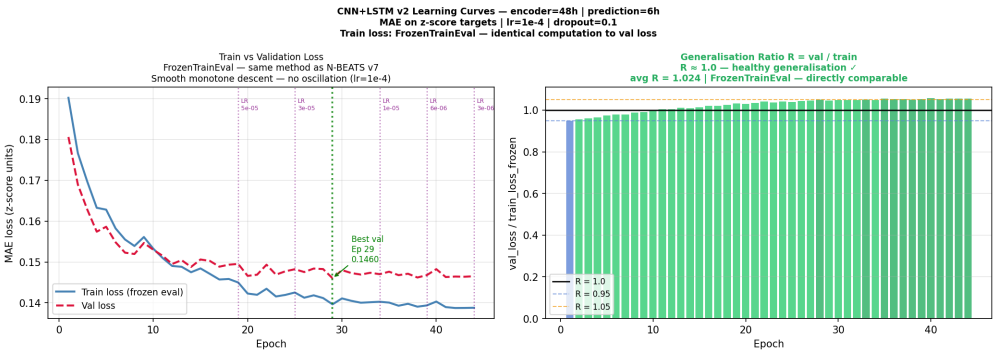


FIGURE 7.1: CNN+LSTM learning curves

7.1.4 Test Results

The CNN+LSTM v2 model performance was evaluated for the test period December 2025 - May 2026, with the same number of sensors for a stratified testing, as was done for all other models in this thesis. All forecasts have been inverse transformed back to vehicles/h with only per-sensor stats, which were derived from the training data.

All six forecast horizons are better than a naive seasonal forecast. The MASE=0.4864 metric score is achieved at the $t+1h$ forecast, with an 51.4% improvement in comparison to the traffic value forecast last week. The model performs better for $t+6h$ forecast period

TABLE 7.2: CNN+LSTM v2 per horizon test performance

Horizon	**MAE (veh/h)**	**RMSE (veh/h)**	**MASE**	**SMAPE (%)**	R^2	**Naive MAE**
$t+1h$	67.71	151.12	0.4864	26.77	0.9782	139.21
$t+2h$	77.67	181.43	0.5578	28.07	0.9686	139.24
$t+3h$	81.61	191.57	0.5860	28.68	0.9649	139.26
$t+4h$	83.77	197.09	0.6014	29.02	0.9629	139.29
$t+5h$	85.04	200.64	0.6105	29.12	0.9616	139.31
$t+6h$	86.15	203.53	0.6183	29.00	0.9604	139.33
Overall	**80.33**	**187.56**	**0.5767**	**28.44**	**0.9661**	139.20

(6 hours ahead and a lot of uncertainty) with 38.2% accuracy (which is MASE=0.6183) than the naive baseline. The deterioration in MASE model between $t+1h$ and $t+6h$ is same across all six horizons as 0.132.

This difference is over 2 times as much as MAE (80.33 veh/h) and RMSE (187.56 veh/h) larger than the normal range (80 veh/h) indicates the presence of large prediction errors. These big errors are due to two specific reasons: (i) traffic incidents (accidents, road closures) that cannot be detected within the 48-hour window used in encoder, thus making them impossible to predict; (ii) M30 motorway sensors during times of high volatility, as can be seen from the subclass-wise decomposition provided below.

For all the horizons, the R^2 value of more than 0.96 indicates that the model accounts for over 96% of the variance. The achieved result is good, given the variety of traffic intensity on the sensors in the considered network (residential streets, motorway ramps, tunnels, ring motorways), which leads to different dynamics in traffic.

The MAE and MASE per horizon are shown in figure 7.2 against the seasonal naive baseline. In the figure, it can be noticed that the blue bars start to rise slowly as the length of the horizon is increased while the red dashed line corresponding to the naive MAE ($\approx$ 139 veh/h) remains mostly unchanged.

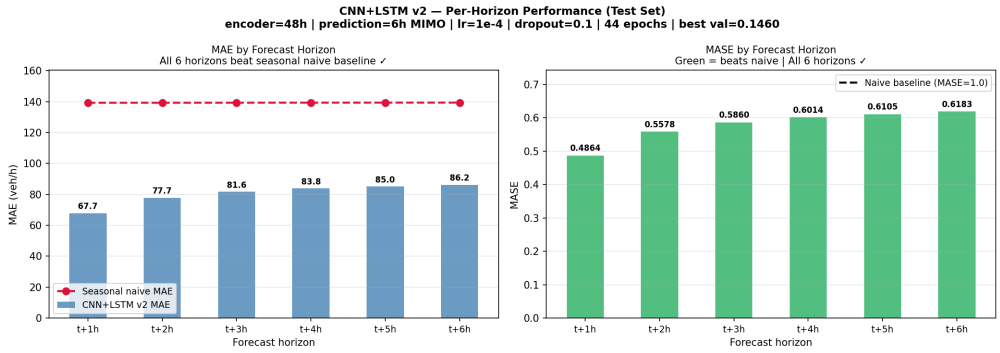


FIGURE 7.2: CNN+LSTM v2 per-horizon test performance

The test results for the overall performance are analyzed again in terms of the four classes of road. Table 7.3 shows the performance of the CNN+LSTM v2 on the four classes of road.

TABLE 7.3: CNN+LSTM v2 per road sub-class test results

Sub-class	**MAE (veh/h)**	**RMSE (veh/h)**	**MASE**	**SMAPE (%)**	R^2	**Windows**
URB_normal	30.45	62.45	0.2188	17.45	0.9674	218,100
URB_ramp	62.61	123.88	0.4498	16.91	0.9519	213,830
M30	136.78	292.45	0.9826	19.37	0.9558	218,100
C30	91.12	191.89	0.6546	59.83	0.9676	217,970

An analysis of the sub-classes indicates the main structural weakness of the model. For URB_normal sensors (residential and secondary urban street sensors with well-behaved diurnal patterns), MASE=0.2188 is 78% better than the naive baseline. The 29-feature multivariate input adequately characterizes the temporal characteristics of urban residential traffic during the previous 48 hours.

The sensors on the M30 motorway are quite a different kettle of fish. CNN+LSTM only achieves a 1.7% accuracy improvement over the previous week's motorway traffic, which is not practically significant (with MASE=0.9826). M30 traffic could be significantly affected by incidents, activation of variable speed limits managed from Madrid traffic control center, and spatial flow relocations to/from neighboring M30 road segments. None of these may necessarily produce distinct traces in the 48-hour log of a single sensor. This is a structural information ceilings, not a failure to train or overfitting of the model. Gómez and Ilarri (2025) [18] found similar degradation in motorway sub-classes from a multivariate deep learning approach, while Asadi and Regan (2020) [3] state that the volatility of motorway sensors is significantly higher in comparison to urban sensors. It directly drives TFT's spatial covariates and variable selection mechanism, which leads to an improvement of M30 MASE to 0.9183 (Section 7.3).

This C30 SMAPE of 59.83% is an artefact of measurement. Predicted traffic is 5 veh/h, actual traffic is 2 veh/h: when both traffic volumes are low, near to zero, for example, during scheduled closure of the tunnel, C30 sensors report near zero traffic, and the denominator of the SMAPE equation (sum of predicted and actual traffic values) becomes numerically unstable, although the absolute error is only 3 veh/h. The MASE value of 0.6546 is reliable for C30, which indicates 35% improvement over naive.

The CNN+LSTM v2 model is tested for its peak and off-peak performance to determine the level of prediction accuracy on different times of traffic. The test results can be found in Table 7.4.

Table 7.4: CNN+LSTM v2 peak vs off-peak test results

Period	**MAE (veh/h)**	**RMSE (veh/h)**	**MASE**	R^2	**Windows**
Morning peak (06–09h)	102.55	230.63	0.7367	0.9570	144,865
Evening peak (17–20h)	78.58	165.73	0.5645	0.9715	144,466
Weekend	69.06	155.49	0.4961	0.9650	247,259
Off-peak weekday	79.79	195.56	0.5732	0.9665	413,764

The morning peak is the worst forecasting period, with MASE=0.7367 being much higher than MASE of the evening peak (0.5645). This asymmetry is not purely arbitrary: onset time and peak height of the morning rush hour will vary based on the status of the school term, the severity of the weather, and pre-holiday adjustments, which are a subset of the data conveyed in the multivariate feature set, but not entirely captured; this data is partially captured as the binary features is_holiday and is_rush_hour. The traffic condition develops over the span of the day; the peaks start slower in the afternoon, allowing the 48-hour encoder to more clearly see the condition developing towards the peak condition. The difference between morning peak and evening peak (MASE) is the same for all three models (0.7367 versus 0.5645) and is not a limitation imposed by a specific model, but rather the natural asymmetry of the predictability of traffic peaks.

The best period result for CNN+LSTM is obtained at weekend with MASE=0.4961. Absent the school-run/commuter variability, weekend patterns are more regular and the is_weekend variable explicitly indicates the traffic regime to the model for each forecast horizon.

Furthermore, as shown in Figure 7.3, a sample forecast is provided for each road subclass, including both the 48-hour encoder sequence and 6-horizon forecast in one figure.

7.2 N-Beats

7.2.1 Context and Motivation

The external 28 features are provided to the CNN+LSTM model and TFT which includes Calendar information, lag variables, spatial properties and distance to points of interest. This poses a relevant and scientific question: What contribution do additional features make to their performance and what does the temporal modeling architecture make? To answer this question, a model must only be shown the raw traffic intensity sequence and no other data, and the performance baseline it sets must be a temporal one.

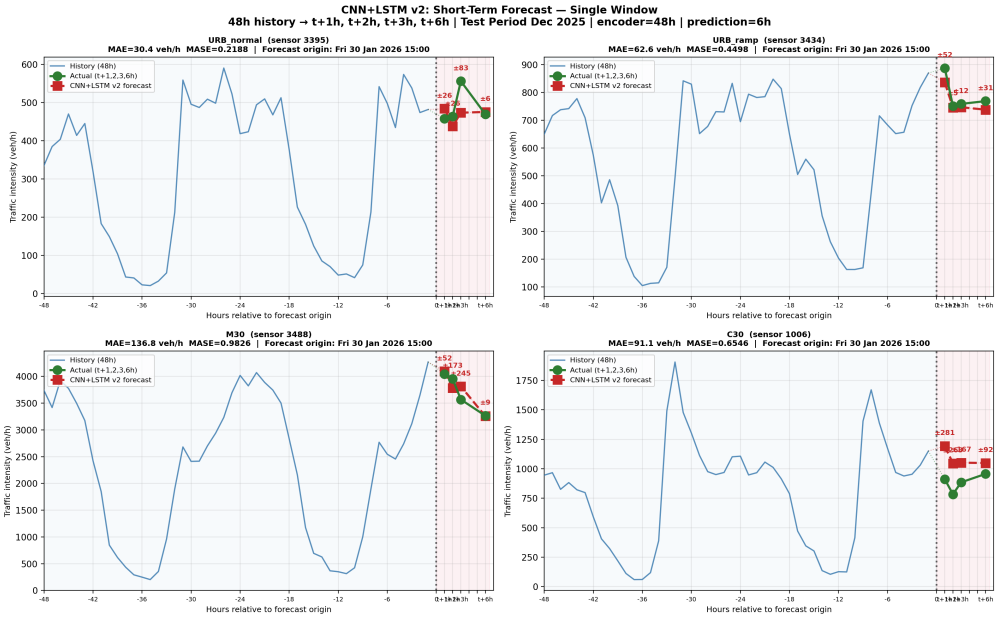


FIGURE 7.3: CNN+LSTM v2 prediction snapshot across four forecasting horizons

Oreshkin et al. (2020) [33] at ICLR have introduced a method, called N-BEATS (Neural Basis Expansion Analysis for Interpretable Time Series Forecasting), especially for this purpose. It is a pure univariate model, with only the historical series of the traffic intensity of a single sensor and which generates multi-horizon forecasts, without any external features. In addition to being used as a baseline, N-BEATS was engineered to be interpretable: the internal computation is constrained to polynomial and Fourier basis functions, which allows for the separation of the forecast into a trend component and a seasonality component, and it is possible to visualize and understand these two components separately.

The N-BEATS architecture is described in Section 5.2.2.

7.2.2 Experimental Progression and Final Configuration

The N-BEATS experiments have been developed into seven separate versions, each correcting for a shortcoming found in the earlier version. The progression of four versions is summarised in Table 7.5.

The most significant change, a 13.5% drop in validation MASE from v5 to v6, was the introduction of weight decay in v6. The generalisation ratio R was about 0.91, if no weight decay was used, which means that the validation period was significantly easier for the model than the training period. To enhance generalisation, L2 regularisation

TABLE 7.5: N-BEATS experimental progression (validation results)

Version	Key change	Best val loss	Val MASE	Epochs
v4	Extended prediction 3h → 6h; FrozenTrainEval introduced	0.1923	0.8676	100
v5	ReduceLROnPlateau scheduler added	0.1884	0.8492	100
v6	Weight decay $\lambda = 1e-4$ via configure_optimizers patch	0.1656	0.7342	100
v7 (final)	**Encoder extended 48h → 72h**	**0.1597**	**0.7109**	**108**

with $\lambda = 1e-4$ was introduced, following Lim et al. [5], who used the same regularisation strength for their TFT on traffic forecasting data. There is no weight decay value provided by Oreshkin et al. [33] in their original paper for the N-BEATS model. Therefore, without specific information about N-BEATS' hyperparameters, the value used by Lim et al. [5] was selected as a reasonable choice since the two models are trained on the same traffic data with similar training set sizes and normalisation procedures. The improvement in validation MASE from v5 (0.8492) to v6 (0.7342) after applying this regularisation, together with the increase in R to around 0.96, indicates improved generalisation. This was an issue because PyTorch Forecasting doesn't yet provide an optimizer_kwargs argument to the class constructor NBeats, which needed to be patched to add that argument to the configure_optimizers method of the class in Code 7.2:

CODE 7.2: N-BEATS optimizer and learning-rate scheduler configuration

```
def _configure_optimizers_with_wd(self):
    optimizer = torch.optim.Adam(
        self.parameters(),
        lr           = learning_rate,
        weight_decay = 1e-4,
    )
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, patience=4, factor=0.5, mode="min"
    )
    return {
        "optimizer":    optimizer,
        "lr_scheduler": {"scheduler": scheduler, "monitor": "
   val_loss"},
    }

# Bind the new method to the model instance
import types
```

```
model.configure_optimizers = types.MethodType(
    _configure_optimizers_with_wd, model
)
```

The v6 to v7 transition resolved performance issues stemming from 'saturation' of the t+5h and t+6h performance levels. Both horizons in v6 achieved almost the same MASE of 0.786 and 0.795, indicating the 48-hour encoder had hit a ceiling on the information it could deliver, and that the model could see two complete diurnal cycles, but not a third. The 72 hours would provide a third day of historical context. N-BEATS does not have access to the weekly lag provided by CNN+LSTM and TFT, so it has to learn the weekly seasonality from the raw sequence itself. Three full diurnal cycles give more context for the day of the week than do two, especially when trying to differentiate between Monday and Tuesday mornings.

TABLE 7.6: N-BEATS v7 final configuration

Parameter	**Value**	**Justification**
Encoder length	72h	Extends v6's 48h to provide third diurnal cycle
Prediction length	6h	Matches CNN+LSTM v2 and TFT v3
Stack types	Trend + seasonality	Interpretable variant — Oreshkin et al. (2020) [33]
Blocks per stack	4 + 4	Maximum interpretable configuration
FC layers per block	4	Standard depth from original paper
Width (trend / seasonality)	256 / 2048	Asymmetric — Fourier needs more capacity
Expansion coefficients	3 (trend) / 7 (seasonality)	Degree-2 polynomial; 7 Fourier harmonics
Weight sharing	True	Structural regularisation across blocks
Dropout	0.1	Applied during training only
Weight decay	$\lambda = 1e-4$	Following Lim et al. (2021) [5]
Batch size	128	Reduced from v6's 256 for larger windows
Loss function	MAE on z-score	Identical to CNN+LSTM and TFT
Trainable parameters	51,870,720	Width-driven — not an advantage
Training time	2,138 min (35.6h)	NVIDIA RTX A2000 12GB

7.2.3 Training Behaviour

N-BEATS v7 was trained for 108 epochs with the best validation checkpoint at epoch 91, which yielded a val MAE of 0.1597. Both losses had a staircase type convergence (stabilizing on a plateau for several epochs, and decreasing by a step each time the learning rate was lowered by the scheduler). The 6 LR reductions in training, from 1e-4 to 9.77e-8, or effectively 0, showed the model to converge before early stopping was triggered at epoch 106.

FrozenTrainEval method yielded directly interpretable R ratios for the entire duration. R stabilised at 0.950–0.955 from epoch 50 onwards. The ability of N-BEATS to attain an R lower than 1.0, as discussed in Chapter 5, is simply because the seasonal naive MAE in the validation period is lower than in the training period (approximately 116.8 veh/h vs 105.63 veh/h).

The loss curves and R ratio profile for the staircase are represented in Figure 7.4. The convergence phase is also found to be healthy, as indicated by the clear colour change from blue bars ($R < 0.95$, early epochs) to green bars ($R \approx 1.0$, from epoch 50 onwards) in the right panel.

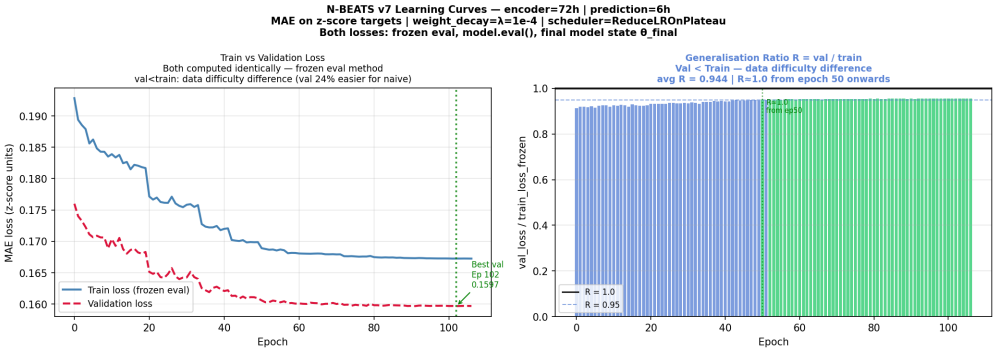


FIGURE 7.4: N-BEATS v7 learning curves

7.2.4 Test Results

The per-horizon test performance of the N-BEATS v7 model is presented in Table 7.7, showing the forecasting accuracy for each prediction horizon.

All six horizons exceeded the naive seasonal level of expectations. The best single horizon result for this model is the t+1h MASE of 0.4746, which is a 52.5% gain over naive. For the simplest model, this seems paradoxical, but it is possible to explain: At t+1h the raw traffic sequence is very informative. Due to the high short-term traffic data autocorrelation, it is highly predictive to know what the previous 72 hours of traffic

Table 7.7: N-BEATS v7 per-horizon test results

Horizon	MAE (veh/h)	RMSE (veh/h)	MASE	SMAPE (%)	R^2	Naive MAE
$t+1h$	66.07	151.00	0.4746	26.25	0.9782	139.21
$t+2h$	80.47	188.71	0.5780	28.36	0.9660	139.24
$t+3h$	87.89	203.27	0.6311	29.70	0.9605	139.26
$t+4h$	91.84	212.17	0.6593	30.16	0.9570	139.29
$t+5h$	92.35	215.65	0.6629	29.96	0.9556	139.31
$t+6h$	93.87	219.68	0.6737	30.30	0.9539	139.33
Overall	**85.42**	**198.41**	**0.6133**	**29.12**	**0.9619**	139.20

were for the next hour. N-BEATS takes advantage of this autocorrelation without any distraction from potentially noisy external features, and gains its greatest relative benefit in the presence of the temporal signal's highest density.

However, the degradation of MASE from t+1h to t+6h is 0.199, which is significantly larger than CNN+LSTM (0.132) and TFT (0.130). This deeper degradation shows the fundamental challenges of longer-term univariate forecasting – the model takes no account of the hour and day of the week that the forecasted timesteps occur, and so depends entirely on the diminishing auto-correlation signal and the extrapolation of the Fourier basis outside the encoder window. Six hours into a very uncertain tomorrow (at t+6h), the raw data is gradually less reliable regarding the time-of-day conditions that prevail.

To further examine the model's performance across different road types, Table 7.8 presents the test results for each road subclass.

Table 7.8: N-BEATS v7 per road sub-class test results

Sub-class	MAE (veh/h)	RMSE (veh/h)	MASE	SMAPE (%)	R^2	Windows
URB_normal	31.80	64.48	0.2285	17.59	0.9653	218,100
URB_ramp	68.54	134.95	0.4924	17.63	0.9429	213,830
M30	149.97	309.89	1.0774	20.60	0.9504	218,100
C30	91.03	202.11	0.6539	60.45	0.9640	217,970

The most obvious discovery from the breakdown of the sub-classes is the M30 result (MASE=1.0774). If the MASE is greater than 1.0, N-BEATS is slightly worse than the seasonal naive baseline model at motorway sensors. This is not a problem of training, it's a problem of information structure constraints. The incidents, the variable speed limit signage and the spatial flow diversion are all events that have strong signatures throughout the sensor network, but not within a single sensor's 72-hour history, and they make up a significant part of the motorway traffic. Last week's value repeated in the

seasonal naive baseline assumes motorway patterns remain fairly constant from week to week which may not be true during incidents, but may be true at other times. The lack of any spatial and contextual information means that N-BEATS is not able to do better on M30 sensors than this implicit stability assumption. Practical implication of this fact is that for management of motorways traffic where incident response is key, univariate model alone is inadequate, and spatial or multivariate models become essential.

URB_normal sensors improve upon naive sensors by 77.2% (by MASE=0.2285). This confirms that for residential urban traffic the traffic flow has regular enough characteristics that a univariate model can be used with significant predictive power. If the application is for urban arterials instead of motorway management, a low computation cost univariate model can be used and N-BEATS v7's training time of 35.6 hours is competitive with TFT v3's 78.7 hours (Section 7.3) for urban-only forecasting.

The model's performance under peak and off-peak traffic conditions is compared in Table 7.9.

TABLE 7.9: N-BEATS v7 peak vs off-peak test results

Period	**MAE (veh/h)**	**RMSE (veh/h)**	**MASE**	R^2	**Windows**
Morning peak (06–09h)	109.17	245.75	0.7843	0.9511	144,865
Evening peak (17–20h)	81.68	171.25	0.5868	0.9695	144,466
Weekend	77.55	173.63	0.5571	0.9563	247,259
Off-peak weekday	83.91	205.35	0.6028	0.9631	413,764

The gap between the MASE of N-BEATS and the MASE of the multivariate models (0.7843 - 0.5868) is larger than the gap between the MASE of CNN+LSTM and the MASE of N-BEATS (0.7367 - 0.5645), and larger than the gap between the MASE of the multivariate models and the MASE of CNN+LSTM (0.7367 - 0.5868). This extra challenge for N-BEATS during the morning peak is also consistent with the architecture's use of the raw sequence: Calendar related aspects of morning rush hour timing and height that cannot be represented in a univariate model, but that are at least captured in the is_rush_hour and is_holiday flags in the feature vector for a multivariate model.

In addition, The forecasting behaviour of N-BEATS v7 is illustrated in Figure 7.5, where the observed 72-hour input sequence is shown together with thesix subsequent forecast horizons same as CNN+LSTM model.

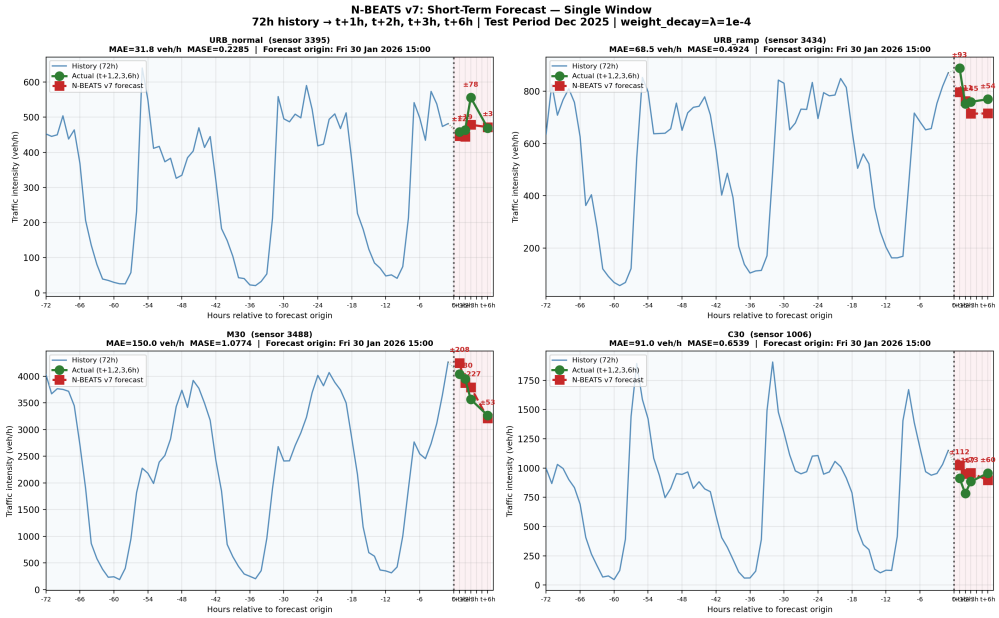


FIGURE 7.5: N-BEATS v7 prediction snapshot across four forecasting horizons

7.3 TFT

7.3.1 Context and Motivation

CNN+LSTM and N-BEATS both determine the model output by a fixed computational structure: CNN slides its filters over all time steps, and N-BEATS uses the same fully connected layers over the entire encoder vector. These two architectures do not pose the question: In this particular prediction context, at this time, will these features be most significant, and will these historical timesteps be most important? In a multi-sensor network that covers residential streets, motorway ramps, tunnels and ring motorways, this lack of context sensitivity is a significant constraint. A feature not important to predicting motorway traffic, e.g. distance to a shopping mall, may be important to predicting traffic near a major shopping mall on a Saturday afternoon. A model that cannot change its feature weights to match the prediction context will always be constrained by the features' worst case relevance.

The Temporal Fusion Transformer (TFT) proposed by Lim et al. (2021) [5] overcomes this limitation by three mutually related mechanisms that enable it to reason explicitly about which inputs are relevant, when in history, and for which of its prediction windows. It is also the most architecturally complex model in this thesis, and as the results show, the best performing model. By comparing the performance of TFT with CNN+LSTM,

both of which had the same features, the effect of context aware input selection and temporal attention over fixed convolutional-recurrent processing is extracted.

The TFT architecture is described in Section 5.2.3.

7.3.2 Experimental Progression and Final Configuration

Two versions of TFT were trained out of three as v1 is excluded because of few limitations of methods such as global per class normalizer, loss values in denormalised values(veh/h). The validation loss for TFT v2 was 0.1305, which was the lowest validation loss of all the models in this thesis. TFT v2 had an encoder of 24 hours and made predictions for 3 horizons. To make a fair comparison with the other models, TFT v3 was extended to 48 hours of encoder and to 6 horizons for prediction.

Table 7.10: TFT experimental versions

Version	Encoder	Prediction	Best val loss	Epochs	Training time
v2	24h	3h	0.1305	60	2,416 min
v3 (final)	**48h**	**6h**	**0.1356**	**56**	**4,722 min**

The very slight increase from 0.1305 to 0.1356 is due to the more difficult task of predicting three horizons ($t + 4h$, $t + 5h$ and $t + 6h$) in addition to the common horizons in v3. The MAE values of v2 and v3 for the overlapping time horizon $t + 1h$ to $t + 3h$ are nearly the same, with differences of 0.01 veh/h for $t + 1h$ and 0.35 veh/h for $t + 3h$. This almost perfect consensus validates the fact that there was no measurable accuracy increase when the encoder was doubled from 24h to 48h for TFT - whereas for N-BEATS, the 48h extension of the encoder gave a 3.2% MASE increase. The reason is architectural – it has explicit access to the encodings of `lag_168h_norm` and day-of-week which already include the weekly seasonal anchor. The task for TFT is to construct the contextual LSTM state from the recent history, which can be done using two diurnal cycles. N-BEATS requires three cycles as it needs to learn the weekly pattern from the raw sequence.

For reasons of practicality the encoder was not expanded to 72 hours (matching N-BEATS). However, to increase the amount of time it can capture, TFT's LSTM must be trained one encoder timestep at a time, which would increase the per-epoch training time from 80 minutes to ∼240 minutes, if 72 hours were used vs. 24 hours, and would be too time-intensive to train on the available hardware. The 48-hour choice is a well-grounded choice based on the comparison of v2 and v3 accuracy in common horizons.

Table 7.11: TFT v3 final configuration

Parameter	Value
Encoder length	48h
Prediction length	6h
Hidden size	64
Attention heads	4
Dropout	0.1
Hidden continuous size	16
Learning rate	1e-3 with ReduceLROnPlateau
Reduce on plateau patience	4
Batch size	128
Max epochs	150 — early stopping at epoch 56
Patience	15
Loss function	MAE on z-score
Trainable parameters	357,071
Training time	4,722 min (78.7h)
Static covariates	11 features
Time-varying known inputs	10 features
Time-varying unknown inputs	8 features

TFT was implemented using the PyTorch Forecasting library (version 1.8.0), which offers a validated implementation that is compatible with Lim et al. (2021) [5]. The `target_normalizer=None` argument is used for the construction of the dataset because the normalisation is done manually to ensure that the dataset is comparable with N-BEATS and CNN+LSTM with given code snippet 7.3 and 7.4 :

Code 7.3: TFT training dataset configuration

```
training_dataset = TimeSeriesDataSet(
    df_train,
    time_idx  = "time_idx",
    target    = "target_norm",
    group_ids = ["sensor_str"],
    max_encoder_length    = 48,
    min_encoder_length    = 48,
    max_prediction_length = 6,
    min_prediction_length = 6,
    static_reals               = STATIC_REALS,
    time_varying_known_reals   = TIME_VARYING_KNOWN_REALS,
    time_varying_unknown_reals = TIME_VARYING_UNKNOWN_REALS,
    target_normalizer     = None,
    add_relative_time_idx = True,
    add_target_scales     = True,
    add_encoder_length    = True,
    allow_missing_timesteps = True,
```

```
)
```

Code 7.4: Temporal Fusion Transformer model configuration

```
tft = TemporalFusionTransformer.from_dataset(
    training_dataset,
    hidden_size            = 64,
    attention_head_size    = 4,
    dropout                = 0.1,
    hidden_continuous_size = 16,
    loss                   = MAE(),
    learning_rate          = 1e-3,
    reduce_on_plateau_patience = 4,
)
```

7.3.3 Training Behaviour

During the training for TFT v3, best model was found at epoch 39 (val MAE=0.1356) with 56 epochs. At epochs 1 and 55, the loss for the training set was 0.1736 and 0.1286, respectively. The learning rate was scaled down by a factor of 4, from 1e-3 to 6.25e-5, resulting in each successive learning rate a step improvement in the loss, followed by another plateau.

The generalisation ratio R was also found to be the closest to 1.0 of all models in this thesis, with an average of 1.014 across 56 epochs. This near-perfect R is caused by two counteracting effects: the dropout effect is that training becomes more difficult than validation when neurons are dropped, and the data difficulty effect is that the validation is generally easier to predict. For TFT, these two effects roughly cancel each other out and produce the healthy $R \approx 1.0$ curve that can be seen in the loss curves. As mentioned in Chapter 5, TFT's training loss is computed using the running average instead of the FrozenTrainEval method, the R ratio is useful for observing convergence, but not strictly comparable to the R ratios of N-BEATS or CNN+LSTM, which are both calculated using the frozen evaluation method.

The loss curves and R ratio for the TFT v3 are plotted in Figure 7.6. The two curves in the left panel are practically identical with the typical staircase shape at each LR reduction event.

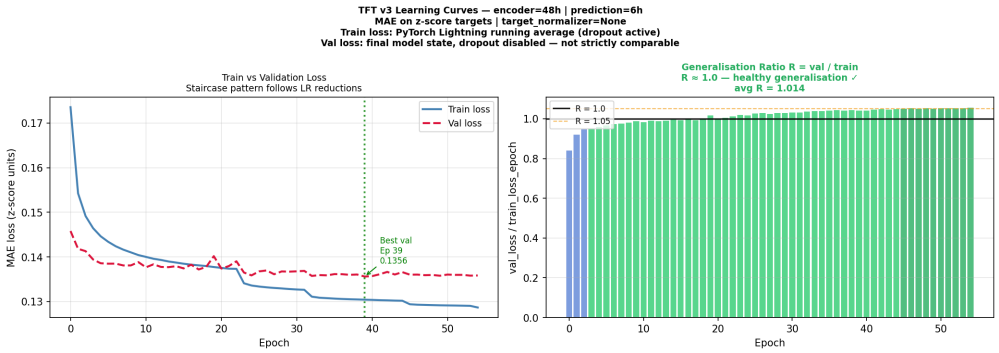


FIGURE 7.6: TFT v3 learning curves

TABLE 7.12: TFT v3 per-horizon test results

Horizon	**MAE (veh/h)**	**RMSE (veh/h)**	**MASE**	**SMAPE (%)**	R^2	**Naive MAE**
$t+1h$	62.95	145.80	0.4522	25.44	0.9797	139.21
$t+2h$	72.12	179.44	0.5180	26.51	0.9692	139.24
$t+3h$	75.74	190.89	0.5439	26.98	0.9652	139.26
$t+4h$	77.94	196.92	0.5595	27.28	0.9630	139.29
$t+5h$	79.48	200.36	0.5705	27.48	0.9617	139.31
$t+6h$	81.10	204.55	0.5820	27.68	0.9600	139.33
Overall	**74.89**	**186.33**	**0.5377**	**26.89**	**0.9665**	139.20

7.3.4 Test Results

The overall MASE value of TFT v3 is 0.5377, which is the lowest among the three models, TFT v3, CNN+LSTM, and N-BEATS with 0.5767 and 0.6133 respectively. All 6 of the horizons were statistically different from the naive baseline. The best single-horizon result among all models is the $t+1h$ MASE = 0.4522, which is a 54.8% improvement over naive.

The per-horizon MASE range for TFT is 0.4522 to 0.5820, a spread of 0.130 units — the smallest of the three models (CNN+LSTM: 0.132, N-BEATS: 0.199). This reduced spread arises from TFT's knowledge of the calendar information for future timesteps: if they know that $t+6h$ is on Tuesday at 14:00 of a non-holiday week, they have direct information about the traffic regime that is more likely to be accurate at longer time horizons than an extrapolation from the traffic history in the encoder.

The biggest improvement over CNN+LSTM is at M30. TFT manages to outperform naive by 8.2%, with MASE=0.9183, whereas CNN+LSTM just managed to get a 1.7% and N-BEATS failed completely. The improvement can be attributed to TFT's static covariates/Variable Selection Network: road sub-class encoding which had the second

TABLE 7.13: TFT v3 per road sub-class test results

Sub-class	MAE (veh/h)	RMSE (veh/h)	MASE	SMAPE (%)	R^2	Windows
URB_normal	28.24	60.89	0.2029	15.84	0.9690	218,100
URB_ramp	58.60	123.27	0.4210	15.04	0.9524	213,830
M30	127.83	289.17	0.9183	17.69	0.9568	218,100
C30	84.56	193.72	0.6075	58.80	0.9669	217,970

largest static importance weight (0.1300) in Chapter 9, showing that TFT learned to modify their prediction approach when seeing a road. For M30 sensors this refers to the use of motorway patterns instead of urban patterns, a distinction which is hard to achieve with CNN+LSTM without explicit gating and cannot be achieved at all by N-BEATS.

TABLE 7.14: TFT v3 peak vs off-peak test results

Period	MAE (veh/h)	RMSE (veh/h)	MASE	R^2	Windows
Morning peak (06–09h)	95.58	229.81	0.6867	0.9573	144,865
Evening peak (17–20h)	73.97	161.90	0.5314	0.9728	144,466
Weekend	64.07	153.84	0.4603	0.9657	247,259
Off-peak weekday	74.34	195.22	0.5340	0.9666	413,764

In each period, the lowest MASE is achieved by TFT. The morning peak improvement over CNN+LSTM is 0.7367 → 0.6867 (6.8 percentage points). The `is_holiday` flag is assigned a variable selection weight of 0.0624 during the encoder (Chapter 9), signifying that TFT actively uses holiday information to make adjustments to its prediction of the up-and-down trends of the morning peak – something that CNN+LSTM or N-BEATS cannot do with the same degree of selectivity.

The seasonal naive baseline is 0.4459, a 54% improvement over the baseline is achieved at weekend MASE of 0.4603. The third highest of all decoder variables is the `is_weekend` decoder variable weight (0.1381, Chapter 9) which shows that TFT explicitly considers the weekend status when building 6-hour ahead forecasts, which is very appropriate, as the rush-hour patterns are appropriately downweighted on Saturdays and Sundays.

Figure 7.7 shows the per-horizon bar chart for TFT v3.

Furthermore, Figure 7.8 shows a representative TFT v3 forecast, including the observed 48-hour input sequence and the six subsequent forecast horizons.

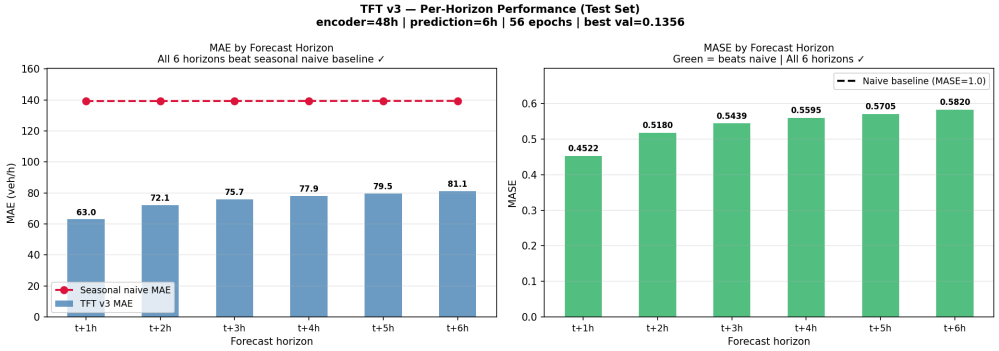


FIGURE 7.7: TFT v3 performance across forecasting horizons

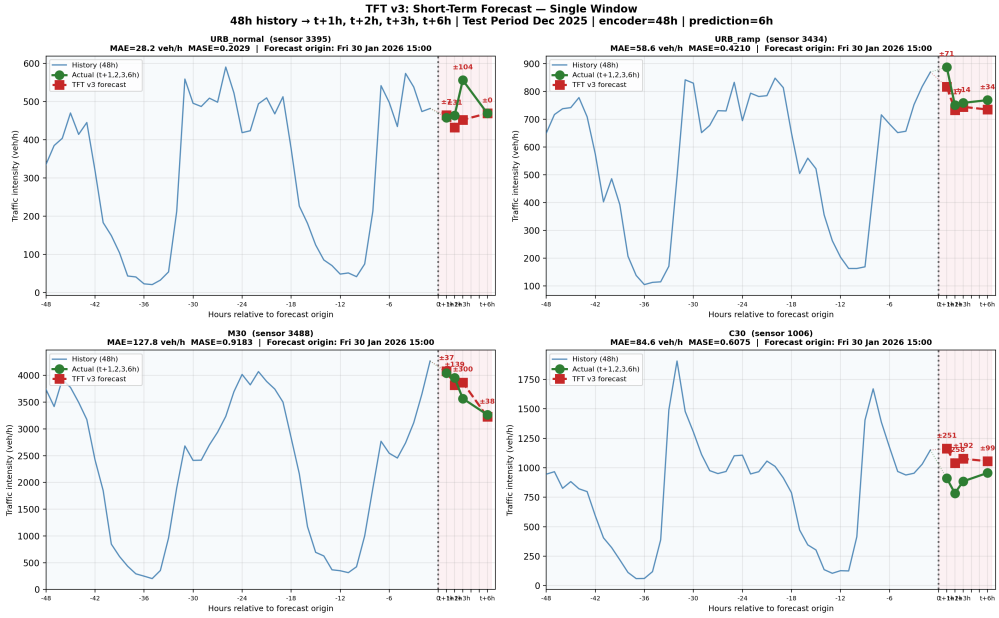


FIGURE 7.8: TFT v3 representative forecast across four horizons

7.4 Summary

Key results of all three models on the test set have been combined in Table 7.15 for the comparison.

TABLE 7.15: Three-model comparison on the overall test set

Model	Encoder	Prediction	MAE	RMSE	MASE	SMAPE	R^2	Time
CNN+LSTM v2	48h	6h	80.33	187.56	0.5767	28.44	0.9661	3.5h
N-BEATS v7	72h	6h	85.42	198.41	0.6133	29.12	0.9619	35.6h
TFT v3	**48h**	**6h**	**74.89**	**186.33**	**0.5377**	**26.89**	**0.9665**	**78.7h**

It is consistent with the architecture progression, with TFT before CNN+LSTM before N-BEATS. That has been TFT's strength over CNN+LSTM (7.3% lower MASE) as

they are both fed with the same 28 features – only the variable selection and temporal attention are different. CNN+LSTM's benefit over N-BEATS (6.0% lower MASE) is that it separates the contribution of the 28 multivariate features from the univariate sequence alone.

All three models outperformed the seasonal naive baseline on all six horizons and all four road sub-classes (with the exception of N-BEATS on M30, MASE=1.0774, which reflects a structural information limitation rather than a modelling failure. The M30 result is the most important one because it shows that the only model that can be used for the motorway sensors in the Madrid network is one that includes spatial and contextual covariates, such as CNN+LSTM and TFT do.

Chapter 8

Model Comparison

8.1 Framing the Comparison

The architecture, the training behaviour and the results of the test sets with different breakdowns of each of these models were explained in the preceding chapter. Each set of results is only a part of the story. The numbers from a single model do answer the question about how well the model performed, but do not answer the more important question of why the model performed the way it did, or what the differences between models show about the nature of the forecasting problem itself.

In this chapter we combine the three models. The comparison is not a rank-and-file operation, it is an effort to separate out the effects of each of the decisions that went into the design. CNN+LSTM and TFT share the same 29 features, and the performance gap is due to the independence of value of context-aware variable selection and temporal attention from fixed convolutional-recurrent processing. The difference between CNN+LSTM and N-BEATS is that CNN+LSTM predicts exactly the same 6 horizons on the same 200 sensors but they predict the value of those 29 features (28 features + target_norm, Section 7.1.1), which is what makes them different from the raw univariate sequence. These contrasts have been intentionally created, it's not just an afterthought, and the outcomes are worth a close look.

8.2 Overall Performance

The headline comparison is straightforward. Table 8.1 summarizes test results for the three models overall.

Table 8.1: Three-model comparison, overall test set results

Model	Encoder	Prediction	MAE (veh/h)	RMSE (veh/h)	MASE	SMAPE (%)	R^2	Training time
TFT v3	48h	6h	74.89	186.33	0.5377	26.89	0.9665	78.7h
CNN+LSTM v2	48h	6h	80.33	187.56	0.5767	28.44	0.9661	3.5h
N-BEATS v7	72h	6h	85.42	198.41	0.6133	29.12	0.9619	35.6h

Ordering TFT > CNN+LSTM > N-BEATS is without exception for all four summary metrics. All three models substantially outperformed the naive seasonal baseline, with the least powerful (N-BEATS) having MASE=0.6133, which is 38.7% more accurate than repeating last week's traffic value. The best (TFT) gets MASE=0.5377, which is a 46.2% improvement.

The similarities among the numbers are more striking than the differences. Each one of the three R^2 values is greater than 0.96, indicating that each of these models accounts for more than 96% of the intensity of traffic in the entire sensor network. This is a good result for a system that includes structurally different traffic dynamics in residential back streets, motorway ramps, tunnels, and a ring motorway. The 4% residual is the part that has the most variation and where the models differ most distinctly.

The ratio of MAE to RMSE is informative. The RMSE for all three models is about 2.2–2.3 times the MAE. This ratio is consistent across architectures: all models have some large errors (incidents, disruptions, that no sensor-history model can predict) as well as some smaller errors that are typically captured by the MAE. This ratio does not tighten with complexity of architecture, indicating that errors are not due to any weakness in architecture that could be designed out, but are due to the fundamental limit on information that can be gathered by a single sensor to forecast.

8.3 Per-Horizon Analysis

The overall metrics in Table 7.1 summarise the performances over the six forecast horizons, but the behaviour over these horizons shows different architectural strengths. Per-horizon MASE of all three models are presented in Table 7.2.

At $t + 1h$, N-BEATS actually beats CNN+LSTM (0.4746 vs 0.4864). This seemingly counterintuitive result of the easiest possible model is well explained. The raw traffic sequence is very informative at the shortest time frame: knowing the last 72 hours of the history of a sensor is very predictive of the next hour since traffic data is highly

autocorrelated in the short term. N-BEATS uses this signal in its purest form without noise from other features that could be of no importance at a given time in a given sensor. CNN+LSTM, which takes all 29 features and passes them through fixed convolutional filters, cannot selectively ignore unhelpful features in a specific prediction context. The variable selection mechanism of TFT makes it the exception, as it is able to de-emphasise unimportant features, thus outperforming at $t + 1h$ on a larger, noisier input space.

The multivariate models differ from N-BEATS in a regular and increasing way from $t+2h$ onwards. The mean absolute error (MASE) of N-BEATS is 0.199, which is higher than that of TFT (0.130) and CNN+LSTM (0.132) over the spread $t + 1h$ to $t + 6h$. This is a direct consequence of the increased value of calendar features at longer time horizons. Information about the day of the week and time of day for $t+6h$ is directly useful as it is during a morning rush hour on a Tuesday, information that N-BEATS, only seeing the raw sequence, cannot have a-priori. The less a model can rely on autocorrelation, the less confident it becomes the further ahead it goes, while a model aware of the calendar context is more consistent and accurate away from autocorrelation.

Figure 8.1 shows the per-horizon MASE curves for all three models.

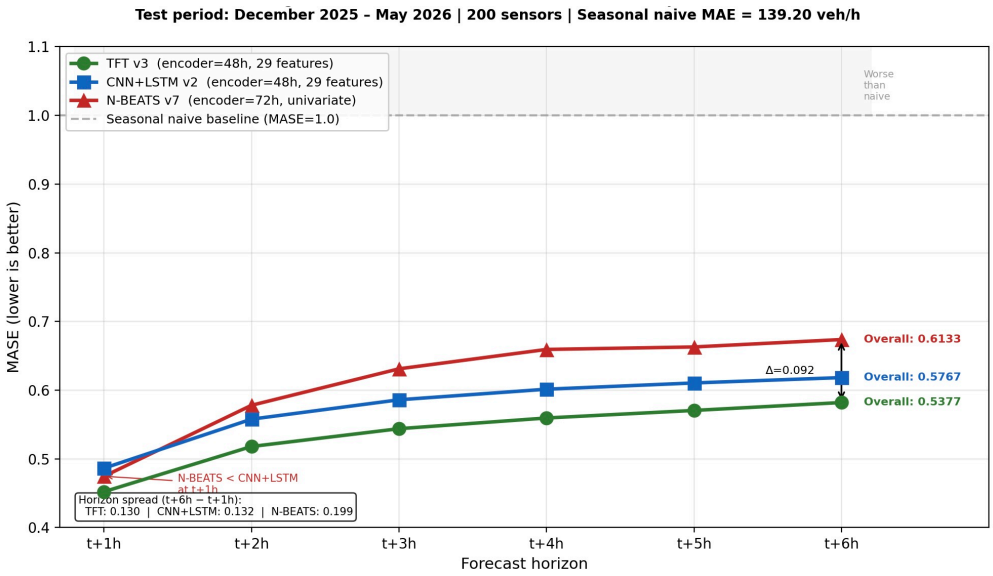


FIGURE 8.1: Per-horizon MASE comparison of the three models

8.4 Sub-Class Analysis

The biggest differences between models are in the sub-class breakdown. Table 8.2 presents the severity of the MASE (by road sub-class) values for all three models.

Table 8.2: Per sub-class MASE comparison (test set)

Sub-class	TFT v3	CNN+LSTM v2	N-BEATS v7
URB_normal	0.2029	0.2188	0.2285
URB_ramp	0.4210	0.4498	0.4924
M30	0.9183	0.9826	1.0774
C30	0.6075	0.6546	0.6539

8.4.1 URB_normal

Every model has the most predictable sub-class that is residential and secondary urban sensors. The range of MASE values for the three models is small, just below 3 percentage points (0.2029 to 0.2285). This convergence is useful to note: once the normal trend of the diurnal activity of a residential street is established, adding further architectural sophistication does not provide a significant amount of information. In other words, a sequence of 72 hours of traffic in a quiet Madrid back street on a Tuesday morning is already more or less enough for accurate prediction, because the week-on-week and day-on-day patterns are stable enough and both a sequence model and an attention-based model learn essentially the same thing.

8.4.2 URB_ramp

The range is larger for the motorway ramp sensors, ranging from MASE 0.4210 (TFT) to MASE 0.4924 (N-BEATS), a 7.14 percentage point gap between the different architectures. These sensors are at the intersection of urban and motorway dynamics, characterized by a more regular diurnal pattern as exhibited in urban traffic and more intensive and volatile as in merge and diverge dynamics. Here the multivariate models are significantly better than at URB_normal, indicating that information beyond the raw sequence (road sub-class encoding, geographic location, proximity to interchange points) provides essentially useful contextual information for ramp sensors that can't be exploited by N-BEATS.

8.4.3 M30

The most significant finding of the comparison is the result of the M30. Their structural information ceiling is not achievable by any model, even if it's architecturally complex, with just a single-sensor history.

The MASE value of N-BEATS is 1.0774, which is lower than seasonal naive baseline. CNN+LSTM is slightly better than naive with MASE=0.9826, 1.7%. The more meaningful number indicates an improvement of MASE=0.9183, which is reached by TFT. These are impressive figures for comparison, when the same models yield 78-80% gains over naive on URB_normal sensors.

The answer is to do with the properties of M30 traffic. The ring motorway suffers from incidents and lane closures as well as variable speeds, all of which cause sudden, large variations in traffic intensity which are difficult for any model based on the history of the single sensor to detect. The seasonal naive baseline (which forecasts the last week of traffic) works reasonably well on M30 in the absence of an incident, as the motorway has generally stable week-to-week traffic flows. If an incident does happen, there is no model that out-performs the naive model because there is no trace to identify an incident from the event from the 48 hours or 72 hours history in the sensor. This is not a modelling mistake, rather the proper response of a model which has learned the right behaviour when the historical signal is informative and when it is not.

The static covariates (road sub-class encoding and geographic coordinates) give a prior information about the dynamics of the motorway to the model, which is why TFT has a slight advantage over the other networks on M30. In Chapter 8, the variable selection analysis results are coherent with the static importance weight of road sub-class, which was the second highest, indicating that the model learned to adapt its prediction strategy when it encountered a motorway sensor. N-BEATS, with no access to this information, cannot make the same adjustment.

The M30 finding is directly motivating a direction for future work: that of spatial models that act together across the M30 corridor, sharing information across adjacent sensors, which could potentially detect the spatial signatures of incidents before they are visible in any single sensor's history. The next step in the development of meaningful motorway forecasting improvement which the M30 clearly points out is not part of the present thesis, but is represented by a class of models (graph neural networks, spatial transformers).

8.4.4 C30

C30 has a odd trend: CNN+LSTM (0.6546) and N-BEATS (0.6539) are almost indistinguishable and TFT (0.6075) is a useful improvement. Architecturally different, it is somewhat surprising that the CNN+LSTM and the N-BEATS parity on C30 are so close. One possible explanation is that, as mentioned, the C30 traffic has a bimodal distribution with near-normal tunnel traffic or near zero during closures, and the gating in the

convolutional filters of CNN+LSTM is just as difficult to learn as the gating in Fourier basis of N-BEATS, which can represent the periodic closure pattern quite well. TFT's variable selection mechanism enables it to implement C30-specific behaviour that both alternatives cannot. The road sub-class encoding is a high-importance static feature in TFT's VSM.

Tunnel closures in this sub-class do not appear in the comparison as they do not have denominator values that are sufficiently high to provide a reliable SMAPE (see reasons set out in Section 5.3.1.5). The metric of the distance to MASE is always used.

8.5 Peak and Off-Peak Analysis

TABLE 8.3: Peak vs off-peak MASE comparison (test set)

Period	**TFT v3**	**CNN+LSTM v2**	**N-BEATS v7**
Morning peak (06–09h)	0.6867	0.7367	0.7843
Evening peak (17–20h)	0.5314	0.5645	0.5868
Weekend	0.4603	0.4961	0.5571
Off-peak weekday	0.5340	0.5732	0.6028

The difference between the morning and the evening peak values of the MASE is between 0.155 for TFT and 0.172 for CNN+LSTM up to 0.198 for N-BEATS, where multivariate forecasting approaches have a smaller difference compared to N-BEATS due to the presence of calendar features like is_rush_hour and is_holiday.

The morning peak difficulty is asymmetric, while the evening peak is not. Morning rush hours are steep, rising rapidly from the overnight activity and reaching a peak in a relatively small time window, and their magnitude and timing vary, in part due to the calendar factors that vary week to week such as the school term, weather, or pre-holiday adjustments, even for the same sensor. The 48-hour encoder would record the pattern of the past few mornings, but if the morning of the current week is different from recent mornings – either because it is the first week of term or because it is raining light in the morning – the model will have no means by which to anticipate that morning! In the afternoon, evening peaks ramp up more slowly and allow the encoder to see the traffic conditions developing more clearly, resulting in more predictable traffic conditions.

The two multivariate models explicitly include an `is_weekend` attribute, and weekend traffic shows a high degree of regularity (not including commute variation), both of which help the weekend accuracy. The regularity of weekend patterns is sufficient for

their detection in the raw sequence alone, as weekend accuracy is still the best for N-BEATS, which cannot use this flag.

The relative advantage is the largest for TFT in the morning peak (5%) and smallest on weekends (3.6%) (CNN+LSTM 0.7367 vs TFT 0.6867). The morning peak advantage is in line with TFT's explicit inclusion of `is_holiday` and `is_rush_hour` flags as time varying known decoder inputs: the model is able to adapt its predictions accordingly when the forecast range is in the morning peak period. The CNN+LSTM shares the same flags as the input features but does not have the ability to selectively gate them as the TFT does. The Chapter 8 variable selection analysis validates this, showing that `is_holiday`'s uniform weight of 1.18× is moderately important (0.0624), but is still more important than CNN+LSTM's SHAP analysis ranked it.

8.6 Training Efficiency vs Performance

There are three models all along the training time-performance continuum and this deserves to be discussed explicitly since training cost is a real practical consideration.

Figure 8.2 illustrates this trade-off.

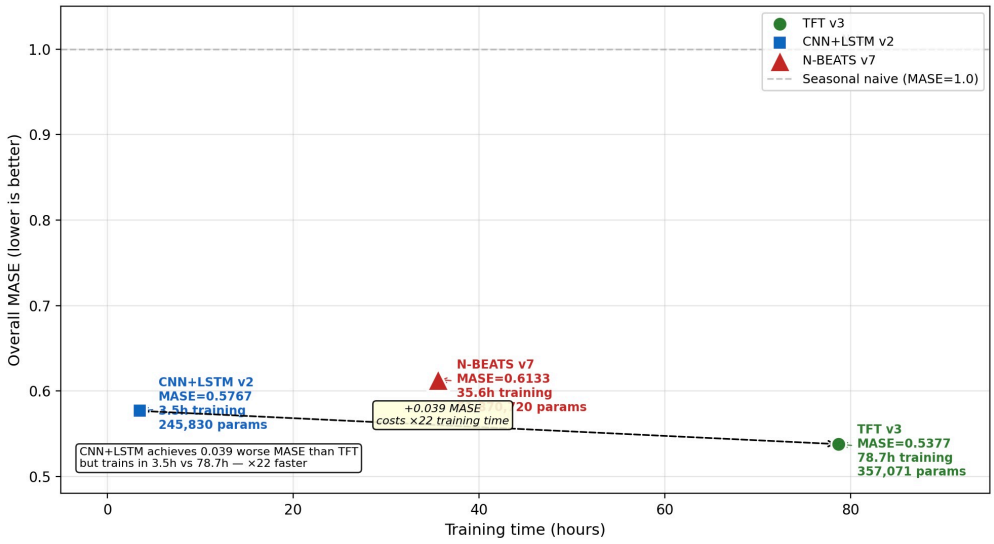


FIGURE 8.2: Training time versus overall MASE for the three models

CNN+LSTM trained for 3.5 hours and obtained the overall MASE=0.5767. TFT was trained for 78.7 hours and got the MASE of 0.5377. It takes 22.5× longer to improve the MASE by 7.3%. It is hard to say whether this trade-off is worth it or not, but it depends on the deployment context. CNN+LSTM's training cycle of 3.5 hours is

much more practical for an operational system that needs retraining when new data is presented. The advantage TFT offers is more defensible in a situation where there is little retraining and prediction quality is the most important criterion.

N-BEATS falls somewhere between the extremes: 35.6 hours of training for MASE=0.6133, which is worse than CNN+LSTM at 10x the training cost. This can be understood by the difference in the number of basic parameters (N-BEATS: 51.9 million; CNN+LSTM: 245,830). It is important to note that the large number of parameters is a result of the width of the architecture (the Fourier blocks are quite wide, and must be fully connected) and does not necessarily correspond to the predictive power of the model: those parameters are learning an approximation of a 6-hour Fourier basis from a 72-hour basis!

8.7 Limitations of This Comparison

The experimental design was selected to ensure that model comparisons would be as clean as possible (same sensors, normalisation, evaluation period and prediction task). However, there are still some limitations.

8.7.1 Encoder length asymmetry

CNN+LSTM v2 and TFT v3 have 48 hours of encoding, where as N-BEATS v7 has 72 hours of encoding. This is not an option, since the longer encoder is required for N-BEATS to estimate the weekly seasonal structure from the sequence, whereas TFT is explicit about the lag in its design. It is therefore not controlled for encoder length, and thus the difference between N-BEATS at longer horizons is partially the difference in the encoders themselves and not only due to the univariate vs multivariate difference. But, even at matched encoder length, N-BEATS v6 (with 48-hour encoder) achieved MASE=0.7342 while the multivariate models achieved 0.6133.

8.7.2 Training loss comparability

CNN+LSTM and N-BEATS adopt the FrozenTrainEval method in training loss, and the generalisation ratios are comparable. TFT utilizes running average training loss which is not the same in computation as PyTorch Lightning's loss. The test set evaluation protocol is the same for all three models, except for the training monitoring. This restriction is reported in Section 5.1.5 and it is not a factor in the validity of the comparison of test sets.

8.7.3 Sample size

The 200 sensors were used out of 4,886 active sensors in the Madrid network for training and evaluation of all three models. The 200 sensors were not chosen on the basis of geographic or functional representativeness, but on the basis of having the most complete training records. The findings will not necessarily apply to the entire network, especially to sensors with lower data quality, which will disproportionately impact the C30 class performance because of the infrastructure issues found in Chapter 3.

Chapter 9

Explainability Analysis

The last two chapters have set up the accuracy of each of these models in predicting traffic intensity and how they compare in terms of the three architectures with the three types of roads and time periods. But accuracy is not the only measure of the usefulness of a model in operational traffic management. The traffic engineer receiving an 850 veh/h forecast on a motorway ramp would want to know not only the number of traffic, but also from where that number came from — whether it was the state of traffic in the area, the time of day, or the distance from the sensor to a large transport hub. This knowledge is essential for making decisions about when to believe in the model, when to disregard it, and how to make it better when it doesn't work.

This chapter tackles this need by two complementary explainability analyses. The Variable Selection Networks and multi-head attention built into the model are used for TFT v3, and these are mechanisms specifically engineered to reveal the model's inner workings. Since the v2 of CNN+LSTM does not have a built-in explainability mechanism, GradientSHAP is used as a post-hoc attribution method for this model. The method used in each model is chosen following Arrieta et al. (2020) [11]: intrinsic methods are used when they are made available by the architecture itself and post-hoc approximations are used where they are not. The chapter concludes by directly comparing the two models on the relative importance of the features, identifying commonalities, differences and how the differences reflect architectural differences between the two models.

9.1 TFT v3 — Intrinsic Explainability

9.1.1 Internal Interpretability Mechanisms of TFT

TFT was designed by Lim et al. (2021) [5] so that two quantities from its forward pass are directly readable without any post-hoc approximation. The first is the Variable Selection Network weight, which is a softmax-normalized score computed for each input feature at each prediction window, representing the importance of this feature to the model at this point. The second one is the attention weight, which is a score assigned to each of the 48 encoder timesteps by the multi-head attention mechanism that indicates how much the historical hour at each timestep contributed to the decoder's predictions. They are not estimates or approximations, but are actually calculated with every prediction. The main strength of an intrinsic explanation is that it is not computed by a separate attribution algorithm, but rather it is what the model actually did. The practical implication is that the interpretation is as reliable as the model itself, because the weights are not affected by approximation errors found in the post-hoc interpretation with the help of gradient-based or perturbation-based methods.

Analyses were based on an interpolated output of 28,400 prediction windows based on a stratified sample (25 sensors per road sub-class, one week from the middle of the test period) generated by the `interpret_output()` function of PyTorch Forecasting. Consequently, internal tensors were returned with the predictions, using the `mode=raw` argument and the weights are averaged across all windows, in the `reduction=mean` argument. Weights were normalised to sum to 1.0 within each feature group as done in Lim et al. (2021) [5].

9.1.2 Static Covariate Importance

Table 9.1: TFT v3 static variable selection weights (top 7 of 14)

Feature	**Normalised weight**	**vs uniform (1/14 = 0.071)**
Dist. railway station	0.1344	1.88×
Road sub-class	0.1300	1.82×
Dist. university	0.1131	1.58×
Dist. stadium	0.1114	1.56×
Dist. hospital	0.0994	1.39×
Latitude	0.0818	1.15×
Longitude	0.0816	1.14×

Distance to railway station was the most important of any static feature, with a weight of 0.1344, which is 1.88 times the weight it would have had if all features had been

considered equal. This corroborates the result of the previous chapter (Chapter 3, Section 3.3), which also revealed that the traffic intensity at sensor 7023 was abnormally high, and was explained by the fact that it was situated 449 metres from the Méndez Álvaro railway and bus interchange. The learning algorithm TFT has learned was based only on the training data, without any explicit instructions given to it on which of the fixed properties is the most important.

Road sub-class ranks second (0.1300, 1.82×). The second most important static discriminator from the model was the type of road, as seen by the huge MASE difference between sub-classes in Chapter 7 (URB_normal MASE=0.2029 vs M30 MASE=0.9183). TFT has also come to understand that the administrative classification is indeed informative in terms of prediction, requiring different prediction strategies for a motorway sensor and a residential sensor.

The two geographic coordinate features come in second, with all five POI distance features coming first. This ordering matters: The raw latitude and longitude location distinguishes sensors by their location, while the traffic demand profile of proximity to various infrastructure types distinguishes them by their functional demand. The POI distance features can provide context, as a sensor at the same latitude can be on a quiet residential street or beside a bus terminal. Target scale centre, target scale scale and encoder length are all at the very bottom (less than 0.012) which is a good sign that the Variable Selection Network is suppressing the features that do not contain any discriminative information.

9.1.3 Encoder Variable Importance

Table 9.2: TFT v3 encoder variable selection weights (top 10 of 19)

Feature	**Normalised weight**	**vs uniform (1/19 = 0.053)**
Relative time index	0.1416	2.69×
Day of week (cos)	0.0844	1.60×
Lag 2h	0.0776	1.47×
Lag 1h	0.0745	1.42×
Hour (sin)	0.0683	1.30×
Is_holiday	0.0624	1.18×
Traffic intensity (lag)	0.0584	1.11×
Rolling mean 24h	0.0523	0.99×
Is weekend	0.0478	0.91×
Lag 3h	0.0465	0.88×
...		
Lag 168h (weekly)	0.0295	0.56×
Rolling mean 3h	0.0185	0.35×

The relative time index is an internal PyTorch Forecasting feature that is just the number of elements in the encoder window given a 0 to 1 scale, which has the highest weight of 0.1416, or 2.69 times the uniform baseline. This is a very important discovery. The model depends more on the knowledge of the position of each observation in the encoder window than on the value of any one of the domain features. It seems TFT treats the 48-hour encoder as a temporally ordered unit with position having independent predictive value. The two observations at $t-1h$ and $t-23h$ have different meaning for the forecast, and TFT explicitly represents the distinction.

The second most important encoder feature is day-of-week cosine (0.0844, 1.60×), which shows that the weekly seasonal pattern is the largest long-range temporal structure that the encoder utilizes. This is similar to the results of Vélez-Serrano et al. (2021) [9] who showed that with the Kruskal-Wallis test, the weekly autocorrelation is statistically stronger than the daily one in the Madrid network.

The lag story is a more complex one. Short-term lags (lag 2h at 0.0776, lag 1h at 0.0745) rank third and fourth respectively — well above the uniform baseline. The explicit weekly lag feature (lag 168h) is ranked 15th with weight 0.0295, which is quite low when compared to uniform. This seemingly contradictory situation, where the weekly pattern is the most prominent temporal structure, while the weekly lag feature is suppressed, is discussed because it can be explained by the concurrent mechanisms by which TFT captures weekly seasonality. The day-of-week cosine (rank 2) indicates which day of the week each observation is recorded on, and the attention mechanism (Section 9.1.5) focuses more on the same-hour position on the previous day. These two mechanisms combine to give a weekly periodicity without the need to examine the exact value 168 hours ago. The lag_168h feature is not much needed and TFT's gating mechanism properly detects and blocks it.

9.1.4 Decoder Variable Importance

Here are the weights for the variable selection for the TFT v3 decoder (all 11 features listed):

The decoder is only fed with the features that are known in advance — the calendar data for the six hours to be decoded. The dominant fraction of decoder importance (0.1763 + 0.1640 = 0.340 of the total weight of the decoder) is dominated by hour (sin) and hour (cos). The most useful future information available to TFT is knowing what time of day the forecast horizons occur, as a forecast at 08:30 is quite different from a forecast at 14:00, even if the encoder history is similar.

Table 9.3: TFT v3 decoder variable selection weights (all 11 features)

Feature	**Normalised weight**	**vs uniform (1/11 = 0.091)**
Hour (sin)	0.1763	1.94×
Hour (cos)	0.1640	1.80×
Is weekend	0.1381	1.52×
Is rush hour	0.1248	1.37×
Day of week (sin)	0.0857	0.94×
Is_holiday	0.0677	0.75×
Month (sin)	0.0535	0.59×
Season	0.0515	0.57×
Relative time index	0.0508	0.56×
Day of week (cos)	0.0502	0.55×
Month (cos)	0.0374	0.41×

Is_weekend ranks third (0.1381, 1.52×) and is rush hour fourth (0.1248, 1.37×). These two binary flags represent the most significant boundaries in the traffic regimes in terms of operation. The decoder heavily depends on them, which is why the above mentioned TFT model's morning peak accuracy (MASE=0.6867) is superior to CNN+LSTM (0.7367), the decoder explicitly indicates the forecast horizon being in a rush time and TFT adjusts the predictions accordingly. CNN+LSTM is a variant of CNN which is also equipped with the same flag as an input feature, but it has no selective gating mechanism to selectively gate that flag as does the Variable Selection Network.

At the level of the features above it, in the scale of 6-hours prediction horizon, position within the month is last (0.0374), which means that the forecast-relevant information of position within the month is less important than above it.

9.1.5 Temporal Attention Weights

Table 9.4: Top 5 most attended encoder positions

Position	**Attention weight**	**Interpretation**
$t - 23h$	0.1576	Yesterday at the same hour
$t - 24h$	0.1108	Yesterday one hour before forecast
$t - 47h$	0.0836	Two days ago at the same hour
$t - 48h$	0.0718	Two days ago one hour before
$t - 1h$	0.0715	Most recent observation

One of the most easily explained results presented in this chapter is the temporal attention profile. The highest weight was given to the position $t - 23h$, which represents the location at the same time of day on yesterday. The next two most popular slots are $t - 24h$ and $t - 47h$, for the slots that were two days and three days before today

respectively. The pattern is clear, the TFT found it on its own without being instructed to search for it on a daily basis. There was no specific teaching that yesterday's traffic at the same time was relevant; it is only a product of the model's training data over a period of 29 months.

This discovery is a serious one and must be interpreted with care. There are two different ways to represent the same-time-yesterday pattern: an explicit `lag_24h` feature, and the attention mechanism — both parts of the model. The attention analysis indicates that the temporal pathway (which focused on the anchor positions per day) is more important than the explicit lag feature pathway (which was ranked seventh as an encoder). The architecture of TFT is able to learn the periodicity of the data through the position-aware attention mechanism, which is not necessarily obtained from the lag values a priori, and the trained model takes advantage of this.

The total attention is divided into 64.4% in the most recent 24 hours and 35.6% in the older 24 hours (hours $t - 25h$ to $t - 48h$). There is a trend towards recentness but it is not strong. The weight for the older half of the window is 35.6%, which seems like a reasonable empirical value, and this is why the 48 hour encoder is proposed for TFT v3 — the older half of the window actually adds value to the prediction, not being ignored.

The overall 48-hour attention profile averaged over all 28,400 prediction windows is displayed in Figure 9.1, and the positions of these anchors in the day can clearly be seen as the peaks at $t - 23h$, $t - 24h$, $t - 47h$ and $t - 48h$.

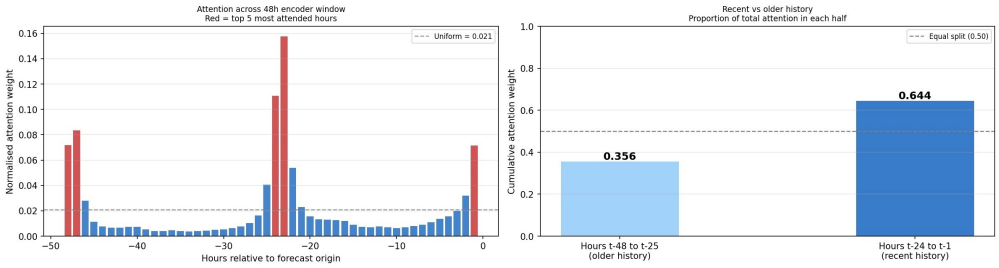


FIGURE 9.1: TFT v3 temporal attention weights across the 48-hour encoder window

9.1.6 Per Sub-Class Attention Analysis

The results from running `interpret_output()` on each road sub-class separately were remarkably consistent — all four sub-classes show a maximum at $t - 23h$, with virtually identical recent-to-older attention splits.

Table 9.5 is a summary of the attention received by each sub-class.

TABLE 9.5: Attention received by each road sub-class

Sub-class	Recent 24h	Older 24h	Peak position
URB_normal	63.4%	36.6%	$t - 23h$
URB_ramp	64.7%	35.3%	$t - 23h$
M30	64.8%	35.2%	$t - 23h$
C30	64.9%	35.1%	$t - 23h$

The gap between the biggest and the smallest fractions that have been given some recent attention is just 1.5 percentage points. The temporal attention strategy used by TFT is universal meaning that it is the same focus anchor for the day, no matter what the road type is. The road-type adaptation is done by the Variable Selection Networks (VSN), which has an important architectural implication as it is not done by the attention mechanism, because the weights of the encoder in the different sub-classes differ in a subtle way, with C30 having a weight of 0.1594 in the time index compared to 0.1341 for URB_normal. The two components are complementary: variable selection is context dependent and attention is context independent, reflecting a universal temporal structure.

For all four sub-classes, attention profiles can be seen in Figure 9.2.

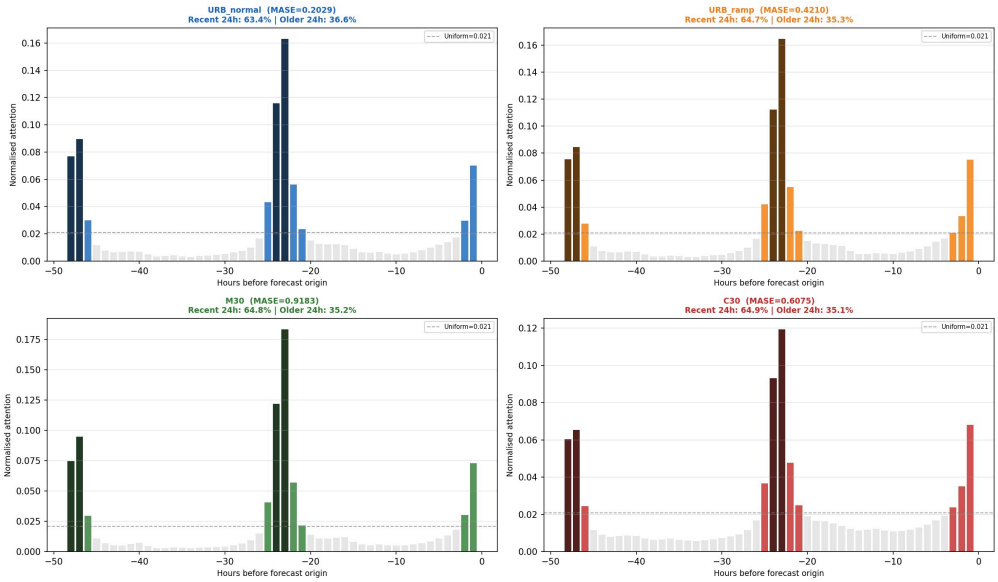


FIGURE 9.2: TFT v3 temporal attention profiles across the four road sub-classes

Figure 9.3 shows importance of each encoder variable (per sub-class).

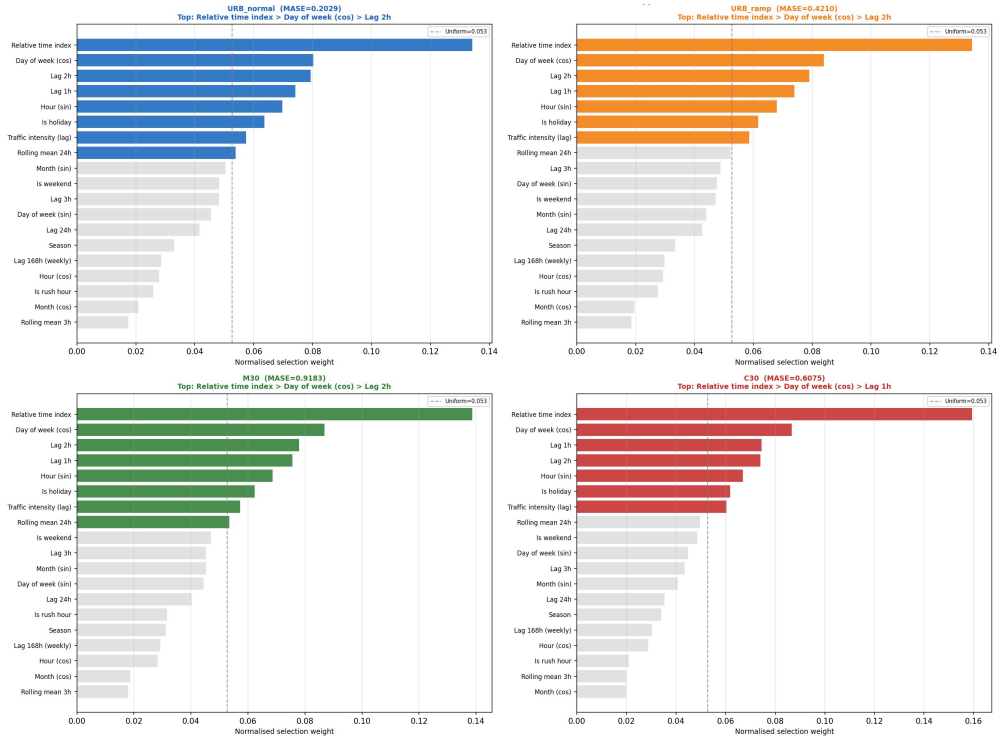


FIGURE 9.3: TFT v3 per sub-class encoder variable importance

9.2 CNN+LSTM v2 — GradientSHAP Attribution

9.2.1 Motivation for Post-Hoc Explainability

All 29 input features are passed through a pre-defined sequence of convolutional filters and recurrent layers, including CNN+LSTM. No gating mechanism to get readable importance weights, no attention head to remember which timesteps it was looking at, no Variable Selection Network to give softmax scores to each feature. To learn something about the features that led to its predictions, an external method of attribution must be used after training — a post-hoc analysis which probes from the outside, rather than reading quantities from the inside of the trained model.

Based on Joseph (2022, Chapter 18) [26], differentiable PyTorch models, we chose GradientSHAP (Lundberg and Lee, 2017) [10]. It is simply a method that calculates an attribution for each element in the input image by generating random samples in a background image, linearly interpolating between each of those samples and the element itself, computing the derivative of the output with respect to the interpolated input, and multiplying by the difference between the input element and the sample. It is repeated 50 times over each window, and the average is used to obtain stable attributions, which

are close to the Shapley values based on cooperative game theory (Lundberg and Lee, 2017) [10]. The important property of Shapley values is consistency: when a feature contributes more to the output when it is added to a larger set of features, it will have a higher contribution value. This enables the resulting importance scores to be comparable between the features.

One practical constraint requires explicit mention. The cuDNN version of LSTM layers in PyTorch only supports back-propagation for the `model.train()` mode. But in training mode dropout can be turned on to add stochasticity, and attributions are not reproducible across runs. The obvious solution was to train the model in training mode for computing the gradient, and to also set all dropout probabilities to zero — thus keeping the computational graph needed for backpropagation, but without introducing the stochasticity. If not for this change the attribution values could vary with each run potentially compromising the results of the analysis.

An extra wrapper was needed for CNN+LSTM, which has six predictions for each sample and GradientSHAP demands a single prediction for each sample. To facilitate GradientSHAP's computation of attributions, the six horizon predictions were averaged across all samples within each mini-batch. Such a suggestion is made by Joseph (2022, Chapter 18) [26] for sequence forecasting models where he suggests aggregating horizons for overall feature importance.

28,400 test windows were analysed from 100 sensors (25 sensors per road sub-class, stratified and drawn from the centre of the test period, to minimise the boundary effect). The reference distribution for baseline sampling was done using a separate background set of 28,116 windows, from a different 100 sensors. The setup of using separate sensors for the background and test sets avoids circular attribution, as this would give the importance to patterns in the data depending on the individual sensor. This sample size is the same for the TFT interpretability analysis, so the two methods are directly comparable on sample covered.

The raw GradientSHAP output from CNN+LSTM has shape (windows, 48 timesteps, 29 features), which was collapsed to (windows, 29) by taking the mean over the timesteps in the encoder, and then taking the means over the windows results in one importance score per feature.

9.2.2 Overall Feature Importance

The most prominent result is the extreme importance of traffic intensity (lag), a normalised target sequence at each timestep of the encoder, that represents 30.2% of total

attribution at 8.75 times the uniform baseline. There is no other that can match this feature. This single feature is the most recent of the available traffic values available at each historical hour, and CNN+LSTM's dependence on such a large proportion of this feature stems directly from the architecture. All 29 features are processed in the same computational path by a convolutional filter and an LSTM layer, with no way to selectively gate, suppress or amplify any given variable. The gradient-based attribution method gives the latest traffic sequence a correspondingly high weight throughout the encoder, if the most informative signal in the input space is the recent sequence of traffic.

TABLE 9.6: CNN+LSTM v2 GradientSHAP importance (all 29 features)

Rank	**Feature**	**Normalised SHAP**	**vs uniform (1/29 = 0.034)**
1	Traffic intensity (lag)	0.3018	8.75×
2	Lag 1h	0.0847	2.46×
3	Hour (cos)	0.0740	2.15×
4	Is rush hour	0.0607	1.76×
5	Lag 24h	0.0564	1.63×
6	Hour (sin)	0.0537	1.56×
7	Lag 168h (weekly)	0.0503	1.46×
8	Rolling mean 3h	0.0365	1.06×
9	Is weekend	0.0341	0.99×
10	Lag 3h	0.0322	0.93×
11	Lag 2h	0.0300	0.87×
12	Day of week (sin)	0.0213	0.62×
13	Rolling mean 24h	0.0207	0.60×
14	Road sub-class	0.0190	0.55×
15	Dist. bus station	0.0175	0.51×
16–29	(remaining features)	0.0150–0.0000	below uniform

The results of the short-term lag features (lag 1h at 0.0847, lag 24h at 0.0564, lag 168h at 0.0503) are all consistent with the autoregressive nature of the prediction strategy used by CNN+LSTM. It is interesting to note the difference in the rank of the weekly lag feature in TFT's encoder analysis (rank 7) vs CNN+LSTM (rank 15) as it relates to the different approaches that both architectures take to the weekly seasonality.The rank of the weekly lag feature is higher in TFT's encoder analysis (rank 7) than in CNN+LSTM (rank 15), and that is consistent with the different ways that they approach the weekly seasonality: in TFT, the weekly pattern is distributed across the attention and calendar features, so that the explicit weekly lag feature is less important; in CNN+LSTM, there is no alternative pattern pathway, and the weekly lag is more fundamental.

The hour cosine and hour sine were the dominating calendar signals (rank 3, 0.0740; and rank 6, 0.0537, respectively) confirming that the most important calendar signal is the time of day. Interestingly, the rank of the filter corresponding to the flag of rush hour is

also quite high, compared to the other filters in TFT's encoder selection, indicating that CNN+LSTM's fixed filters are more sensitive to this binary explicit flag — a strong and easy-to-detect signal that is not as smooth as the hour encodings.

All eight spatial and static features (road sub-class, geographic coordinates, and POI distance features) fall between ranks 14 and 26, which is below the uniform baseline. The significant difference is found when compared to TFT, which had a distance from railway station as the most important static feature with weight of 0.1344. CNN+LSTM's convolutional filters treat all 48 timesteps of the encoder equally, and a static feature (e.g., that has the same value at each timestep) does not change the gradient signal over the time dimension, making it unimportant for GradientSHAP. There is no selective amplification to other constant features like TFT's static Variable Selection Network provides.

The effectiveness of `Is_holiday` is at its lowest. Holiday effects are very uncommon in any test sample, and over the 28,400 windows analysed, very few match up to actual public holidays. Nearly the entire window flag is all zeroes, so the gradient is small. This is not a fault of the gradient-based attributions, but instead of a sampling constraint — it does not mean that holiday status is not relevant to traffic prediction. With all 29 months of training (with holidays present and generating distinct patterns), TFT is able to correctly give holiday a weight of 0.0624.

The complete ranked importance bar chart is displayed in Figure 9.4.

9.2.3 Per Sub-Class Analysis

Table 9.7 shows that the top 5 SHAP features for the GradientSHAP model, based on road sub-class, are:

Table 9.7: CNN+LSTM v2 top 5 GradientSHAP features by road sub-class

Feature	**URB_normal**	**URB_ramp**	**M30**	**C30**
Traffic intensity (lag)	0.3007	0.3035	0.3023	0.3007
Lag 1h	0.0845	0.0851	0.0847	0.0847
Hour (cos)	0.0736	0.0766	0.0713	0.0745
Is rush hour	0.0604	0.0623	0.0594	0.0610
Lag 24h	0.0554	0.0552	0.0573	0.0575

The per sub-class analysis yields a very interesting result: feature importance ranks are almost exactly the same for all the four road types. The intensity of traffic (lag) represents between 30.1% and 30.4% of the attribution in each sub-class and only differs by 0.003 between the four classes. The top 5 for the ranking are the same for each of

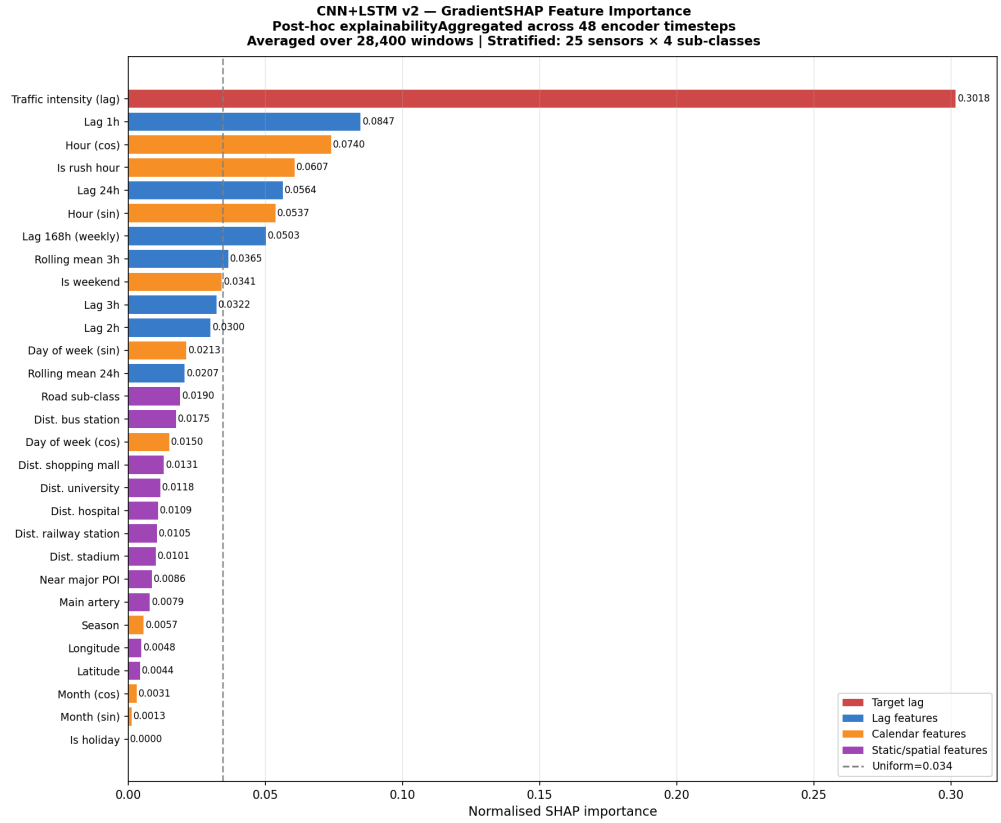


FIGURE 9.4: CNN+LSTM v2 overall GradientSHAP feature importance

the following: URB_normal, URB_ramp, M30, and C30. The uniformity expected to be the architectural consequence is that CNN+LSTM will learn a single set of weights that is applied uniformly to all the sensors and all of the contexts. The model can't learn that the features for road sensors are more relevant to the motorway than the residential ones as there is no gating mechanism that changes feature weighting based on context.

In contrast, TFT reported a relatively small difference in weights for the different sub-classes of the encoder, with the weight for the C30 sub-class (0.1594) being significantly higher than for the URB_normal sub-class (0.1341), demonstrating that the model is capable of learning the positional weight for each sub-class in response to the irregular closure patterns of the tunnel sensors. This is a distinction CNN+LSTM wouldn't be able to make.

Figure 9.5 shows the per sub-class importance plots.

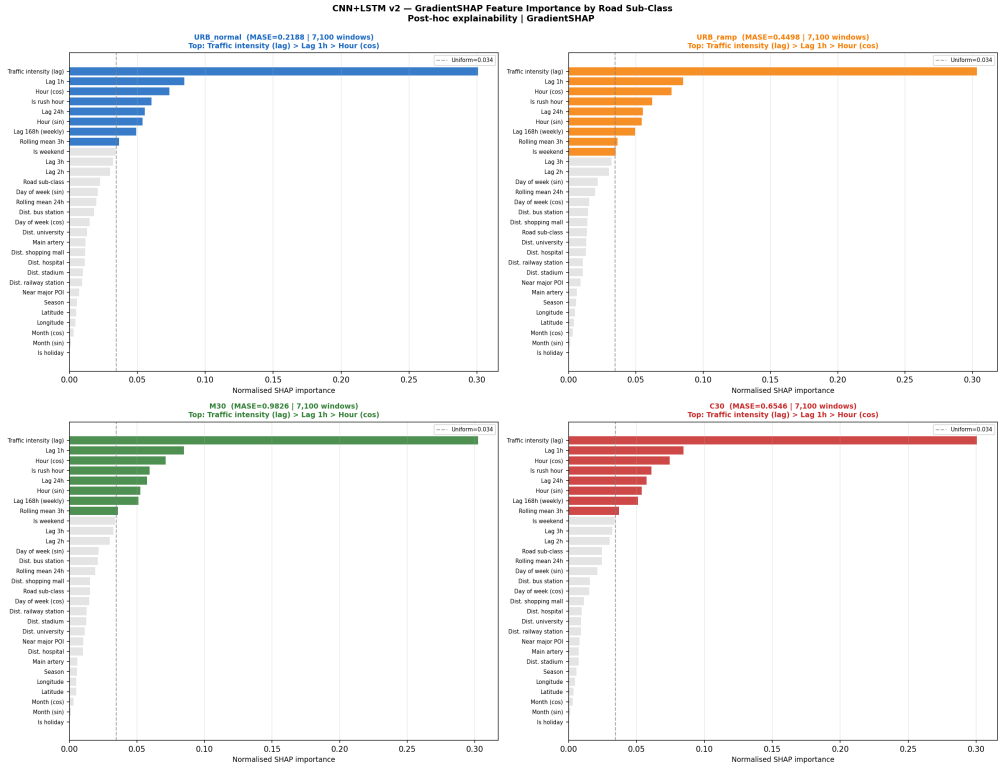


FIGURE 9.5: CNN+LSTM v2 GradientSHAP feature importance by road sub-class

9.3 Cross-Model Feature Importance Comparison

After ranking the importance of every model, the next interesting question is whether the two models, TFT and CNN+LSTM, agree on the features that are important. Care must be taken with the comparison, as the two models do not perform all 29 features in the same way. At each timestep of the encoder, CNN+LSTM is fed with all 29 features, as a flat input matrix, with static spatial features (road sub-class, coordinates, POI proximity distances) and time-varying features being the same. In contrast, TFT separates static features from time-varying features architecturally: Static covariates are broadcast to all timesteps of encoder and decoder in a dedicated static VSN, and time-varying features are fed to the encoder VSN at each timestep. The 11 static spatial features are not directly compared with the numbers due to their different spatial locations in the two architectures. In TFT they go into a special static VSN and are played as a context vector, with the distance to railway station being number 1 (0.1344) and road sub-class number 2 (0.1300) as seen in Table 9.1. In CNN+LSTM, they feed the encoder with the same feature value for each timestep in the sequence — a constant feature has little variation across the sequence, and GradientSHAP assigns all 11 of the static features below-uniform importance (ranks 14–26, Section 9.2.2). It would

be mistaken to directly compare the static weights of TFT with the gradient-based attribution for the same features that are derived from CNN+LSTM. Spatial context is therefore discussed qualitatively: while TFT's architecture is capable of learning that it's the most important fixed property of a sensor, CNN+LSTM's architecture is structurally incapable of reaching the same conclusion not because it cannot learn anything about spatial context, but because the inputs to the CNN+LSTM are constant-valued and the gradient variation needed for post-hoc attribution methods is not achievable.

Shared 18-feature comparison involves the TFT's encoder variable selection weights and CNN+LSTM's GradientSHAP values, which were both normalised to sum=1.0 for the common 18 features. Agreement is defined as a CNN-to-TFT importance ratio from 0.5 to 2.0 (inclusive), allowing for a more meaningful variation without letting one model weigh a feature several times more. In this table, the importance values of the TFT v3 and CNN+LSTM v2 models, using 18 shared encoder features, are compared.

Table 9.8: TFT v3 vs CNN+LSTM v2 importance comparison (18 shared encoder features)

Feature	**TFT weight**	**CNN SHAP**	**Ratio CNN/TFT**	**Agreement**
Lag 1h	0.0868	0.0961	1.11	✓ agree
Lag 3h	0.0541	0.0365	0.67	✓ agree
Lag 24h	0.0465	0.0639	1.37	✓ agree
Lag 168h (weekly)	0.0343	0.0570	1.66	✓ agree
Rolling mean 3h	0.0215	0.0414	1.93	✓ agree
Hour (sin)	0.0796	0.0609	0.77	✓ agree
Is weekend	0.0557	0.0387	0.70	✓ agree
Traffic intensity (lag)	0.0680	0.3424	5.03	× differ
Lag 2h	0.0904	0.0341	0.38	× differ
Rolling mean 24h	0.0609	0.0235	0.38	× differ
Hour (cos)	0.0333	0.0839	2.52	× differ
Day of week (cos)	0.0983	0.0171	0.17	× differ
Day of week (sin)	0.0531	0.0241	0.45	× differ
Month (sin)	0.0525	0.0015	0.03	× differ
Month (cos)	0.0230	0.0035	0.15	× differ
Is_holiday	0.0726	0.0000	0.00	× differ
Is_rush hour	0.0308	0.0689	2.23	× differ
Season	0.0383	0.0065	0.17	× differ

Seven of 18 shared features agree and 11 disagree. The seven agreements mentioned above (lag 1h, lag 3h, lag 24h, lag 168h, rolling mean 3h, hour (sin) and is weekend) are the main predictive signal which any well-trained architecture that is applied to this data will learn, no matter what internal structure it possesses. Architecture independent short-term autocorrelation, daily periodicity, weekly periodicity and weekend regime identification are powerful features. The above-uniform importance of the features is

strengthened when both approaches—one based on the intrinsic gating weights of TFT and the other based on the gradient-based post-hoc attributions of CNN+LSTM—as highly different methods agree on the importance of those features.

Even more interesting to scientists is the list of 11 disagreements, which shows the different aspects of the feature set that each architecture takes advantage of — or neglects to take advantage of. The greatest disagreement is on traffic intensity (lag), which CNN+LSTM weights 5.03 times higher than TFT. As per Section 9.1, the autoregressive component is captured by the attention mechanism of TFT that focuses directly on the same-hour positions of the encoder sequence (the receiving weight for $t - 23h$ is 0.1576, and the receiving weight for $t - 24h$ is 0.1108). This partially reduces the need for the explicit target sequence feature in TFT's computation; the model has an explicit path to the past traffic values via attention without the pre-computed lag feature. CNN+LSTM is a network without any attention mechanism and is only able to use the raw target sequence. This difference accounts for a portion of the overall MASE gap between the two models and is due to the fact that the autoregressive signal can be extracted more efficiently from the temporal architecture, which allows temporal features to occupy more capacity than CNN+LSTM.

Day-of-week cosine is weighted almost six times more by TFT than CNN+LSTM (ratio 0.17). This cyclical weekly encoding was identified by TFT's Variable Selection Network as the second most important encoder feature, similar to that found by Vélez-Serrano et al. (2021) [9] who showed that weekly autocorrelation is the most prevalent pattern in the traffic data of Madrid using Kruskal-Wallis testing. CNN+LSTM uses fixed convolutional filters for day-of-week cosine at each encoder timestep, where there is no mechanism to selectively amplify the contribution of the cosine, and the resulting gradient signal is weak. That TFT learns to take advantage of this property while CNN+LSTM does not is a significant architectural result, because both have the same information, but only the former can do something useful with it.

Is_holiday the biggest disagreement — TFT gives it a weight of 0.0726, while CNN+LSTM gives an essentially zero weight. This is partly because public holidays occur rarely on any test sample and because of a sampling constraint of the gradient-based methods that the gradient in a binary flag with almost no active window is small. Trained on the entire 29 months of train data including holidays, which create unique traffic patterns, TFT's VSN correctly learns what the holiday is, without being "chained" to the gradient signal during evaluation. In this case, the post-hoc approach has a true methodological disadvantage over the intrinsic approach if the categorical features are rare.

On the other hand, Lag 2h is in opposition — TFT weights it 2.6 times less compared to CNN+LSTM (ratio 0.38). Lag 2h is the third most important feature of the TFT

encoder (0.0776), implying that the VSN of the TFT recognized a specific predictive function of lag 2h other than lag 1h and 3h. The relation between rolling mean 24h and the gradient attribution over CNN+LSTM is similar (ratio 0.38), while the relation between TFT's VSN and the gradient attribution over CNN+LSTM is not similar (ratio 0.77).

CNN+LSTM assigns near zero importance (weights 0.03 and 0.15 for month encodings and season, respectively) to these, while TFT gives moderate importance (combined weight ∼0.128). In CNN+LSTM, the fixed architecture is not able to extract the signals of slow seasonal drift, while TFT's gating mechanism can learn to select the weak signals and give them higher weights. This is in line with the results observed per horizon in Chapter 8, where TFT continues to be accurate at longer horizons where the seasonal calendar context is more important, but CNN+LSTM degrades more rapidly, in line with its weaker exploitation of the features that help it to be accurate between $t + 4h$ and $t + 6h$. Rush hour, on the other hand, shows the opposite trend, with CNN+LSTM weights being 2.23 times higher. CNN+LSTM's convolutional filters are very sensitive to the explicit binary rush hour flag, which signals the on/off condition very powerfully. During encoding, TFT uses less of the binary input, distributing this info over the hour (sin) and hour (cos) encoder features, and the `is_rush_hour` decoder flag. It is because of this architectural difference that CNN+LSTM's peak period performance is closer to TFT than its overall MASE gap would lead one to believe, being one of the few CNN+LSTM exploits as well as TFT.

The importance comparison by side is given in Figure 9.6.

9.4 Summary

The first finding is that TFT automatically discovers a 24-hour traffic pattern for all four road sub-classes. The highest attention is placed on the day before on the same traffic position ($t - 23$h, weight 0.1576), and there are also peaks at $t - 24$h and $t - 47$h as well as $t - 48$h. It is worth noting that 64.4% of the total attention is contained in the last 24 hours and 35.6% of the total attention comes from the older half of the 48-hour input, indicating that both recent past and older past contribute.

The second finding is that road-type adaptation is primarily carried out by the Variable Selection Networks and not attention. The relative time index is more relevant for C30 tunnel sensors (0.1594) compared to URB_normal sensors (0.1341) for which the spatial position is less uniformly distributed. Spatial differences are also evident in the static

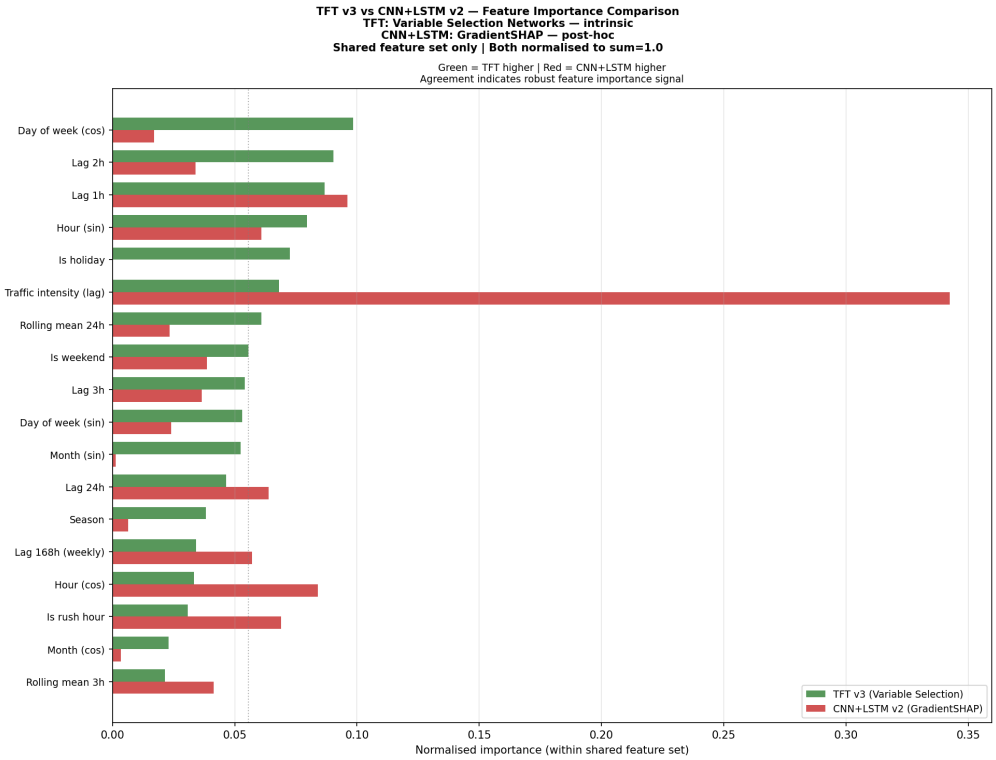


FIGURE 9.6: TFT v3 and CNN+LSTM v2 feature importance comparison for the 18 shared encoder features

features as represented by the high weights of railway-station distance (0.1344) and road subclass (0.1300).

The third insight is that CNN+LSTM heavily depends on traffic-intensity lags (30.2% attributed). The signed analysis reveals that these features do the same thing they are supposed to do: higher traffic values with the busiest periods result in higher predictions and vice versa, lower traffic values with the quietest periods result in lower predictions. The sign changes between the two regimes are also as expected for all six features analysed.

Finally, there are 7 common features, namely `lag_1h`, `lag_3h`, `lag_24h`, `lag_168h`, `rolling_mean_3h`, hour (sin), and weekend. They are used as a common backbone for six-horizon forecasting. The rest of the model-specific features revealed that TFT can also leverage slower seasonal patterns, static spatial features and temporal attention, whereas CNN+LSTM relies more directly on the time varying lag features. The differences provide a motivation for future research on enhancing the use of spatial features.

Chapter 10

Conclusion

10.1 Overview of the Study

The traffic forecasting problem is a combination of data engineering, machine learning and operational systems design problem in large urban road networks. These decisions are consequential, as their decisions rely on forecasts—like signal timing, speed limit activation, incident response, routing—will have direct impacts on the millions of people who rely on those roads on a daily basis. Miss the target by repeating the same mistakes in a predictable manner is just as useful as hitting it, since it will reveal practitioners where the boundaries of existing solutions exist and where new solutions are required.

This thesis aimed at this double objective: precise predictions and honest failure mode characterisation, taking place over a period of three and a half years of real traffic sensor data on the Madrid road network. The study started by creating a common analytical dataset from three open public datasets: 135 million hourly observations from 4,971 sensors; weather data from the municipal weather station network; and road infrastructure attributes derived from OpenStreetMap. To complement the administrative classification of the road network in the portal, a classification of the 145 urban sensors was created in four classes, which distinguish between the 4 classes of the urban road network, including `URB_ramp`, whose physicality is separated from the other classes. The data were prepared for deep learning training through a systematic process of data quality assessment, a tiered missing value imputation with a cap of 30% fabricated data, and a feature engineering pipeline resulting in 28 external covariates.

Then, three structurally different models were trained on the same sensor stratified sample (50 sensors per road sub-class), normalised sensor data, identical data splits, and the same six-hour prediction task ($t + 1$h to $t + 6$h). We used CNN+LSTM, which

is the baseline multivariate convolutional-recurrent model. N-BEATS gave a purely univariate interpretable counterpoint. The Temporal Fusion Transformer was the most architecturally complex, which included learned variable selection, LSTM encoding and interpretable multi-head attention. The same seasonal naive baseline was used for all three, and the evaluation of the three models was performed using the MAE, RMSE, MASE, SMAPE and R^2 in physical units following inverse transformation.

The experimental program ended with an explainability analysis, applying to each model the most suitable explainability method: the built-in Variable Selection Networks and multi-head attention for TFT; and GradientSHAP applied post-hoc to CNN+LSTM. The two analyses were then compared directly on the 18 time-varying features that were common to both models, which gave a cross-architecture perspective of which features are strongly relevant and which are used differently by the different models as they process them.

10.2 Principal Findings

The most striking overall results are that all three deep learning models significantly outperform the seasonal naive baseline for all six forecast horizons and all four road sub-classes — except for one case. The test MASE for TFT v3 was 0.5377, CNN+LSTM v2 was 0.5767 and N-BEATS v7 was 0.6133, which improved over the naive baseline of 46.2%, 42.3%, and 38.7% respectively. The order of the three models remains the same for all sub-classes and all time periods tested, and corresponds exactly to the added capability of each step: multivariate inputs, context-aware feature selection, and temporal attention, and the accuracy gain from each step.

The performance difference between N-BEATS and CNN+LSTM is overall 0.037 MASE, representing a 6.0% reduction in scaled error, which allows to separate the effect of 28 external features from the univariate sequence. Combining calendar encodings, lag features, and spatial covariates reduces prediction error significantly, and this benefit is most apparent at longer horizons where the autocorrelation signal from the calendar signal diminishes, and the signal from calendar context in the future is relatively more important. The MASE spread is 0.199 for N-BEATS, whereas it is 0.132 for CNN+LSTM and 0.130 for TFT, demonstrating that knowledge regarding the hour and day when the forecasted hours fall on is beneficial to the forecast accuracy.

The performance difference between CNN+LSTM and TFT, again an approximately 0.039 MASE, representing a 6.8% further reduction in scaled error, separates the contribution of the context-aware architecture from fixed processing of the same set of features.

TFTs Variable Selection Networks learned to switch the weights of features that are not relevant to a given prediction task, e.g. lag 168h was assigned below-uniform encoder importance because TFT captures weekly seasonality through attention and day-of-week encodings, while increasing the weights to features that carry the most relevant signal. The explainability analysis shows that TFT, without explicit instruction, pays attention to the state of the previous day at the same hour ($t - 23$h with a weight of 0.1576), which creates the daily periodicity of Madrid's traffic. This isn't just a curiosity as the architecture is learning that traffic is actually structured in that way, not just shaped to noise.

The most practically significant finding in the study is the results of the M30 motorway. The results of all three models do not meaningfully outperform the seasonal naive baseline on the motorway sensors, with MASE values of 1.0774 (below naive), 0.9826 and 0.9183 respectively for MASE using MASE with CNN+LSTM and TFT. CNN+LSTM and TFT do make small improvements over naive on M30, but these improvements seem to be due to the spatial context features, including road sub-class encoding and proximity to POIs; the temporal modelling makes no discernable contribution. The lack of improvement on all three architectures indicates a limit in the information that can be delivered by a single sensor, when the traffic variation is caused mostly by the propagation of incidents on the network. The lack of spatial information on what is going on in neighbouring parts of the motorway cannot be offset by architectural sophistication.

The highest values of MASE were recorded for the morning peak in all the architectures with the TFT model achieving a score of 0.6867, CNN+LSTM of 0.7367, and N-BEATS of 0.7843 — significantly higher than the same peak in the evening, and approximately 50% higher than the same peak in the weekend. The morning peak is physically meaningful and structurally asymmetric, with more prominent peaks and more random variation in the time when the peak occurs, and more effect from calendar influences (school terms, pre-holiday moves, weather), which the feature set only partially represents. Traffic is already built up by the afternoon, resulting in more gradual peaks in the evening, allowing the encoder to see and understand the developing traffic situation before prediction is needed.

The explainability analysis showed that the adaptation of the road type in TFT does not happen by attending to a specific image but by selecting the appropriate one. The attention mechanism is universal, with the same $t - 23$h peak for all 4 sub-classes and similar splits of attention for recent-older elements. The attention mechanism is universal; all 4 sub-classes have nearly identical splits of recent-older attention, but variable weights differ somewhat by sub-class in the encoder. The sensors used in the C30 problem were more sensitive to the relative temporal position (0.1594 vs 0.1341 for `URB_normal`), which

is due to the irregular closing pattern of underground objects. CNN+LSTM found that the architecture gives less than uniform importance to all 11 spatial features, irrespective of their type, as the inputs of constant values over the 48-hour window in the encoder results in negligible variation in the gradient. Distance to railway station (0.1344 the single most important static feature, as identified by TFT) is almost imperceptible to CNN+LSTM — not that it is unimportant but because the architecture cannot make use of it as a static input pathway can.

The cross-model feature comparison shows that seven out of 18 time varying features match, between the intrinsic weights of TFT and the GradientSHAP values of CNN+LSTM: lag 1h, lag 3h, lag 24h, lag 168h, rolling mean 3h, hour (sin), is weekend. These are the powerful predictive backbone of six-horizon traffic forecasts in Madrid and its structural difference validated by two independent methods. The methods disagree on 11 features, all of which are crucial to exploiting for the model: traffic intensity lag, day-of-week encodings, holiday status, and seasonal features.

10.3 Limitations of the Study

It is important to note that the results presented above have certain limitations that should be taken into account when interpreting the results.

The simplest condition is that all three models use only one time series for their sensors. They are given the past traffic information of one sensor and assume that the sensor will record the same traffic in the future, without any direct information about what is happening at neighbouring sensors at the same time. This constraint can be handled for urban residential sensors, where traffic changes significantly without the influence of traffic two streets away. If the modelling of the sensors is based on a single sensor, it becomes structurally inadequate for M30 because congestion can propagate along the ring road and a single incident could affect the flow over several kilometres. The empirical evidence, rather than any modelling failures, is what the M30 results are, with all three models near or below the naive baseline.

The 200-sensor stratified sample represents a small percentage of the 4,886 active sensors in the Madrid network. The criterion applied for selection was not geographic or functional representativeness, but the completeness of the training records, and this led to systematic under-representation of some sensors, in particular those belonging to the C30 class, and those in some areas of urban regions. The results might not be applicable in the same way to sensors not in the sample, especially those whose data had been

erratically collected over time such that the imputation pipeline could not reconstruct all of the data.

The test period (December 2025 to May 2026) was chosen temporal integrated, that is, there is no gap from the validation period and it is long enough to span multiple seasons. It does not, however, include the Christmas/New Year holiday period or several Spanish national holidays or spring events that give rise to traffic patterns not well-represented in any previous period. This structural challenge is reflected in the higher seasonal naive MAE during this time period (139.20 veh/h, higher than 105.63 veh/h during validation), and should be taken into consideration when comparing the test MASE values with results from other studies with different evaluation periods.

The explainability analysis of CNN+LSTM is based on GradientSHAP, a post-hoc approximation, not a window into the computation of the model. The attribution values will necessarily have some approximation error to the true Shapley values they estimate based on the choice of background distribution, the number of noise samples, and the method for interpolation. It's important to note that the attribution for `is_holiday` is near zero for gradient-based methods because it's a rare categorical feature, not that holiday status is unimportant. This is not one of the limitations of intrinsic methods, like the ones used in TFT's Variable Selection Networks, which are why the two methods differ greatly on this feature.

TFT v3 was trained using a point forecast loss (or MAE), and therefore forecasts only one value per horizon, but not a prediction interval. It is noteworthy that TFT's architecture is fully capable of quantile forecasting using the QuantileLoss configuration in PyTorch Forecasting, and in such cases, will provide calibrated uncertainty estimates along with the point forecast, which is directly beneficial for operational traffic management decisions where over- and under-prediction have differing costs. The point forecast configuration was selected in order to have the same comparability as the CNN+LSTM and the N-BEATS, which are inherently point forecast models. We did not implement this because a quantile-configured TFT cannot be fairly compared to point forecast models on MASE.

10.4 Directions for Future Research

The M30 finding is most directly pointing to the next thing to come. Spatial graph neural networks, which model the road network as a graph with sensors as nodes and road connections as links, and which enable information to flow between neighbouring sensors, have been found to consistently outperform temporal-only models on motorway

networks. A graph based architecture would be appropriate for the M30 ring road because the information is topologically structured in space, and the effects of incidents is known in advance, traveling upstream. The Madrid network is an adequate network for this extension: the topology of the roads is available from openstreetmap, the id of the sensors is stable and the adjacency of the M30 can be defined from the existing infrastructure data used in this thesis.

An obvious extension of the TFT is to use a quantile loss rather than a point loss, which would result in prediction intervals for the six horizons. This would enable the model to share not only a prediction but a level of confidence — a valuable capability to provide during morning peak times when the level of prediction uncertainty is highest. The quantile extension does not require a change in architecture, and would not incur much training overhead. The post hoc calibration of TFT's prediction intervals with respect to the calibration of conformal ones would give a more complete understanding of uncertainty quantification methods for this network.

The explainability analysis showed that CNN+LSTM fails to make good use of the static spatial features due to the lack of gradient variance in the inputs to the CNN for constant-valued encoders. One architectural approach is to separate the static and dynamic input path in the CNN+LSTM, as TFT does with the static VSN, while maintaining the computational advantages of CNN+LSTM over TFT. Notably, a combination of static embedding layer for spatial covariates and a convolutional-recurrent temporal pathway would investigate the performance difference between CNN+LSTM and TFT without the complexity of training TFT itself.

In this study we used a sample of 200 sensors, which was limited by the capacity of the GPU memory. The NVIDIA RTX A2000 with 12 GB of VRAM could only train about 200 sensors simultaneously at 128-window batches. The more recent GPU hardware with 24–80 GB VRAM would enable the entire 4,886-sensor network to be trained as one global model which would reveal spatial patterns not seen in the stratified sample and validate the performance ranks of the sub-class models. The training with the entire sensor network would also give a more realistic estimation of the C30, as the three problematic tunnel locations identified in Chapter 3 may not be representative of the geographic variability of the tunnel infrastructure underground.

Lastly, the feature importance comparison done in Chapter 9 revealed 7 features that are important according to both the TFT and CNN+LSTM, which are: lag 1h, lag 24h, lag 168h, rolling mean 3h, hour (sin), `is_weekend`; and 11 features that are important by one model and not the other. A natural follow up is to test ablation models, i.e. models of each architecture trained on subsets of the features, to check whether the features the explainability methods flag as important are really causally responsible

for the different performance or if the methods are picking up patterns in the gradient landscape. These types of ablation experiments would reinforce the causal interpretation of the explainability findings and give more direct insights into the design of the feature set in future traffic forecasting deployments on this network.

Bibliography

[1] Julian Moral-Carcedo. Dissuasive effect of low emission zones on traffic: the case of madrid central. *Transportation*, 2024. doi: 10.1007/s11116-022-10318-4.

[2] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[3] R. Asadi and A. Regan. A spatial-temporal decomposition based deep neural network for time series forecasting. *arXiv preprint arXiv:1902.00636*, 2020. doi: 10.48550/arXiv.1902.00636.

[4] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017. doi: 10.48550/arXiv.1706.03762.

[5] Bryan Lim, Sercan Ö. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021. doi: 10.1016/j.ijforecast.2021.03.012.

[6] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. In *International Conference on Learning Representations (ICLR)*, 2018.

[7] Ling Zhao, Yujiao Song, Chao Zhang, Yu Liu, Pu Wang, Tao Lin, Min Deng, and Haifeng Li. Temporal graph convolutional network for urban traffic flow prediction method. In *Proceedings of the 2019 IEEE International Conference on Data Mining (ICDM)*, 2019. doi: 10.48550/arXiv.1811.05320.

[8] Vikas Chauhan, Aruna Tiwari, and Arun Kumar. An attention mechanism-based hybrid TimeAttentionBiLSTM architecture for long-term traffic forecasting. *The Journal of Supercomputing*, 2025. doi: 10.1007/s11227-025-07747-0.

[9] D. Vélez-Serrano, A. Álvaro-Meca, F. Sebastián-Huerta, and J. Vélez-Serrano. Spatio-temporal traffic flow prediction in madrid: An application of residual convolutional neural networks. *Mathematics*, 2021. doi: 10.3390/math9091068.

[10] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30*, pages 4765–4774, 2017. doi: 10.48550/arXiv.1705.07874.

[11] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser, Adrien Bennetot, Siham Tabik, Alberto Barbado, Salvador Garcia, Sergio Gil-Lopez, Daniel Molina, Richard Benjamins, Raja Chatila, and Francisco Herrera. Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. *Information Fusion*, 58:82–115, 2020. doi: 10.1016/j.inffus.2019.12.012.

[12] C. F. Daganzo. The cell transmission model: A dynamic representation of highway traffic consistent with the hydrodynamic theory. *Transportation Research Part B: Methodological*, 28(4), 1994. doi: 10.1016/0191-2615(94)90002-7.

[13] Chun-Hsin Wu, Jan-Ming Ho, and D. T. Lee. Travel-time prediction with support vector regression. *IEEE Transactions on Intelligent Transportation Systems*, 5(4): 276–281, 2004. doi: 10.1109/TITS.2004.837813.

[14] Shaojiang Deng, Shuyuan Jia, and Jing Chen. Exploring spatial–temporal relations via deep convolutional neural networks for traffic flow prediction with incomplete data. *Applied Soft Computing*, 78:712–721, 2019. doi: 10.1016/j.asoc.2018.09.040.

[15] Yu Gu, Zherui Xu, and Zhenzhe Yang. Modeling and analysis of key temporal features for traffic flow prediction based on long short-term memory networks. In *Proceedings of the ACM*, 2025. doi: 10.1145/3796315.3796322.

[16] Zhumei Wang, Xing Su, and Zhiming Ding. Long-term traffic prediction based on lstm encoder-decoder architecture. *IEEE Transactions on Intelligent Transportation Systems*, 22(10):6561–6571, 2021. doi: 10.1109/TITS.2020.2995546.

[17] K. Priya, K. Omkar, R. Naresh, and K. Venkatesh. Spatio-temporal traffic prediction in urban area using convolutional temporal network system. In *2025 6th International Conference on Mobile Computing and Sustainable Informatics (ICMCSI)*, 2025. doi: 10.1109/ICMCSI64620.2025.10883158.

[18] I. Gómez and S. Ilarri. Advanced prediction of traffic at different temporal scales using heterogeneous data sources. *IEEE Open Journal of Intelligent Transportation Systems*, 2025. doi: 10.1109/OJITS.2025.3637305.

[19] G. Boeing. OSMNX: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, 2017. doi: 10.1016/j.compenvurbsys.2017.05.004.

[20] European Petroleum Survey Group. EPSG geodetic parameter registry: EPSG:25830 – ETRS89 / UTM zone 30n. Online, 2003. URL https://epsg.io/25830. Accessed: Sep. 3, 2026.

[21] OpenStreetMap Foundation. Openstreetmap wiki: Key: highway. Online, 2024. URL https://wiki.openstreetmap.org/wiki/Key:highway. Accessed: Sep. 3, 2026.

[22] J.-P. Rodrigue. *The Geography of Transport Systems*. Routledge, 5th edition, 2020. doi: 10.4324/9780429346323.

[23] R. B. Cervero and K. M. Kockelman. Travel demand and the 3ds: Density, diversity, and design. *Transportation Research Part D: Transport and Environment*, 2(3), 1997. doi: 10.1016/S1361-9209(97)00009-6.

[24] R. Cervero. Land market impacts of urban rail transit and joint development: An empirical study of rail transit in washington, d.c. and atlanta. 1992. URL https://api.semanticscholar.org/CorpusID:129414217.

[25] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 3rd edition, 2021. URL https://otexts.com/fpp3/.

[26] M. Joseph. *Modern Time Series Forecasting with Python*. Packt Publishing, 2022.

[27] Mark Raasveldt and Hannes Mühleisen. Duckdb: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. Association for Computing Machinery, 2019. doi: 10.1145/3299869.3320212.

[28] Shawn Turner, Rich Margiotta, and Tim Lomax. Monitoring urban freeways in 2003: Current conditions and trends from archived operations data. Technical Report FHWA-HOP-05-018, Texas Transportation Institute and Cambridge Systematics, 2004. URL https://rosap.ntl.bts.gov/view/dot/39877.

[29] Guillem Boquet, Antoni Morell, Javier Serrano, and Jose Lopez Vicario. A variational autoencoder solution for road traffic forecasting systems: Missing data imputation, dimension reduction, model selection and anomaly detection. *Transportation Research Part C: Emerging Technologies*, 115:102622, 2020. doi: 10.1016/j.trc.2020.102622.

[30] S. Gopal Krishna Patro and Dr. Kishore Kumar Sahu. Normalization: A preprocessing stage. *International Advanced Research Journal in Science, Engineering and Technology*, 2(3), 2015. doi: 10.17148/IARJSET.2015.2305.

[31] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456, 2015. doi: 10.48550/arXiv.1502.03167.

[32] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. *arXiv preprint arXiv:1211.5063*, 2013. doi: 10.48550/arXiv.1211.5063.

[33] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. N-beats: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations (ICLR)*, 2020. doi: 10.48550/arXiv.1905.10437. URL https://arxiv.org/abs/1905.10437.

[34] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. doi: 10.48550/arXiv.1412.6980. URL https://arxiv.org/abs/1412.6980.